Claude Code Skills Tutor

Build an agent with many skills that run Python code, and call Claude Code from Python instead of the model's API

For the Claude Code CLI on Windows, paid by a Claude Pro subscription.

Verified with Claude Code 2.1.301 and claude-agent-sdk 0.2.168, 15 October 2026.

Project folder: E:\may10\claudecodecli_and_python

Printing: designed for black-and-white, double-sided A4. No colors or shaded areas are used.

Contents

Start here: what this course gives you 5
What you will be able to do 5
What surprises most people 6
What you get in the project folder 7
How to read this book 7

Introduction 8
Words used in this course 8
Lessons 9
The 9 skills of the example agent 10
Quick start (20 minutes) 11
Folder map 14
SKILL.md template 15

Lesson 0 - Setup: Windows, Claude Code CLI, Pro subscription 16
Install Claude Code CLI and log in (only once) 16
The idea in one paragraph 16
Start here (every time) 16
What happens if you forget to activate 18
1. Check that your Pro subscription is used (not an API key) 18
2. What uses your Pro allowance 19
3. Why you see dollar amounts 19
4. Make your Pro allowance last 20
5. Your own Python programs and Pro 22
6. Windows notes 22
7. See what Claude thinks and does (Ctrl+O) 22

Lesson 1 - What a skill is 28
The idea in one paragraph 28
Step 1 - See which skills this project has 28
Step 2 - Look inside one skill 28
Step 3 - Read the SKILL.md file 29
Step 4 - Run the script yourself, as Claude would 30
Step 5 - See the three loading levels as numbers 30
Step 6 - Watch Claude use the skill 31
Step 7 - Where else skills can live 32
Step 8 - A skill or something else? 32

Lesson 2 - How Claude picks the right skill 33
How the choice works, step by step 33
Step 1 - See exactly what Claude sees before it chooses 33
Step 2 - Watch Claude choose 34
Step 3 - Try the free word-counting preview (and see its limits) 34
Step 4 - Find mistakes in descriptions with the linter 35
Step 5 - How to write a good description 36
Step 6 - A skill that only you can start 37
Step 7 - If the wrong skill (or none) is chosen 38

Lesson 3 - Skills that run Python code 39
How a skill runs your Python, step by step 39
Part 1 - The basic idea in three steps 39
Part 2 - Five patterns, each one tried out 41
Part 3 - Bonus: a command that runs before Claude reads the skill 52
Summary: which pattern to use 53

Lesson 4 - Designing an agent with many skills 54
The idea in one paragraph 54
Step 1 - List all skill names 54
Step 2 - Give every skill one job, with its own words 54
Step 3 - Keep the menu small 55
Step 4 - Share Python code between skills 56
Step 5 - Big reading jobs go to a subagent 57
Step 6 - Rules that must never be skipped 57

Step 7 - Use your skills in more places 57
Checklist before you add a skill 58

Lesson 5 - Testing skills that run Python 59
Why test at all? 59
Test 1 - Unit tests: does the Python code compute correctly? 59
Test 2 - The linter: are the SKILL.md files correct? 63
Test 3 - The preview: a free, rough guess of the choice 64
Test 4 - The routing test: what does Claude really choose? 64
What to do when a routing question fails: a real example 66

Lesson 6 - Your own agent program with the Claude Agent SDK 67
The idea in one paragraph 67
The big idea: Claude Code instead of a direct model call 67
Why go through Claude Code at all: the harness around the model 69
When the harness is the right choice, and when it isn't 70
Using Claude Code only as the model: switching the harness off 75
Three questions: when, how, and how cheaply Claude Code beats a plain model call 78
Step 1 - Check that the Agent SDK is installed 81
Part 0 - What the SDK does when your program asks Claude 82
Part 1 - An agent program that uses the skills 83
Part 2 - Your own Python functions as tools (no skills) 86
Part 3 - Ask Claude from Python with only Pro (no API key) 87
Part 4 - `claude -p` or the SDK: the same jobs both ways 89
Part 5 - Reasoning effort: how much Claude thinks 98
Part 6 - Control the effort of each skill, with the same model 102
Beyond your own computer (optional) 109

Lesson 7 - Skills that need pip packages (an isolated venv) 111
The idea in one paragraph 111
Step 1 - See that pandas is only in the venv 111
Step 2 - See which packages a skill needs 111
Step 3 - Install the packages (only once) 112
Step 4 - Use the Excel script yourself 112
Step 5 - Draw a chart and look at it 113
Step 6 - The skill's text is as simple as any other 113
Step 7 - See what happens without the venv 114
Step 8 - Use the skills in Claude 114
Step 9 - Run the tests both ways 115
Step 10 - The same skills from a program 115
Another way, not used here: write the venv's path into the skill 116
Try it yourself: add a package 116

Lesson 8 - Messy input: Claude reads, Python computes 117
The idea in one paragraph 117
Step 1 - Install the two example skills 117
Step 2 - Run the strict script yourself 117
Step 3 - Read the SKILL.md: how it teaches the translation 118
Step 4 - Messy input, started with `/calculating-math` 119
Step 5 - Messy input, and Claude chooses the skill 119
Step 6 - `$ARGUMENTS`, `$0`, `$1`: how arguments reach a skill 120
Step 7 - Clean up 121
Summary 121

Lesson 9 - Exercises 122
1. Watch routing happen (5 min) 122
2. Break a description, then fix it (10 min) 122
3. Create a collision on purpose (15 min) 123
4. Add a new skill with a Python function library (30 min) 124
5. Make a skill file-aware (10 min) 126
6. Run a big reading job in a subagent (15 min) 127
7. Specialized agents from the same skills (15 min) 127
8. A rule that can't be skipped: a hook (stretch) 128

What you have learned 130

Your achievements 130
The parts of an agent harness, and your ready solution for each 130
Copy-paste solutions for your work 131
`claude -p` or the SDK: the rule you now know 132
The numbers to remember 132
What to do next, at work 132

Cheat sheet 134

Questions and answers 135
Setup, Pro and seeing inside 135
What a skill is, and how Claude chooses 135
Skills that run Python 135
Many skills 136
Testing 136
Your own programs: `claude -p` and the SDK 136
Effort and cost 137
pip packages and messy input 137
Building a real harness 138

Appendix A - The nine SKILL.md files 140
converting-units 140
analyzing-csv 141
analyzing-text 142
calculating-dates 143
calculating-invoices 143
tracking-todos 145
clear-todos 145
analyzing-excel 146
plotting-charts 146

Appendix B - Python code examples 148
fncli.py 148
dates.py 150
convert.py 152
chart.py 155

Start here: what this course gives you

Most people use Claude Code like a chat: they type a request, read the answer, and hope it is right. This course takes you much further. When you finish it, you can **build, test and control an AI agent that does real work with your own Python code**. You can also **call Claude Code from your own Python programs instead of the model's API**, and you know when that gives better answers than a plain model call, and when it doesn't. You need only Windows and a Claude Pro subscription. You will not just know about these things. You will have run each one yourself and seen the real output, and you keep working code that you can copy into your own projects.

What you will be able to do

After this course, you can	Typical use without it	Where you learn it
Make Claude **run your Python** for exact answers, instead of calculating by itself	trust the model's arithmetic, which can be a cent off	Lesson 3
Give one agent **many skills** that don't compete, and **prove** that Claude picks the right one	write a description and hope	Lessons 2, 4 and 5
Check every skill for free before you use it: a linter, 74 unit tests, a routing test	find the mistakes when users do	Lesson 5
Call Claude **from your own Python programs with only Pro**, no API key, in two ways: `claude -p` and the Agent SDK	believe that an API key is needed	Lesson 6, Part 3
Use **Claude Code instead of a direct model call** in your own agent: trimmed for easy requests, the whole harness for hard ones	call the model's API, and build the loop, the tools and the checks yourself	Lesson 6, "The big idea"
Choose between `claude -p` and the SDK for each job, and switch every program between them	use one way for everything	Lesson 6, Part 4
Move your **Anthropic API code** (model, effort, max_tokens, usage) to Pro	pay for the API separately	Lesson 6, Step 20
Hold a **conversation with follow-up questions** from a program	start from zero with every question	Lesson 6, Step 14
Enforce **your own rules** on every command and file change, with hooks the model can't skip	write "please don't" into the prompt	Lesson 6, Step 15
Give Claude **your own functions as tools**, inside your program or as a stand-alone MCP server	not possible without a big framework	Lesson 6, Step 8
Save allowance with the same model: an effort level per skill, and the thinking tokens measured	switch to a weaker model, or never think about it	Lesson 6, Part 6
See every step Claude takes: its thinking, each command, each effort level, each token	see only the final answer	Lesson 0, section 7
Use **pip packages** such as pandas and matplotlib in skills, safely, with a venv	"it works on my machine"	Lesson 7
Turn **messy, misspelled user input** into strict, testable commands	let the model do everything, untested	Lesson 8

What surprises most people

These are the findings that change how you work. The course proves each one with real runs.

Most people think	What you will see for yourself	Proof
Calling the model's API directly is the most capable way	Claude Code wraps the same model in a **harness**: agent loop, tools, skills, context management, rules, sessions. On a public benchmark the harness alone moved the same model from about 58% to 82%	Lesson 6, "Why go through Claude Code at all"
The Agent SDK is a separate, more powerful engine	it starts **the same Claude Code program** (`claude.exe`), with the same login and skills	Lesson 6, Step 2
The SDK is always the better way for programs	**`claude -p` is often better**: nothing to install, works from `.bat` files and schedulers, text in and text out	Lesson 6, Steps 12 and 13
`claude -p` can do everything the SDK can	it can, but the SDK is **much better for conversations** (0.9 seconds per follow-up against 5.6). It is also better for rules or tools that live in your program	Lesson 6, Step 14
Scripting Claude needs an API key	**a Pro subscription is enough**. But if `ANTHROPIC_API_KEY` is set, it quietly switches the bill to that API account	Lesson 0, section 1
A long script makes a skill expensive	**scripts cost nothing in context**: Claude runs them and reads only their output	Lesson 1, Step 5
To save allowance you need a cheaper model	**the same model with lower effort** thinks less; a skill can set its own effort	Lesson 6, Step 22
Most tokens are Claude's thinking and answer	for a short job, most tokens are the **context read again at every step**	Lesson 6, Step 28
A good description is enough	only a **routing test with real questions** shows what Claude chooses: Haiku chose no skill in 3 of 20, Opus passed all 20	Lesson 0, section 4; Lesson 5, Test 4
Instructions in a skill are rules	Claude can still decide otherwise; **only a hook enforces** a rule	Lesson 6, Step 15
Claude Code is a tool for programmers only	its harness is **general**: none of the course's 9 skills writes code; Anthropic built its non-coding agent, Cowork, the same way	Lesson 6, "When the harness is the right choice"
A Pro login can run your company's agents	Pro is for **your own, individual use**; products and company services need an API key, and the same programs run with one	Lesson 6, "When the harness is the right choice"
Claude Code's own instructions make even a plain call smarter	with the tools off, Claude Code's prompt brought no gain in a measured test, at about 4 times the tokens. One flag, `--strict-mcp-config`, cut a plain call from about 4,300 to 870 tokens	Lesson 6, "Using Claude Code only as the model"
Allow Claude Code to run Python, and it will compute	**tools alone didn't**: Haiku worked in its head, or wrote commands the allow rule refused. One appended sentence made it compute: right every time on a 300-row table, at a quarter of a plain call's cost per right answer	Lesson 6, "Three questions"
`$1` is the first argument of a skill	**`$0` is the first**: counting starts at 0	Lesson 8, Step 6
A permission callback sees every command	the SDK's `can_use_tool` **never sees** commands that a skill's `allowed-tools` approved; a hook does	Lesson 6, Step 15

What you get in the project folder

Everything in this course runs, and stays yours to copy:

- **9 working skills** that run Python: unit conversion, CSV and Excel analysis, text statistics, dates and working days, invoices, a to-do list, charts.
- **7 programs in `agent\`** that call Claude from Python, with their helpers (hook scripts and a tool server). Each of them, and the two measuring tools, works in both ways: the SDK and `claude -p`. One option switches between them.
- **Tools** that check skills (a linter), measure their size and cost, and test Claude's choice.
- **74 unit tests** that check all of it for free.
- **A header in every `.py` file**: what it does, why, how it works, how to run it, how to test it (with the expected output), what to do when something goes wrong, and which files it changes.

The last part of the course, **"What you have learned"**, maps each of these to a job you may have at work.

How to read this book

You have	Read
One evening	Lesson 0 ("Start here (every time)"), Lesson 1, Lesson 3, Part 1, Lesson 6, Steps 9 to 11 and Step 16, then the **cheat sheet**
A weekend	Lessons 0 to 9 in order. Every step has the same parts: **What**, **Why**, **Run**, **Expected result** and **How to read it**
A question later	the **cheat sheet**, the **questions and answers**, the appendices, and the header of each `.py` file

Every "Expected result" is a real output from the course machine. Compare yours with it: when they match, you know the step worked.

Introduction

This course teaches two things, with the Claude Code CLI and Python, on Windows, with only a Claude Pro subscription.

1. An agent with many skills. You learn:

- how Claude decides which skill fits your request, and
- how a skill's text makes Claude run **functions you wrote in Python files**.

2. Claude Code from your own Python programs, instead of a direct call to the model's API. You learn:

- how to use it as a plain LLM call, with `claude -p` or the Agent SDK, with your Pro login and no API key;
- why it is often **better** than a plain LLM call: Claude Code wraps the model in a harness (the agent loop, tools, skills and rules), so hard requests come back right. On a 3,000-row table, Claude Code was right 4 times out of 4, a plain call once;
- how to trim the harness, so easy requests stay cheap, and when a plain call is the better choice (Lesson 6, "The big idea").

Everything in this course runs. The project folder contains a working agent with 9 skills, Python programs that call Claude Code both ways, tests, and tools that check whether Claude picks the right skill and measure what a run costs.

New here? Read Start here: what this course gives you first. At the end come What you have learned, a one-page cheat sheet, and 67 questions and answers to test yourself.

Words used in this course

Read this list once. The lessons use these words without explaining them again.

Word	Meaning
Claude Code CLI	The `claude` program that you start by typing `claude` in cmd. CLI means "command-line interface": you type, it answers in the same window.
Session	One run of `claude`, from start until you type `/exit`. One session is one conversation.
Prompt	A message you type to Claude.
Model	The AI that answers: Opus (most capable), Sonnet, Haiku (fastest). Bigger models use more of your Pro allowance.
Token	The unit in which AI text is counted. One token is about three quarters of an English word. Usage limits count tokens.
Context window	Everything Claude can see at one moment: instructions, the conversation so far, file contents, command outputs. It has a size limit. The more it contains, the more each message costs.
Tool	An action that Claude Code performs for Claude. Examples: **Read** (read a file), **Write**/**Edit** (change a file), **Bash** and **PowerShell** (run a command), **Glob**/**Grep** (find files or text), **Skill** (load a skill).
Permission prompt	The question "Allow Claude to ...?" that Claude Code shows before some tools. You answer Yes or No.
Thinking	The reasoning Claude does before it acts or answers. Claude Code shows it only for a moment, then folds it away.
Transcript view	The detailed view that **Ctrl+O** opens: Claude's thinking, and every tool call with its full command and output. **Esc** goes back (Lesson 0, section 7).
Skill	A folder with a `SKILL.md` file: a description, instructions, and often a Python script.

Word	Meaning
Frontmatter	The settings block at the top of `SKILL.md`, between two `---` lines (for example `name:` and `description:`). It uses YAML: simple `key: value` lines.
Description	The `description:` line in the frontmatter. It tells Claude what the skill does and when to use it.
Skill listing	The list of the names and descriptions of all skills. Claude Code puts it in the context window at the start of every session, so Claude knows which skills exist.
Routing	Choosing the skill that fits a request. Claude does this itself, by comparing your request with the skill listing.
Slash command	A message that starts with `/`, for example `/clear`, `/model`, or `/converting-units`. It runs a built-in command, or starts a skill directly.
Headless mode	`claude -p "question"`: Claude Code answers one prompt without the interactive screen, prints the answer and exits. Programs use it. Nobody is there to answer permission prompts.
Agent SDK	The Python package `claude-agent-sdk`. It runs Claude Code from your own Python program (Lesson 6).
stdout, stdin	stdout is the text a program prints. stdin is text a program reads as input, for example through a pipe: `echo hello | python script.py`.
JSON	A text format for data, for example `{"region": "North", "amount": 2150}`. The skill scripts print JSON so Claude can read their results exactly.
Exit code	The number a program returns when it ends. 0 means success; any other number means an error.
Standard library	The modules that come with Python (`json`, `csv`, `datetime`, ...). No installation is needed.
pip package	A Python library that you install with `pip install`, for example pandas or matplotlib.
venv	"Virtual environment": a separate Python folder with its own pip packages. This course uses `C:\pyenvsmy\claudeclioct26`.
PATH	The list of folders where Windows looks for programs. It decides which `python.exe` runs when you type `python`.
CLAUDE.md	A text file with project rules. Claude Code reads it at the start of every session in that folder.
Subagent	A helper Claude that Claude starts for a big sub-task. It works in its own context window and returns only a short result.
Hook	A command that Claude Code itself runs automatically on an event (for example, before every file edit). The model cannot skip it.
Plugin	An installable package of skills (and other extensions) that other people made.
MCP	"Model Context Protocol": a standard way to give Claude extra tools that run in a separate program or inside your Python program.
Linter	A program that checks files for mistakes without running them. Here: `tools/skills_lint.py` checks every `SKILL.md`.
Unit test	A small automatic test that checks that one function gives the right result.
Eval	Short for evaluation: running a list of test questions and measuring the results. Here: does Claude pick the expected skill?
Pro allowance	How much Claude your Pro subscription lets you use per 5 hours and per week. The claude.ai chat uses the same allowance.

Lessons

#	Lesson	You learn
0	Setup: Windows, CLI, Pro subscription	How to start every time, which Python runs, how to make sure Pro pays (not an API key), how to see Claude's thinking and every command (Ctrl+O)
1	What a skill is	The parts of a skill folder, and how Claude loads it step by step

#	Lesson	You learn
2	How Claude picks the right skill	Writing descriptions so the right skill is chosen; manual-only skills
3	Skills that run Python code	How SKILL.md tells Claude to run your Python, and five ready-made patterns
4	Designing an agent with many skills	Planning many skills so they don't compete; naming; sharing code
5	Testing skills that run Python	Four kinds of tests: unit tests of the Python code, the linter, the preview, and real Claude runs. How to read a failure
6	Your own agent with the Agent SDK	Claude Code instead of a direct model call: when its harness wins, and how to trim it; running Claude Code from Python; your functions as tools; Claude as an LLM with Pro; when to use `claude -p` and when the SDK; reasoning effort; effort per skill
7	Skills that need pip packages	Skills that use pandas and matplotlib from your venv
8	Messy input: Claude reads, Python computes	A calculator skill for requests in any order, with words and typos; `$ARGUMENTS`, `$0`, `$1` explained
9	Exercises	Eight hands-on exercises, from 5 minutes to a stretch goal

The 9 skills of the example agent

Skill	What it does	Who can start it	Python file	What it teaches
`converting-units`	Converts km/miles, kg/lb, °C/°F, GB/GiB, ...	Claude, or you with `/converting-units`	`convert.py`	A script with several commands; error messages that tell Claude how to fix the call
`analyzing-csv`	Statistics, totals and top rows of a .csv file	Claude, or you	`csv_tool.py`	Extra details in a separate `reference.md` that Claude reads only when needed
`analyzing-text`	Word count, reading time, readability of a text	Claude, or you	`text_stats.py`	Reading text that you pasted into the chat; separating it from a similar skill
`calculating-dates`	Days between dates, working days, weekday, age	Claude, or you	`dates.py` + `fncli.py`	**Calling any Python function by its name**
`calculating-invoices`	Invoice totals with discount and tax	Claude, or you	`invoice.py`	Check the data first, then calculate (plan, validate, execute)
`analyzing-excel`	Sheets, statistics and pivot tables of .xlsx files	Claude, or you	`excel_tool.py` (needs pandas)	**Using pip packages from your venv**
`plotting-charts`	Bar and line charts as PNG pictures	Claude, or you	`chart.py` (needs pandas, matplotlib)	**pip packages**; a result that Claude can look at
`tracking-todos`	A to-do list saved in `data/todos.json`	Claude, or you	`todo.py`	Data that is kept between sessions
`clear-todos`	Deletes tasks from the to-do list	**Only you**: type `/clear-todos done` (delete finished tasks) or `/clear-todos all` (delete all). Claude can never start it.	uses `todo.py` from `tracking-todos`	Making a dangerous skill manual-only (`disable-model-invocation: true`), and giving a skill the words you type after its name (`$ARGUMENTS`)

Quick start (20 minutes)

The quick start takes you once through the whole course. You start correctly, check that the Python code works, use the skills in Claude, and test Claude's choices. Each step says **what** you do, **why**, the **expected result**, and **what to do if you see something else**. The lessons explain each part in depth.

You need the Claude Code CLI, logged in with your claude.ai Pro account (Lesson 0). You also need the venv `C:\pyenvsmy\claudeclioct26`, with pandas, openpyxl and matplotlib installed (Lesson 7).

Step 1 - Open cmd and start the right way

What: two commands. The first switches on your virtual environment (venv). The second goes to the project folder.

Why: the skills run Python scripts. Two of them need pandas and matplotlib, and these packages are installed only in the venv. After `activate`, the word `python` means the venv's Python, in this window and for every command Claude runs from it. The project folder is where Claude Code finds the skills (`.claude\skills\`).

Run:

```
C:\pyenvsmy\claudeclioct26\Scripts\activate

cd /d "E:\may10\claudecodecli_and_python"
```

Expected result: the prompt starts with the venv's name and shows the project folder:

```
(claudeclioct26) E:\may10\claudecodecli_and_python>
```

If you see something else: if `(claudeclioct26)` is missing, the first command failed. Check that the folder `C:\pyenvsmy\claudeclioct26` exists. If the path is wrong after `cd`, check that you typed `/d`.

Step 2 - Check that the Python code works (unit tests)

What: the project folder contains a test file, `tests\test_skill_scripts.py`, with 74 small **unit tests**. A unit test is a short piece of Python. It runs one function of a skill script with a known input, and checks that the result is exactly right. For example, one test calls the conversion function with "100 km to miles" and checks that the answer is 62.137119.

Why: when Claude uses a skill, it runs the skill's Python script and **trusts its output**. If the script computes something wrong, Claude repeats the wrong number to you, with confidence. The tests prove that every script gives correct results **before** Claude uses it. They run on your computer only: they don't contact Claude, so they are free and take about 20 seconds. Run them after installing, and every time you change a script.

Run:

```
python -m unittest discover -s tests
```

How it works: `python -m unittest` starts Python's built-in test runner. `discover -s tests` tells it to look in the folder `tests` for files named `test_*.py`. It then runs every function whose name starts with `test_`.

Expected result:

```
.....................................................................
----------------------------------------------------------------------
Ran 74 tests in 31.351s

OK
```

How to read it: each dot (`.`) is one test that passed: 74 dots for 74 tests. `Ran 74 tests` counts them, and the time doesn't matter. `OK` means **all** passed.

If you see something else:

- **An `F` among the dots and `FAILED (failures=1)` at the end:** a test found a wrong result. Below the dots, a block that starts with `FAIL:` names the test. It shows what was expected and what came instead. Lesson 5, Step 2 shows how to read such a block, with a real example.
- **`OK (skipped=6)`:** you ran it without the venv, so pandas is missing. The 6 tests that need it (Excel, charts, the sample workbook) were skipped. Do Step 1 again.
- **An `E` and `ERROR`:** a script crashed. The block below the dots shows the error message.

Step 3 - Check that the skill files are written correctly (the linter)

What: `tools\skills_lint.py` is a **linter**: a program that reads every `SKILL.md` and checks it against the rules of this course, without running anything.

Why: Claude Code is silent about many mistakes in a `SKILL.md`. With a wrong name, a missing description, or a script path that doesn't exist, the skill is not chosen, or it fails. You don't see why. The linter finds these mistakes in a second, for free.

Run:

```
python tools/skills_lint.py
```

Expected result (the end of the output):

```
Description overlap (pairs that may compete for the same request):
   none above 0.30 - descriptions are well separated

9 skills, 0 error(s).
```

How to read it: there is one `[ok]` block per skill above this. `0 error(s)` means all 9 skills follow the rules. "No overlap" means no two descriptions are so similar that Claude could confuse them.

If you see something else: an `ERROR` line must be fixed. It says what is wrong, and in which skill. A `WARN` line is advice. Lesson 2, Step 4 shows each kind of message on broken practice skills.

Step 4 - Use the skills in Claude

What: you start Claude Code and ask questions that the skills can answer.

Why: to see the whole mechanism once. Claude reads the skill descriptions, chooses a skill, runs the skill's Python script, and answers with the script's result.

Run:

```
claude
```

Then type these requests one at a time. After each one, look at the tools that Claude Code shows: first **Skill** (with the skill's name), then **PowerShell** or **Bash** (with a `python ...` command). Claude Code often folds the commands into one line, like `ran 3 shell commands`. Press **Ctrl+O** to see each command, its output, and Claude's thinking. Press **Esc** to go back (Lesson 0, section 7):

```
What skills are available?
Which region sold the most in data/sales.csv?
If a job starts on 2026-10-15, what date is 10 working days later?
Total up data/invoice.json including the discount and tax.
Build a pivot of amount by region and product from data/sales.xlsx.
Plot the monthly sales amount from data/sales.xlsx as a line chart.
Remind me to call the bank on 2026-10-16.
What's on my to-do list?
/clear-todos done
What is the capital of Australia?
```

Expected result: for example, "Which region sold the most" uses `analyzing-csv` and answers North with 2,150. The last question uses **no** skill, because none fits. That is correct.

If you see something else: the first time Claude wants to use a skill, Claude Code asks your permission: answer **Yes**. If the Excel or chart request says `Python package 'pandas' is not installed`, you started `claude` without Step 1. Type `/exit` and start again from Step 1.

Type `/exit` when you are done.

Step 5 (optional) - Test Claude's skill choices automatically

What: `tools\routing_eval.py` asks Claude a list of test questions (in `tools\routing_cases.json`). For each question, it checks that Claude chooses the expected skill.

Why: Step 4 showed single examples. This test checks many questions at once. You see right away if a change to a description made Claude choose wrongly. It uses real Claude runs, so it uses a little of your Pro allowance.

Run:

```
python tools/routing_eval.py --limit 6
```

Expected result: a `PASS` line for each of the first 6 questions, and `6/6 routed as expected`. Lesson 5 explains the output and how the test works.

Step 6 (optional) - Run Claude from your own Python program

What: `agent\run_agent.py` is a Python program that runs Claude Code, with its skills, in the background, through the Agent SDK.

Why: so you see how to build your own agent program on top of the same skills (Lesson 6).

Run: (the first command installs the SDK once; skip it if it is already installed)

```
python -m pip install -r agent\requirements.txt
python agent\run_agent.py --model haiku "Which region sold the most in data/sales.csv?"
```

Expected result: lines that start with `[init]`, `[skill]`, `[tool]`, `[claude]` and `[done]`. They show each step Claude took. Lesson 6 explains them.

Folder map

```
.claude/skills/<name>/SKILL.md      the skills (description + instructions)
.claude/skills/<name>/scripts/*.py  the Python functions the skills run
data/                               sample inputs (sales.csv, sales.xlsx, ...); todos.json is created
    here
charts/                             PNG files made by the plotting-charts skill
lessons/                            the course
lessons/images/                     screenshots used in the lessons
tools/skills_lint.py                checks SKILL.md files for mistakes (free, offline)
tools/skill_sizes.py                shows the three loading levels of every skill as numbers (free)
tools/demo_bad_skills/              broken practice skills for the linter (never loaded by Claude)
tools/show_sdk_command.py           shows which claude.exe the Agent SDK starts, and with which
    options (free)
tools/route_preview.py              rough offline guess of the skill choice (free, a toy)
tools/routing_eval.py               real test: which skill does Claude pick? (claude -p; --sdk: the
    SDK; uses Pro allowance)
tools/routing_cases.json            the test questions and the expected skills
tools/run_cost.py                   one run: each step, its model, and the tokens (claude -p; --sdk:
    the SDK)
tools/model_mode_compare.py         Claude Code as the model: the same questions in several modes,
    right answers and tokens
tools/harness_profile_compare.py    non-coding data tasks: model only against Claude Code with tools,
    skills or everything
examples/skills/                    extra skills for Lesson 8 (calculator, argument demo); copied in
    when needed
tools/hooks/only_data_folder.py     hook for exercise 8 (blocks writes outside data/); off until you
    enable it
agent/ask_claude.py                 asks Claude a question, with only a Pro login (claude -p; --sdk:
    the SDK)
agent/api_equivalent.py             your Anthropic API call, done with Pro (claude -p; --sdk: the
    SDK)
agent/run_agent.py                  runs Claude Code with the skills, step by step (the SDK; --cli:
    claude -p)
agent/custom_tools_agent.py         gives Claude Python functions as tools (the SDK; --cli: claude -
    p)
agent/tutor_mcp_server.py           the same tools as a stand-alone MCP server, for the --cli way
agent/chat_agent.py                 several questions in one conversation (the SDK; --cli: claude -p
    --resume)
agent/guarded_agent.py              your Python rules allow or refuse each tool call (the SDK; --cli:
    claude -p)
agent/guard.py                      the rules of guarded_agent.py, as plain Python (tested)
agent/guard_hook.py                 the same rules as a hook script, for the --cli way
agent/effort_probe.py               shows the effort level in effect at each step (the SDK; --cli:
    claude -p)
agent/effort_hook.py                the effort hook, as a function (SDK way) and a hook script (--cli
    way)
agent/daily_report.bat              a scheduled job with claude -p only: no Python program, no SDK
tests/test_skill_scripts.py         automatic tests of every script
Claude_Code_Skills_Tutor.docx       this course as a Word file (+ .pdf), made for printing
```

SKILL.md template

Copy this to start a new skill. Everything inside the box is the text of the file. The ``` lines belong to the file too: in a SKILL.md they mark command text, the same way as in this course.

````
---
name: doing-the-thing
description: <Verb>s <objects> - <capability>, <capability>, <capability>. Use when the user asks
  <situations, words users say, file types>. Not for <a similar job> (use <other-skill>).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Doing the thing

<One line: why Claude should use the script instead of guessing.>

1. <First step - usually look at the input>:
   ```bash
   python "${CLAUDE_SKILL_DIR}/scripts/thing.py" inspect <file>
   ```
2. <Main action>:
   ```bash
   python "${CLAUDE_SKILL_DIR}/scripts/thing.py" do <args>
   ```
3. Report the numbers from the JSON output. If the output is `{"error": ...}`, fix the arguments as
   the message says and run again.

Details: [reference.md](reference.md)
````

Sources (Claude Code 2.1.x documentation, October 2026): Skills (https://code.claude.com/docs/en/skills) · Agent SDK skills (https://code.claude.com/docs/en/agent-sdk/skills) · Agent SDK custom tools (https://code.claude.com/docs/en/agent-sdk/custom-tools) · Skill authoring best practices (https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices)

Lesson 0 - Setup: Windows, Claude Code CLI, Pro subscription

Goal: start everything in this course the right way, and understand **why** this way is needed. Then check, with commands, that your **Claude Pro subscription** pays (not a separate API account).

Install Claude Code CLI and log in (only once)

1. **Install Claude Code.** Run this in **PowerShell**. It downloads the official installer from claude.ai and runs it:

   ```
   irm https://claude.ai/install.ps1 | iex
   ```

   Also recommended: install "Git for Windows". Claude Code can then run commands in Git Bash as well as in PowerShell.
2. **Install Python** 3.10 or newer from python.org. In the installer, tick "Add python.exe to PATH".
3. **Log in.** Start `claude` for the first time. A browser opens. Choose **Claude account (claude.ai)** and sign in with your Pro account. Claude Code saves the login on this computer, so you do this only once. To switch accounts later, type `/logout` and then `/login`.

The idea in one paragraph

A skill in this course runs a Python script with the plain command `python`. Claude Code does not choose a Python by itself. It asks Windows, and Windows uses the first `python.exe` that it finds in the folder list called **PATH**. Each window has its own PATH, and every program started in a window gets a copy of it. So you prepare **one** cmd window (activate the venv, go to the project), and then start `claude` in that window. Claude Code, and every `python` command it runs for a skill, then use the venv's Python. That's all the "connection" between skills and your venv.

Start here (every time)

Open **cmd** (Command Prompt): press the Windows key, type `cmd`, and press Enter.

Step 1 - Switch on the venv

What: a **venv** (virtual environment) is a folder with its own copy of Python and its own installed packages. Yours is `C:\pyenvsmy\claudeclioct26`. `activate` is a small script in it that "switches it on" for the current window.

Why: two skills need pandas, openpyxl and matplotlib. These packages are installed **only** in the venv, not in the system Python.

Run:

```
C:\pyenvsmy\claudeclioct26\Scripts\activate
```

How it works: `activate` puts the venv's folder `C:\pyenvsmy\claudeclioct26\Scripts` at the **front** of this window's PATH. When anything in this window asks for `python`, Windows looks there first. It changes nothing on your computer outside this window.

Expected result: the prompt now starts with the venv's name. For example, if you were in `C:\Users\Workstation`:

```
(claudeclioct26) C:\Users\Workstation>
```

If you see something else: `The system cannot find the path specified.` means the path is wrong. Check the folder name in File Explorer.

Step 2 - Go to the project folder

What: `cd` ("change directory") moves the window to another folder.

Why: Claude Code looks for skills in the folder `.claude\skills\` **inside the folder where it starts**. The skills' examples also use paths like `data/sales.csv`, relative to this folder.

Run:

```
cd /d "E:\may10\claudecodecli_and_python"
```

How it works: `/d` also changes the drive (`C:` to `E:`). Without `/d`, cmd would remember the new folder but stay on drive `C:`. The quotes are needed for paths with spaces, and harmless otherwise.

Expected result:

```
(claudeclioct26) E:\may10\claudecodecli_and_python>
```

Step 3 - Check which Python you have now

What: `where python` lists every `python.exe` that Windows finds, in the order it searches.

Why: to **see** that the activation worked, instead of trusting it.

Run:

```
where python
```

Expected result:

```
C:\pyenvsmy\claudeclioct26\Scripts\python.exe
C:\Python312\python.exe
```

How to read it: the **first** line is the one that runs when you type `python`: the venv's Python. The system Python (`C:\Python312`) is still there, but second.

If you see something else: if `C:\Python312\python.exe` is first, the venv is not active. Do Step 1 again.

Step 4 - Start Claude Code

What: `claude` starts the Claude Code CLI in this window.

Why: started here, Claude Code gets a copy of this window's PATH, with the venv first. So every `python` command that Claude runs for a skill also uses the venv's Python.

Run:

```
claude
```

Check it inside Claude. Claude Code runs a line that starts with `!` directly as a command, without asking Claude. So it costs nothing:

```
! python -c "import sys; print(sys.executable)"
```

How it works: `python -c "..."` runs the Python code in quotes. `sys.executable` is the full path of the Python that runs it.

Expected result:

```
C:\pyenvsmy\claudeclioct26\Scripts\python.exe
```

Good to know: the first time Claude wants to use a skill, Claude Code asks your permission. Answer **Yes**, or choose the option that stops it asking again for this project. To leave Claude Code, type `/exit`.

Do this once, too: section 7 of this lesson sets up **Ctrl+O**, which shows Claude's thinking and every command it ran. The lessons ask you to "watch" Claude's steps. Ctrl+O shows you all of them.

In PowerShell instead of cmd, the three commands are `C:\pyenvsmy\claudeclioct26\Scripts\Activate.ps1`, `cd "E:\may10\claudecodecli_and_python"` and `claude`.

What happens if you forget to activate

What: you run the chart skill's script once with the system Python and once with the venv's Python.

Why: you will forget to activate one day. Then you should recognize the message, and know the fix.

Step 5 - Run a pandas script with the system Python

Run: (the full path forces the system Python, as if you had not activated)

```
C:\Python312\python.exe .claude/skills/plotting-charts/scripts/chart.py check
```

Expected result:

```
{"error": "Python package 'pandas' is not installed in C:\\Python312\\python.exe", "hint": "Claude
    Code was started without the skills' virtual environment. Activate the venv, then start claude
    again. ..."}
```

How it works: `check` asks the script to report which Python and package versions it uses. The script tries `import pandas` inside a `try`. When the import fails, the script doesn't crash. It prints this JSON message. Claude reads the message and passes the hint on to you.

Step 6 - Run the same script with the venv's Python

Run:

```
python .claude/skills/plotting-charts/scripts/chart.py check
```

Expected result:

```
{"python": "C:\\pyenvsmy\\claudeclioct26\\Scripts\\python.exe", "pandas": "3.0.6", "matplotlib":
    "3.11.2"}
```

How to read it: pandas is installed only in the venv. If you start `claude` **without** activating, Claude's `python` is the system Python. Then the Excel and chart skills give you the first message. The other 7 skills still work, because they use only Python's standard library.

What to do in each situation:

Situation	What to do
Normal use	"Start here": activate, go to the folder, start `claude`. All 9 skills work.
You forgot to activate	Type `/exit`, run `activate`, then start `claude` again.
You activated *after* starting `claude`	That has no effect on the running session, because it has its own copy of PATH. Type `/exit` and start `claude` again.
A skill needs a new package	In the activated window: `python -m pip install <package>` (Lesson 7).
You want to run the Lesson 6 Python programs	In the activated window, once: `python -m pip install -r agent\requirements.txt`.

1. Check that your Pro subscription is used (not an API key)

Step 7 - Show your login

What: `claude auth status` shows how Claude Code is logged in. It doesn't send a message to Claude, so it is free.

Why: to be sure your Pro plan pays, and not some other account.

Run:

```
claude auth status
```

Expected result: several lines. These two matter (the course machine shows exactly this):

```
"authMethod": "claude.ai",
"subscriptionType": "pro"
```

How to read it: `claude.ai` means you are logged in with your claude.ai account, and `pro` is your plan. Inside a session, `/status` shows the same.

Step 8 - Make sure no API key is set

What: an **API key** is a password for Anthropic's pay-per-token service. Claude Code reads it from the environment variable `ANTHROPIC_API_KEY`, if one exists.

Why: if that variable exists, Claude Code uses it **instead of** your Pro login. In headless mode (`claude -p`) and in the Agent SDK, it does this **without asking**. Then that API account is billed per token. You have no API key, so this should never happen. Check anyway.

Run: (`set ANTHROPIC` shows every environment variable whose name starts with ANTHROPIC)

```
set ANTHROPIC
```

Expected result, most likely:

```
Environment variable ANTHROPIC not defined
```

How to read it: there is no `ANTHROPIC_...` variable, so your Pro login is used.

If you see something else: a line `ANTHROPIC_BASE_URL=...` is harmless: it is only an address. Only a line that starts with `ANTHROPIC_API_KEY=` is a problem. To remove such a key permanently, run this in **PowerShell**, and then open a new window:

```
[Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", $null, "User")
```

The course's Python programs check this too, when they start. They print `auth: claude.ai subscription` when your Pro login is used.

2. What uses your Pro allowance

Why this matters: your Pro plan has a usage limit. The commands that contact Claude use some of it. The others run only on your computer and are free, so run them as often as you want.

Command	Uses Pro allowance?	What it does
`python -m unittest discover -s tests`	no	runs the automatic tests of the scripts
`python tools/skills_lint.py`	no	checks the SKILL.md files for mistakes
`python tools/route_preview.py "..."`	no	guesses the skill by counting shared words
`python tools/skill_sizes.py`	no	shows how big each skill is
`claude`	**yes**	every message you send uses some allowance
`python tools/routing_eval.py`	**yes**, a little per question	runs `claude -p` once per test question, and stops it as soon as Claude has chosen a skill
`python agent\run_agent.py "..."` and the other `agent\` programs	**yes**	your Python program runs Claude Code in the background (Lesson 6), with the same login

The claude.ai chat uses the same allowance. It renews over a rolling 5-hour window, plus a weekly limit. **Check it:** in Claude Code, type `/usage`.

3. Why you see dollar amounts

What: some outputs show a cost, for example `cost≈$0.2026` at the end of `run_agent.py`.

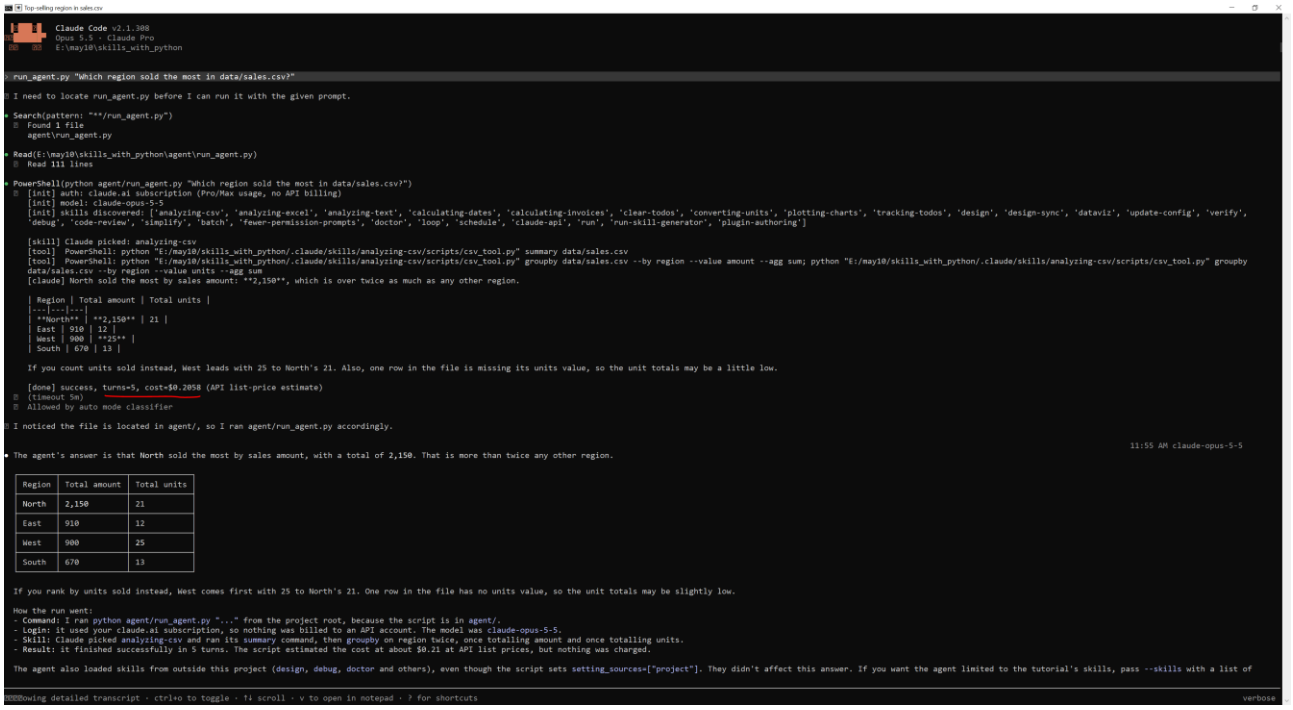
Why it is there: Claude Code counts the tokens of each run and multiplies them by the API price list. This shows what the same work *would* cost with an API key. **On Pro you don't pay this.**

How to use it: to compare how big runs were. A run with $0.20 used about four times as much of your allowance as one with $0.05.

Measured while writing this course (estimates, not charges):

What ran from Claude Code CLI	Model	Estimate	Why
`run_agent.py "Which region sold the most in data/sales.csv?"`	Opus 5.5	≈ $0.20	5 turns: chose the skill, ran two scripts, wrote a long answer
`run_agent.py --skills converting-units "...weekday..."`	Opus 5.5	≈ $0.06	3 to 4 short turns
the chart skill, start to end (Lesson 7)	Haiku 5.5	≈ $0.004	a small model, one script run
`ask_claude.py "Explain a Python decorator..."`	Haiku 5.5	≈ $0.001	one question, no tools

For example:



(Here `run_agent.py` was started from inside Claude Code CLI with Opus 5.5. The underlined part shows `turns=5, cost≈$0.2058`.)

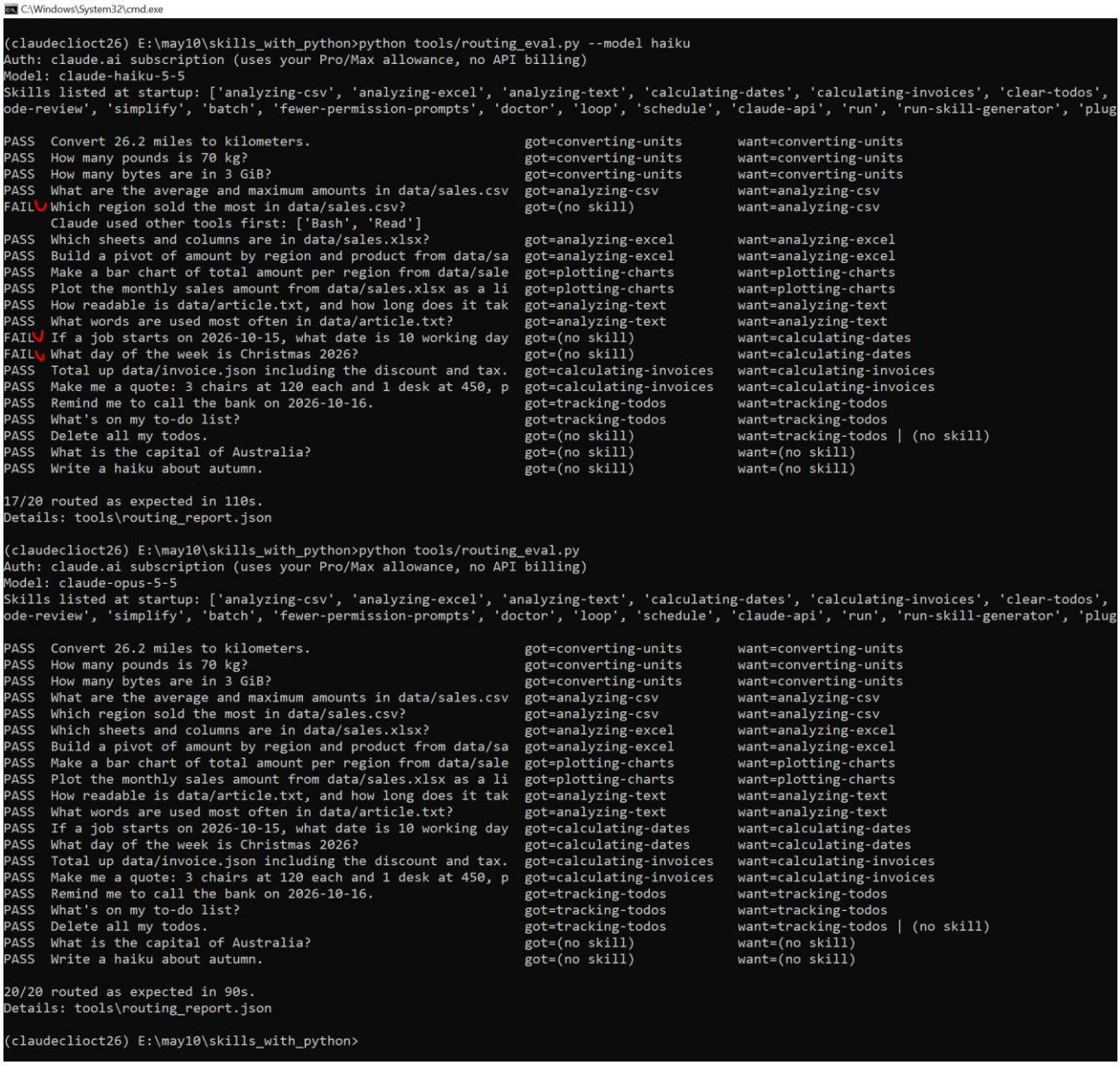
A "turn" is one round of Claude thinking and then acting, for example running one script.

4. Make your Pro allowance last

Do this	Why it saves allowance
Run the free checks first (tests, linter, preview, skill sizes)	`routing_eval.py` then only runs on skills that are already correct
`python tools/routing_eval.py --model haiku`	Haiku is a small model and uses much less allowance. In the chat, `/model` switches the model
`python tools/routing_eval.py --limit 4`	tests only the first 4 questions

Do this	Why it saves allowance
Type `/clear` when you start a new topic	Claude re-reads the whole conversation for each new message. So long conversations make every message more expensive. `/clear` empties it
Type `/effort low` for simple work (`--effort low` in `claude -p`, `effort="low"` in the SDK)	Claude then thinks less before it answers. Lesson 6, Part 5 measures how much
`max_turns=15` in `agent\run_agent.py`	stops a run that goes wrong, after 15 turns
Give simple skills `effort: low` in their SKILL.md, and keep `high` for skills that must reason	with the same model, Claude thinks less while the simple skills run (Lesson 6, Part 6)

But low-cost models can perform badly, as in this example:



Haiku failed 3 tests of 20:

```
FAIL  Which region sold the most in data/sales.csv?              got=(no skill)  want=analyzing-csv
FAIL  If a job starts on 2026-10-15, what date is 10 working day  got=(no skill)  want=calculating-
    dates
FAIL  What day of the week is Christmas 2026?                    got=(no skill)  want=calculating-
    dates
```

Opus 5.5 passed all of them.

5. Your own Python programs and Pro

How it works: when your Python program uses the Agent SDK (Lesson 6), the SDK starts Claude Code in the background. Claude Code uses the login saved on this computer. So your own programs, on your own computer, run on your Pro plan. You don't need an API key.

Rule: Pro is for your own use. Anthropic's Agent SDK documentation sets this rule. Unless Anthropic has approved it, developers may not offer claude.ai login or subscription limits in products for other people. A program for other people must use an API key.

Programs that run while you are away, for example a scheduled task at night, can't open a browser to log in. `claude setup-token` creates a long-lived login token for your subscription. Save it in that task's environment variable `CLAUDE_CODE_OAUTH_TOKEN`, and keep it secret like a password.

6. Windows notes

Note	Why	What the course does
Quote paths with spaces	`C:\Program Files\x.py` without quotes is read as two words: `C:\Program` and `Files\x.py`	the skills write `"${CLAUDE_SKILL_DIR}/scripts/..."` in quotes, so they keep working in any folder
PowerShell and Bash	on Windows, Claude often runs commands with its PowerShell tool, and sometimes with Bash (Git Bash)	every skill allows both: `Bash(python *)` and `PowerShell(python *)`
Special characters	a Windows terminal uses an old character table that can't print ≈ or →. A script that prints one stops with `UnicodeEncodeError`	every script starts its output with `sys.stdout.reconfigure(encoding="utf-8")`. Add it to your own scripts too
`python` not found	Python was installed without "Add python.exe to PATH"	use `py` instead, for example `py -m unittest discover -s tests`, or reinstall Python with the box ticked

7. See what Claude thinks and does (Ctrl+O)

What: by default, Claude Code shows you a short version of each answer. Two things are hidden:

1. **Claude's thinking.** Before it acts, Claude reasons about your request: which skill fits, which command to run, what the result means. While Claude works, you see this for a moment. Then it folds into one line, like `Crunched for 15s`.
2. **The details of the tools.** Several tool calls are folded into one line like `Read 1 file, ran 3 shell commands`. You don't see which commands ran, or what they printed.

This is the normal view on the course machine (Claude Code 2.1.303), for one question:

```
> Which region sold the most in data/sales.csv?

● Skill(analyzing-csv)
    Successfully loaded skill · 2 tools allowed
    Allowed by auto mode classifier

  Read 1 file, ran 3 shell commands

● North sold the most by revenue. Its total is 2,150, more than twice the next region.
  ...
  Crunched for 15s · done 5:19 PM
```

(`Allowed by auto mode classifier` appears only in **auto mode**, which you switch with Shift+Tab. There, Claude Code decides permissions automatically, instead of asking you.)

Why it is worth it: this course is about a chain of steps, from your question to the Python script and back. Lesson 3 shows it in the table at the top. The normal view shows only the first link (the **Skill** line) and the last one (the answer). **Ctrl+O** opens the **transcript view**, which shows every link in between:

In the transcript view you see	What it tells you	Explained in
Claude's thinking before the Skill line	**why** Claude chose this skill, or why it chose none	Lesson 2
`Skill(analyzing-csv)`	the choice itself	Lesson 2
each file Claude read, for example a skill's `reference.md`	Claude loaded extra know-how (level 3)	Lessons 1 and 4
each shell command, for example `python "E:/may10/.../csv_tool.py" groupby data/sales.csv ...`	the **exact** command, with `${CLAUDE_SKILL_DIR}` already replaced by the full folder	Lesson 3
the output of each command	the JSON the script printed: the **only** thing Claude knows about the result	Lesson 3
Claude's thinking between the steps	how Claude reads the JSON, and decides what to do next	Lesson 3

So you can **check** every claim of this course yourself, instead of trusting it. It also helps when something goes wrong:

Problem	Where to look in the transcript view
Claude chose the wrong skill, or none	the thinking before the Skill line: Claude says why. Often it thinks it knows the answer without the skill (Lesson 5, the failing routing example)
The answer has a wrong number	the command and its output. If the script printed the wrong number, fix the script and its unit test. If the script was right but Claude read it wrong, make the SKILL.md instructions clearer
A command failed	the JSON error and its hint, and the next command, where Claude fixed its call (Lesson 3, Step A3)

Step 9 - Turn on thinking summaries (only once)

What: you add one setting, `"showThinkingSummaries": true`, to your personal Claude Code settings file, `C:\Users\<you>\.claude\settings.json`.

Why: Ctrl+O shows the tool details without this setting. But Anthropic hides the thinking text by default. The transcript view then shows only a short placeholder where the thinking was. With this setting, it shows a **summary** of the thinking. The setting changes only what you see: Claude thinks the same way and uses the same amount of your Pro allowance.

Why your personal file, and not the project's `.claude\settings.json`: your personal file applies to every project. And Lesson 9, Exercise 8 deletes the project's settings file at the end.

Do:

1. In Claude Code, type `/exit`. Claude Code reads its settings when it starts.
2. Make a backup of the file, so you can undo a mistake:

   ```
   copy %USERPROFILE%\.claude\settings.json %USERPROFILE%\.claude\settings.backup.json
   ```

   Expected result: `1 file(s) copied.` If you see `The system cannot find the file specified.`, the file doesn't exist yet: skip the backup. In step 3, Notepad asks whether to create it: answer **Yes**.
3. Open the file in Notepad:

   ```
   notepad %USERPROFILE%\.claude\settings.json
   ```

The file is in **JSON**: `"name": value` pairs, separated by commas, inside `{ }`. On the course machine it looked like this:

```
{
  "autoUpdatesChannel": "latest",
  "theme": "dark"
}
```

4. Click at the very end of the **last** setting line, here right after `"dark"`.
5. Type a comma: `,`. **Why:** JSON needs a comma between two settings.
6. Press **Enter**, and type this line:

   ```
     "showThinkingSummaries": true
   ```

   Use straight quotes ", and write `true` in small letters, without quotes. There is no comma at the end, because it is now the last setting.
7. The file must now look like this (your other settings may differ):

   ```
   {
     "autoUpdatesChannel": "latest",
     "theme": "dark",
     "showThinkingSummaries": true
   }
   ```

   If the file was new and empty, type the whole thing with only this one setting:

   ```
   { "showThinkingSummaries": true }
   ```
8. Save with **Ctrl+S**, and close Notepad.

Check it: this command reads the file as JSON, and prints it as Python sees it. It fails if a comma or a quote is wrong:

```
python -c "import json, os; print(json.load(open(os.path.expanduser('~/.claude/settings.json'))))"
```

Expected result:

```
{'autoUpdatesChannel': 'latest', 'theme': 'dark', 'showThinkingSummaries': True}
```

If you see something else: `json.decoder.JSONDecodeError` names the line and column of the mistake, usually a missing comma or quote. Open the file again and compare it with step 7, or restore the backup:

```
copy /y %USERPROFILE%\.claude\settings.backup.json %USERPROFILE%\.claude\settings.json
```

Step 10 - Use the transcript view

What: you ask one question, and then read everything that happened, step by step.

Run: start Claude Code the usual way ("Start here": activate, `cd`, `claude`), and type:

```
Which region sold the most in data/sales.csv?
```

Do: wait until the answer is finished, and the line `Crunched for ...s` (or a similar word) appears. Then:

1. Press **Ctrl+O**. The transcript view opens.
2. Go through it from top to bottom, and find each row of the first table above: the thinking, the Skill line, the commands, their JSON output, and the answer.
3. Press **Esc** (or `q`) to go back to the normal view.

Expected result: the same conversation as in the normal view, but with every step written out. Each command shows the full path of the skill's script. Below it is the JSON that it printed, for example the totals per region: North 2150, East 910, West 900, South 670. The exact layout depends on your Claude Code version.

If you see something else:

- No thinking text, only a short placeholder: the setting is missing or has a typo. Check it with the command in Step 9, and restart `claude`.
- Some long outputs are shortened: in the transcript view, **Ctrl+E** shows all content. It works in Claude Code's classic display, not in its fullscreen display.

Step 11 - Read a real transcript: why is clear-todos missing?

What: a real transcript view from the course machine (Claude Code 2.1.305, Opus 5.5), for the prompt `list skills`. Claude notices that one skill is missing from its list, and finds out why.

Why: it shows in one short example what the transcript view adds: Claude's thinking **and** each tool call with its result. Without Ctrl+O you would see only the answer and `Worked for 13s`.

Try it yourself: in `claude`, type `list skills`, wait for the answer, and press **Ctrl+O**. Your transcript will be worded differently, but it shows the same kinds of lines.

The real transcript (shortened; long lines are wrapped). In the terminal the thinking is gray text; here it is marked `[thinking]`:

```
> list skills

[thinking] The user wants the list of skills, and I already have the project skills from the
.claude/skills/ listing - the first eight entries. I could double-check the directory for
anything not shown, like disabled skills, with a quick glob, but the existing listing should
be enough.

● Search(pattern: ".claude/skills/*/SKILL.md")
    Found 9 files
      .claude\skills\calculating-dates\SKILL.md
      .claude\skills\calculating-invoices\SKILL.md
      .claude\skills\tracking-todos\SKILL.md
      .claude\skills\clear-todos\SKILL.md
      .claude\skills\plotting-charts\SKILL.md
      .claude\skills\analyzing-excel\SKILL.md
      .claude\skills\converting-units\SKILL.md
      .claude\skills\analyzing-csv\SKILL.md
      .claude\skills\analyzing-text\SKILL.md

[thinking] I notice clear-todos isn't showing up in the listing, so I should check its
frontmatter to see why.

● Read(E:\may10\claudecodecli_and_python\.claude\skills\clear-todos\SKILL.md)
    Read 27 lines
                                                          07:50 PM claude-opus-5-5
● This project has 9 skills in .claude/skills/:

  Skill              What it does
  analyzing-csv      Statistics, group-by totals and top-N rows for .csv files
  ...
  clear-todos        Deletes finished tasks, or all tasks with /clear-todos all. You have to
                     run it yourself because it has disable-model-invocation: true, so Claude
                     never starts it on its own.

  There are also built-in and plugin skills in this session, such as code-review, simplify,
  skill-creator, pdf, docx, xlsx and pptx. They don't come from this repo.

  Worked for 13s · done 7:50 PM

Showing detailed transcript · ctrl+o to toggle · ↑↓ scroll · v to open in notepad · ? for shortcuts
```

How to read it, from top to bottom:

Part of the transcript	What it shows	Explained in
`> list skills`	your prompt	

Part of the transcript	What it shows	Explained in
the first `[thinking]`	Claude already has the **skill listing** (level 1) with "the first eight entries", not 9. It plans to check the folder for skills that are not shown	Lesson 1, Step 5
`Search(pattern: ".claude/skills/*/SKILL.md")` and `Found 9 files`	Claude's file-search tool lists every skill folder on disk: there are **9**	Lesson 1, Step 1
the second `[thinking]`	Claude compares the two: 9 on disk, 8 in its list. So one skill is hidden, and it decides to find out why	
`Read(...\clear-todos\SKILL.md)` and `Read 27 lines`	Claude reads the hidden skill's file, and finds `disable-model-invocation: true`. (Today the file has 28 lines: it got an `effort: low` line later, Lesson 6, Part 6)	Lesson 2, Step 6
`07:50 PM claude-opus-5-5`	the time and the model of this answer. The transcript view adds them	
the answer	the table, with the reason in the `clear-todos` row	
`Worked for 13s · done 7:50 PM`	the line that stays in the normal view, after the thinking is folded away	
the status bar	you are in the transcript view. `ctrl+o` switches back, and ↑↓ scrolls. `v` opens the whole transcript in Notepad, where you can search it (Ctrl+F) or save it. `?` lists the shortcuts	

What this example teaches:

1. **"Only you" skills are really left out of Claude's list.** You don't have to trust the lessons: Claude itself says its listing has eight entries. That is what `disable-model-invocation: true` does (Lesson 2, Step 6).
2. **The setting stops Claude from *starting* the skill, not from *seeing its file*.** Claude found and read `clear-todos\SKILL.md` with its normal Search and Read tools. It didn't run it. This is the same lesson as Lesson 6, Step 6: the skill list is not a lock.
3. **Read the thinking as Claude's reasoning, and the tool lines as what really happened.** The first thinking says "the existing listing should be enough", but Claude searched anyway. The thinking is a summary of Claude's reasoning, not a promise.

Step 12 - Ask the same question again

What: you ask exactly the same question a second time, in the same session.

Why: to see that Claude remembers the conversation, and what that means for skills.

Type:

```
Which region sold the most in data/sales.csv?
```

Expected result (a real run on the course machine):

```
● North sold the most, with total sales of 2,150. East is next at 910, then West at 900 and South at
    670.

  That's the same answer as before. I didn't re-run the script, so if data/sales.csv has changed
  since then, tell me and I'll run it again.
```

How to read it: there is **no** Skill line this time, and Ctrl+O shows no commands. The first answer is still in the conversation (the context window). So Claude reused it, instead of running the script again. That saves allowance, but if the file had changed, the answer would be old. To make Claude run the skill again, type `/clear` (it empties the conversation), and ask again.

Good to know: Ctrl+O exists only in the interactive window. In headless mode and in your own Python programs, you see the same steps as JSON events. Use `--output-format stream-json --verbose` (Lesson 5, Test 4), or the SDK's message objects (Lesson 6, Step 4).

Next: Lesson 1 - What a skill is

Lesson 1 - What a skill is

Goal: find out, by looking and running, what a skill is made of, why it is built that way, and how Claude uses it.

Before you start: open cmd and type the start commands (Lesson 0): first `C:\pyenvsmy\claudeclioct26\Scripts\activate`, then `cd /d "E:\may10\claudecodecli_and_python"`. Don't start `claude` yet.

The idea in one paragraph

A **skill** is a folder with instructions for Claude, and often a Python script. Claude Code reads the short **description** of every skill at the start. When your request matches a description, Claude reads that skill's instructions and follows them. In this course, the instructions say: "run this Python script". So Claude runs the script, reads the result it prints, and answers you. Python does the exact work (math, files, packages), and Claude does the understanding and the talking.

Step 1 - See which skills this project has

What: you list the skill folders of this project.

Why: Claude Code finds skills **only** by their folders. If a skill is not in this list, Claude Code doesn't know it.

Run: (`dir /b` lists the names in a folder, without dates and sizes)

```
dir /b .claude\skills
```

Expected result:

```
analyzing-csv
analyzing-excel
analyzing-text
calculating-dates
calculating-invoices
clear-todos
converting-units
plotting-charts
tracking-todos
```

How to read it: every folder in `.claude\skills\` is one skill, and the folder name is the skill's name. Claude Code looks in this folder when you start `claude` in the project folder.

Step 2 - Look inside one skill

What: you list the files of the unit conversion skill.

Why: to see the two parts every skill in this course has: instructions for Claude, and a script.

Run:

```
dir /b .claude\skills\converting-units

dir /b .claude\skills\converting-units\scripts
```

Expected result:

```
scripts
SKILL.md

convert.py
```

How to read it:

File	Required?	Who reads it	Purpose
`SKILL.md`	yes	Claude	the description, and the instructions: which command to run, and how to use its output
`scripts\convert.py`	no	nobody: Claude **runs** it	the exact work: conversion factors and math

Some skills also have extra text files that Claude reads when needed. Try `dir /b .claude\skills\analyzing-csv`: it also has `reference.md`.

Step 3 - Read the SKILL.md file

What: you print the skill's instruction file.

Why: this one file decides **when** Claude uses the skill, and **what** Claude does then.

Run: (`type` prints a file)

```
type .claude\skills\converting-units\SKILL.md
```

Expected result: (shortened)

````
---
name: converting-units
description: Converts physical quantities between units of length, mass, volume, speed, temperature,
    and digital data size (km to miles, kg to lb, °F to °C, liters to gallons, mph to km/h, MB to
    GiB) with exact factors. Use for every unit conversion request, even simple ones like miles to
    kilometers, and whenever the user asks "how many X in Y".
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Converting units

Never do unit math in your head. Run the script; it is exact and handles aliases.

## Convert one value

```bash
python "${CLAUDE_SKILL_DIR}/scripts/convert.py" convert 100 km mi
```
...
````

How to read it: the file has two parts.

1. **The frontmatter: the settings between the two `---` lines.** It is written in YAML: one `key: value` per line.

Key	What it does	Why it is needed
`name:`	the skill's name. You can also type it as a command: `/converting-units`	it must match the folder name
`description:`	what the skill does, and **when** to use it	Claude reads only this to decide whether the skill fits your request (Lesson 2)
`allowed-tools:`	allows Claude to run `python ...` commands **without asking you**, while it uses this skill	otherwise Claude Code asks your permission for every script run
`effort:`	how much Claude thinks while this skill runs: `low`, because the script does the work	the same model uses fewer tokens (Lesson 6, Part 6)

2. **The instructions: everything after the second `---`.** This is written for Claude, in normal language. It says "never do unit math in your head, run the script", and shows the command. The lines with ``` are part of the file: they mark the command, so Claude sees exactly what to type.

Check where the two `---` lines are:

Run: (`findstr /n "^---"` prints, with line numbers, every line that starts with `---`)

```
findstr /n "^---" .claude\skills\converting-units\SKILL.md
```

Expected result:

```
1:---
8:---
```

Why it matters: the first `---` must be on **line 1**. If anything comes before it, even an empty line, Claude Code doesn't see the settings at all. Then the skill has no description. Lesson 2, Step 4 shows how the linter catches this mistake.

Step 4 - Run the script yourself, as Claude would

What: you type the same command that the SKILL.md tells Claude to run.

Why: to see **exactly** what Claude sees. If the script works for you, it works for Claude.

Run:

```
python .claude/skills/converting-units/scripts/convert.py convert 26.2 mi km
```

How it works: `convert` is the action, `26.2` the number, `mi` the unit you have, and `km` the unit you want. The script looks up both units in its table, converts through meters, and prints the result as JSON.

Expected result:

```
{"value": 26.2, "from": "mi", "to": "km", "result": 42.164813, "category": "length"}
```

How to read it: this one line of JSON is everything Claude gets from the script. **JSON** is a text format of `"name": value` pairs that both Python and Claude read reliably. Claude reads `"result": 42.164813` and answers "about 42.16 km".

Where the path comes from: in the SKILL.md the command starts with `"${CLAUDE_SKILL_DIR}/scripts/..."`. Before Claude sees the instructions, Claude Code replaces this placeholder with the skill's folder (`E:\may10\claudecodecli_and_python\.claude\skills\converting-units`). So the command finds the script, whatever folder Claude is working in.

If you see something else: `{"error": "unknown unit 'mile'", ...}` or similar means the script doesn't know a unit name. Try `python .claude/skills/converting-units/scripts/convert.py units` to list all units it knows.

Step 5 - See the three loading levels as numbers

What: `tools\skill_sizes.py` measures how much of each skill Claude reads, and when.

Why: this is the most important idea of the course. Claude does **not** read all skills completely. It reads each skill in three levels, and each level only when needed. That is why you can have many skills without slowing Claude down.

Run:

```
python tools/skill_sizes.py
```

Expected result: (shortened)

```
skill                 who starts it       level 1        level 2   level 3
                                        description   instructions   extra files (scripts are run, not
    read)
-------------------------------------------------------------------------------------------------------
analyzing-csv         Claude or you     372 chars       29 lines   reference.md (49 lines),
    scripts/csv_tool.py (226 lines)
...
clear-todos           only you          107 chars       17 lines   -
converting-units      Claude or you     334 chars       31 lines   scripts/convert.py (222 lines)
...
-------------------------------------------------------------------------------------------------------
Level 1 total (in EVERY request): 2,672 characters, about 668 tokens.
Level 2 is added only for the skill Claude chooses. Level 3 scripts never enter the context:
Claude runs them and reads only what they print.
```

How to read it: look at the `converting-units` row from left to right.

Level	Column	When Claude sees it	For `converting-units`
1	description	**Always**, in every request, for every skill	334 characters
2	instructions	Only after Claude has chosen this skill	31 lines of SKILL.md
3	extra files	Only if the instructions send Claude there	`convert.py`: Claude **runs** it and sees only its output (Step 4), never its 222 lines of code

A **token** is the unit in which Claude reads text: roughly 4 characters. All 9 skills together cost only about 668 tokens per request (the last line). `clear-todos` says "only you": its description is not in Claude's list at all. So it costs nothing until you type `/clear-todos`.

Step 6 - Watch Claude use the skill

What: you start Claude Code and ask questions that the skill answers.

Why: to see the three levels happen: Claude matches the description (level 1), loads the instructions (level 2), and runs the script (level 3).

Run:

```
claude
```

Type in Claude, one after the other:

1. `What skills are available?` **Expected:** Claude lists the 9 skills of this project, plus some built-in Claude Code skills.
2. `How many kilometers is a marathon (26.2 miles)?` **Expected:** first you see the **Skill** tool with `converting-units`. Claude chose it, because your question matched the description. Next you see the **PowerShell** (or **Bash**) tool with the `python ... convert.py convert 26.2 mi km` command: Claude followed the instructions. Last comes the answer: about 42.16 km (from the script's JSON). If the command is folded into a line like `ran 1 shell command`, press **Ctrl+O** to see it with its JSON output. Press **Esc** to go back (Lesson 0, section 7).
3. `/converting-units 5 kg lb` **Expected:** the same tools, and about 11.02 lb. This time you started the skill yourself, by its name, so Claude didn't need to match your words to a description.

If you see something else: if Claude answers question 2 without the Skill tool, it answered from memory. That can happen with very easy questions. Lesson 5 shows how a description change fixed it.

Type `/exit` to go back to cmd.

Step 7 - Where else skills can live

What: you look at your **personal** skills folder.

Why: a skill can belong to one project or to you. Where you put it decides who can use it.

Run: (`%USERPROFILE%` is your user folder, `C:\Users\<you>`)

```
dir /b %USERPROFILE%\.claude\skills
```

Expected result (on the course machine):

```
synced
```

How to read it: a skill folder you put here works in **every** project on this computer. `synced` holds skills copied from your claude.ai account. If you see `File Not Found`, you have no personal skills yet, which is fine. There are three places for skills:

Place	Folder	Who can use it
Project	`E:\may10\claudecodecli_and_python\.claude\skills\<name>\`	Anyone who starts `claude` in this project folder. If you copy the project, the skills come with it.
Personal	`C:\Users\<you>\.claude\skills\<name>\`	Only you, in every project
Plugin	inside an installed plugin	Anyone who installs the plugin. Its skills are named `/plugin-name:skill-name`.

If a personal skill and a project skill have the **same name**, Claude Code uses the personal one and ignores the project one. So give your skills unique names.

Step 8 - A skill or something else?

What: you look at `CLAUDE.md`, another way to give Claude instructions.

Why: a skill is not always the right tool. Knowing the alternatives helps you choose.

Run:

```
type CLAUDE.md
```

Expected result: the project's rules, for example `- Run skill scripts from the project root; sample inputs are in data/.`

How it differs from a skill: `CLAUDE.md` is read **fully at the start of every session**. A skill's instructions are read **only when needed**. Choose like this:

You want...	Use	Example in this course
Rules for **every** request in the project	`CLAUDE.md`	the file you just printed
Know-how and a script for **one kind** of task	a **skill**	`converting-units`
A helper for a **big reading job**, so your conversation stays small	a **subagent**: a second Claude with its own context window that returns only a summary	Lesson 9, exercise 6
Your own **Python function** as a tool that Claude calls directly	an SDK `@tool`, or an MCP tool	Lesson 6, Part 2
A rule that must **never** be skipped	a **hook**: Claude Code runs your check automatically, and Claude can't skip it	Lesson 9, exercise 8

Next: Lesson 2 - How Claude picks the right skill

Lesson 2 - How Claude picks the right skill

Goal: see, by trying it, how Claude chooses a skill from its description, and understand why it works that way. Then learn to write descriptions that lead to the right choice.

Before you start: open cmd, `C:\pyenvsmy\claudeclioct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`.

How the choice works, step by step

1. When a session starts, Claude Code collects the **name and description** of every skill it may offer (level 1, Lesson 1, Step 5).
2. It gives Claude a tool called **Skill**, together with this list of names and descriptions.
3. For each request, Claude compares your words **and their meaning** with the descriptions. If one fits, Claude calls the Skill tool with that name. You see this as the **Skill** line in Claude Code.
4. Only now does Claude Code load that skill's instructions (level 2). Claude follows them, for example by running the Python script.
5. If no description fits, Claude doesn't call the Skill tool, and answers by itself.

So the **description is the only thing Claude has** for choosing. Everything in this lesson follows from that.

Step 1 - See exactly what Claude sees before it chooses

What: you print the description line of every skill.

Why: these lines are the "menu" Claude chooses from (point 1 above). If a word is not here, it can't help Claude choose.

Run:

```
findstr /s /b "description:" .claude\skills\SKILL.md
```

How it works: `findstr` searches text in files. `/s` searches the folder and all its subfolders, so `.claude\skills\SKILL.md` means "every file called SKILL.md below `.claude\skills`". `/b` keeps only lines that **begin** with `description:`.

Expected result: (each line is shortened here)

```
.claude\skills\analyzing-csv\SKILL.md:description: Summarizes and aggregates tabular data in .csv
    files - column types, ...
.claude\skills\analyzing-excel\SKILL.md:description: Reads Excel workbooks (.xlsx) with pandas -
    lists sheets and columns, ...
.claude\skills\analyzing-text\SKILL.md:description: Measures prose in .txt or .md documents - word,
    sentence and paragraph counts, ...
.claude\skills\calculating-dates\SKILL.md:description: Calculates with calendar dates - days or weeks
    between two dates, ...
.claude\skills\calculating-invoices\SKILL.md:description: Builds and checks invoices and quotes from
    line items - ...
.claude\skills\clear-todos\SKILL.md:description: Deletes tasks from data/todos.json - finished tasks
    only by default, ...
.claude\skills\converting-units\SKILL.md:description: Converts physical quantities between units of
    length, mass, volume, ...
.claude\skills\plotting-charts\SKILL.md:description: Draws bar and line charts as PNG images from
    .csv or .xlsx data ...
.claude\skills\tracking-todos\SKILL.md:description: Keeps a persistent to-do list for this project in
    data/todos.json - ...
```

How to read it: each line is `file:line found`. `clear-todos` is in the files, but Claude doesn't see it in its menu: Step 6 shows why. The rest of each SKILL.md is **invisible** until Claude has chosen the skill. So words that appear only in the instructions can't help Claude choose.

Step 2 - Watch Claude choose

What: you ask Claude five questions and watch which skill it chooses.

Why: to see that Claude chooses by meaning, and that "no skill" is a correct choice too.

Run: `claude`, then type each request below, one at a time. For each one, look at which skill appears with the **Skill** tool.

Expected result: the right column shows what happened on the course machine.

Type this	Skill Claude chose
`Which region sold the most in data/sales.csv?`	`analyzing-csv`
`Build a pivot of amount by region and product from data/sales.xlsx.`	`analyzing-excel`
`Plot the monthly sales amount from data/sales.xlsx as a line chart.`	`plotting-charts`
`If a job starts on 2026-10-15, what date is 10 working days later?`	`calculating-dates`
`What is the capital of Australia?`	no skill (Claude just answers)

How to read it:

- Claude understands **meaning**, not only words. The chart request names an `.xlsx` file, but asks for a *picture*, so it went to `plotting-charts`, not to `analyzing-excel`.
- Every description ends with a "Not for ..." sentence that names the neighbor skill. For example, `analyzing-excel` says: *"Not for .csv files (use analyzing-csv) or charts (use plotting-charts)."* That sentence tells Claude directly where the border between two similar skills is.
- When no description fits, Claude answers without a skill. That is correct.

If you see something else: Claude can choose differently from the table on another day, because its answers vary a little. If a skill is chosen wrongly again and again, Step 7 shows what to do.

Type `/exit`.

Step 3 - Try the free word-counting preview (and see its limits)

What: `tools\route_preview.py` guesses the skill by counting the words that your request and each description share.

Why: it doesn't contact Claude, so it is free and instant. It warns you early when a description lacks the words people type.

How it works: it uses **BM25**, a standard method of search engines. A shared word scores more points when it is rare among the descriptions (like "csv"), and fewer when many descriptions contain it (like "data"). It ignores small words like "the", and it ignores the "Not for ..." sentence, because a word counter can't understand "not".

Run:

```
python tools/route_preview.py "Which region sold the most in data/sales.csv?"
```

Expected result:

```
Request: Which region sold the most in data/sales.csv?

   2.80  analyzing-csv          shared words: csv, data
   2.79  plotting-charts        shared words: csv, data
   1.87  analyzing-text         shared words: most

  (not scored, /command only: clear-todos)

  -> Close call between the top two. Make their descriptions more distinct.
```

How to read it: the three best skills, with their score and the shared words. The word counter says "close call", because both descriptions contain "csv" and "data". But in Step 2, **Claude** chose `analyzing-csv` without trouble. Claude understands that "which region sold the most" asks for a number, not a picture. So the preview is only a rough warning, never the final answer.

Run:

```
python tools/route_preview.py "What is the capital of Australia?"
```

Expected result:

```
   0.00  tracking-todos         shared words: -
   0.00  plotting-charts        shared words: -
   0.00  converting-units       shared words: -

  -> No clear match. Claude would most likely answer without a skill.
```

How to read it: no description shares any word with the request. That fits: no skill is needed.

How to use the preview: if a **typical** request for your skill shares **no** words with its description, the description probably lacks the words people really type. Add them.

Step 4 - Find mistakes in descriptions with the linter

What: the folder `tools\demo_bad_skills\` contains four broken practice skills. Claude Code never loads them, because they are not in `.claude\skills\`. They are only there for the linter.

Why: to learn the typical mistakes, and the linter message for each, on skills where breaking something does no harm.

Run:

```
python tools/skills_lint.py tools/demo_bad_skills
```

How it works: without a folder, the linter checks `.claude\skills`. With a folder, it checks that folder instead.

Expected result:

```
[ok] csv-stats  (auto + /command, description 151 chars)

[ok] csv-totals  (auto + /command, description 145 chars)

[check] Helper_Tool  (auto + /command, description 30 chars)
   ERROR name must be lowercase letters, digits and single hyphens.
   WARN  name 'Helper_Tool' differs from folder 'helper' (both will work as /commands; confusing).
   WARN  Write the description in third person ('Converts...'), not 'I can' / 'you can'.
   WARN  Add a trigger clause such as 'Use when the user asks ...' so Claude knows WHEN to pick it.
   WARN  Body line 4: quote the path - "${CLAUDE_SKILL_DIR}/..." - it may contain spaces.
   WARN  Body line 4: '$5' is replaced by word 6 typed after the skill name ($0 is the first). Write
     \$5 for a literal dollar sign.
   ERROR Referenced file does not exist: scripts/helper.py

[check] no-frontmatter  (auto + /command, description 0 chars)
   ERROR Frontmatter must start with '---' on the very first line.

Description overlap (pairs that may compete for the same request):
   WARN  csv-stats <-> csv-totals: 1.00 shared words ['analyze', 'average', 'column', 'csv', 'data',
     'file', 'ranking', 'statistic', 'total']

4 skills, 3 error(s).
```

How to read it: one block per skill. `[ok]` has no problem of its own, and `[check]` has at least one. `ERROR` must be fixed. `WARN` is strong advice. The last line counts only the errors: 3.

Now look at the file behind most of the messages:

```
type tools\demo_bad_skills\helper\SKILL.md
```

```
---
name: Helper_Tool
description: I can help you with your data.
---

# Helper

Run python ${CLAUDE_SKILL_DIR}/scripts/helper.py and tell the user the price is $5.
```

Message	The mistake	Why it is a problem
`ERROR name must be lowercase...`	the name `Helper_Tool` has a capital letter and _	names may only use `a-z`, `0-9` and `-`. Claude Code may not load the skill
`WARN name ... differs from folder`	the name is not the folder name `helper`	two different names for one skill confuse you and Claude
`WARN Write the description in third person`	`I can help you with your data.`	the description is read as part of a list about skills. "I can" is unclear: who is "I"?
`WARN Add a trigger clause`	no "Use when ..." sentence	Claude can't know **when** to use the skill. "your data" fits almost any request
`WARN Body line 4: quote the path`	`${CLAUDE_SKILL_DIR}/scripts/helper.py` without "	if the project folder has a space, for example `E:\my skills`, the command breaks into two words
`WARN Body line 4: '$5' is replaced...`	`the price is $5`	in a SKILL.md, `$0`, `$1`, ... are placeholders for the words you type after `/name`, counted from 0 (Lesson 8). `$5` becomes the 6th word, usually nothing. Write `\$5`
`ERROR Referenced file does not exist`	`scripts/helper.py` is mentioned, but the file isn't there	Claude would run a command that fails
`ERROR Frontmatter must start with '---'`	`no-frontmatter` starts with an empty line	Claude Code doesn't see the settings at all (Lesson 1, Step 3). The skill has no name and no description
`WARN csv-stats <-> csv-totals: 1.00`	both descriptions use the same words (run `type tools\demo_bad_skills\csv-stats\SKILL.md` and `...csv-totals\SKILL.md`)	overlap `1.00` means all their important words are shared. Claude can't tell them apart. Merge them into one skill, or make them clearly different

Now check the real skills:

Run:

```
python tools/skills_lint.py
```

Expected result: (the end)

```
Description overlap (pairs that may compete for the same request):
   none above 0.30 - descriptions are well separated

9 skills, 0 error(s).
```

Step 5 - How to write a good description

What: you look at a real description and take it apart.

Why: you will write descriptions for your own skills, and they decide everything (see the top of this lesson).

Run:

```
findstr /b "description:" .claude\skills\analyzing-csv\SKILL.md
```

Expected result:

```
description: Summarizes and aggregates tabular data in .csv files - column types, missing values,
    min/max/mean/median, group-by totals and averages, top-N rows by a column. Use when the user asks
    about the contents, statistics, totals, or rankings in a CSV file. Not for .xlsx workbooks (use
    analyzing-excel), charts (use plotting-charts), or prose .txt documents (use analyzing-text).
```

How to read it: it has the three parts of a good description:

Part	In this description	Why
1. **What it does,** with concrete abilities	"Summarizes and aggregates tabular data in .csv files - column types, missing values, min/max/mean/median, group-by totals ..."	Claude can match requests like "what's the average" or "totals per region"
2. **When to use it,** with the words people type	"Use when the user asks about the contents, statistics, totals, or rankings in a CSV file."	tells Claude directly which requests belong here
3. **What it is NOT for,** naming the neighbor skills	"Not for .xlsx workbooks (use analyzing-excel), ..."	draws the border to similar skills. Keep it as the **last** sentence: the course's tools skip from "Not for" to the end

More rules, each with a good and a bad example:

Rule	Good	Bad
Write about the skill, not "I" or "you"	"Converts units..."	"I can help you convert..."
Name concrete abilities	"group-by totals, top-N rows"	"works with data"
Use the words people type, and file types	".csv", "VAT", "due date", "chart"	only your own technical terms
Put the main job first	"Converts units ..."	a long introduction first

One more rule, learned by testing: say if the skill is also for easy questions. In a test, "Convert 26.2 miles to kilometers." got **no** skill: Claude knew the answer (42.16 km) and didn't use the skill. The fix was one sentence in the description:

Run:

```
findstr /b "description:" .claude\skills\converting-units\SKILL.md
```

Expected result: (the end of the line)

```
... with exact factors. Use for every unit conversion request, even simple ones like miles to
    kilometers, and whenever the user asks "how many X in Y".
```

After this change the test passed. Lesson 5 shows the before and after results.

Size limits:

Limit	Why
a `name` has at most 64 characters	it is also the `/command` name
keep a `description` under 1,024 characters	Claude Code shows at most 1,536 characters and cuts the rest, so put the most important sentence first
all descriptions together fit in about 1% of Claude's context window	they are in every request. With 9 skills that is easy (Lesson 1, Step 5 showed 2,672 characters). With hundreds of long descriptions, Claude Code would list your least-used skills without their descriptions

Step 6 - A skill that only you can start

What: you look at the one setting that makes `clear-todos` a "manual only" skill.

Why: some skills must never run because Claude **thinks** it should: deleting, publishing, spending money. You decide when they run.

Run:

```
findstr /b "disable-model-invocation" .claude\skills\clear-todos\SKILL.md
```

Expected result:

```
disable-model-invocation: true
```

How it works: this line tells Claude Code not to put `clear-todos` in the list it gives to Claude (point 1 at the top). That is why it was missing from Claude's menu in Step 1. Claude can't choose what it doesn't see. You still can, by typing `/clear-todos`.

Try it: start `claude` and type `Delete all my todos.`.

Expected result: Claude can't use `clear-todos`. It uses `tracking-todos` instead (chosen in the course test). Its instructions tell Claude to ask you to type `/clear-todos` yourself. Then type `/clear-todos done`. Now the skill runs, because you started it, and it deletes the finished tasks.

All the options:

Setting in the frontmatter	You can type `/name`	Claude can choose it	In Claude's menu
(no setting: the normal case)	yes	yes	yes
`disable-model-invocation: true`	yes	**no**	no
`user-invocable: false`	no	yes	yes

`user-invocable: false` is the opposite: only Claude uses the skill, and it is hidden from your `/` menu. Use it for background knowledge that makes no sense as a command.

Step 7 - If the wrong skill (or none) is chosen

What: a checklist, from the cheapest check to the most expensive.

Why: a wrong choice has a few typical causes, and each has a free check.

#	Check	How	What it finds
1	Is the skill loaded at all?	in `claude`, type `What skills are available?`	a skill missing from the list is not loaded
2	If not loaded: is the file correct?	`python tools/skills_lint.py`	a missing first `---`, a wrong name, a broken path
3	If loaded but **not chosen**: does it share words with the request?	`python tools/route_preview.py "your request"`	no shared words: add the words people type, and a "Use when ..." sentence
4	If **another** skill was chosen	read both descriptions	add a "Not for ..." sentence to the skill that was wrongly chosen
5	If chosen **too often**	read the description	make it narrower, or make the skill "only you" (Step 6)
6	Measure the result	`python tools/routing_eval.py` (Lesson 5)	how many test questions go to the right skill

You can always force a skill: start your message with `/skill-name`.

Next: Lesson 3 - Skills that run Python code

Lesson 3 - Skills that run Python code

Goal: understand, by doing it yourself, how a skill makes Claude run the functions in your `.py` files. You see what happens at each step, why it is built that way, and how the Python code does its part.

How this lesson works. For each idea you first run the script yourself in cmd, **exactly as Claude runs it**, and look at the output. Then you look at the Python code that produced it. Then you ask Claude and watch it do the same thing. Every output shown here was copied from a real run on the course machine.

Before you start, open cmd and type the start commands from Lesson 0:

```
C:\pyenvsmy\claudeclioct26\Scripts\activate
cd /d "E:\may10\claudecodecli_and_python"
```

Don't start `claude` yet. You will type the commands below in this cmd window.

How a skill runs your Python, step by step

When you ask "How many miles is 100 km?", this happens:

#	Who	What happens
1	Claude	reads the skill descriptions, and chooses `converting-units` (Lesson 2)
2	Claude Code	loads the skill's SKILL.md. It replaces `${CLAUDE_SKILL_DIR}` with the skill's folder, and gives the text to Claude
3	Claude	reads the instructions: "run `python "../convert.py" convert VALUE FROM TO`". It fills in `100 km mi`
4	Claude Code	runs the command with its **PowerShell** or **Bash** tool: a normal program start, exactly as if you typed it in cmd
5	Python	runs `convert.py`. The script **prints** one line of JSON, and ends with an **exit code**: `0` for success, `1` for an error
6	Claude Code	catches everything the script printed, and gives it to Claude
7	Claude	reads the JSON, and answers you in normal language

Why let Python compute, and not Claude? Claude writes text. It usually gets math right, but not always, and it can't read an Excel file or draw a PNG by itself. A Python script gives the **same exact** answer every time, and can use any Python package. Claude does what it is good at: understanding your request, choosing the script, and explaining the result.

Why JSON? Python's `json.dumps()` turns a dictionary into one line of text with clear names (`"result": 62.137119`). Claude reads that more reliably than a free sentence, and a test can check it exactly (Lesson 5).

Part 1 - The basic idea in three steps

Step 1 - Run a skill's script yourself

What: you run the conversion script with the same command that Claude uses (step 4 above).

Why: to see exactly what Claude sees (step 6). If the output is right for you, it is right for Claude.

Run:

```
python .claude/skills/converting-units/scripts/convert.py convert 100 km mi
```

Expected result:

```
{"value": 100.0, "from": "km", "to": "mi", "result": 62.137119, "category": "length"}
```

How to read it:

- `convert` is the **command** you gave the script. `100 km mi` are its **arguments**: convert 100 from km to miles.
- The script prints its result as **JSON**: `"result": 62.137119` is the answer.
- This line is **all** that Claude sees when it runs the script. Claude never reads the 222 lines of Python code inside `convert.py`.

How the Python code does it. The end of `convert.py` (shortened):

```
def main(argv=None):
    sys.stdout.reconfigure(encoding="utf-8")       # Windows: print any character safely
    parser = argparse.ArgumentParser(description="Convert between units.")
    sub = parser.add_subparsers(dest="command", required=True)
    p_convert = sub.add_parser("convert", help="convert VALUE FROM TO")
    p_convert.add_argument("value", type=float)    # "100" becomes the number 100.0
    p_convert.add_argument("from_unit")            # "km"
    p_convert.add_argument("to_unit")              # "mi"
    sub.add_parser("units", help="list supported units by category")

    args = parser.parse_args(argv)                 # reads the words after convert.py
    if args.command == "convert":
        result = convert(args.value, args.from_unit, args.to_unit)   # the real work: a normal
    function
        print(json.dumps(result))                  # the one line Claude reads
        return 1 if "error" in result else 0       # the exit code
    ...

if __name__ == "__main__":                         # true only when run as a command
    sys.exit(main())
```

- `argparse` is Python's standard module for reading command words. It turns `convert 100 km mi` into `args.command="convert"`, `args.value=100.0`, `args.from_unit="km"`, `args.to_unit="mi"`.
- `convert(...)` is a normal Python function that returns a dictionary. `main()` only connects the command line to it, and prints the dictionary as JSON.
- `if __name__ == "__main__":` runs `main()` only when the file is started as a command. When a test imports the file, nothing runs by itself. The test can then call `convert(...)` directly (Lesson 5, Step 3).

Step 2 - Find the same command inside the skill

What: you find the lines in SKILL.md that tell Claude which command to run.

Why: to see the link between the skill's text and your Python file: it is just a command line.

Run: (`findstr /n` prints the lines of a file that contain a word, with line numbers)

```
findstr /n "python" .claude\skills\converting-units\SKILL.md
```

Expected result:

```
5:  - Bash(python *)
6:  - PowerShell(python *)
17:python "${CLAUDE_SKILL_DIR}/scripts/convert.py" convert 100 km mi
31:python "${CLAUDE_SKILL_DIR}/scripts/convert.py" units
```

How to read it:

- **Lines 17 and 31** are the commands that the skill tells Claude to run. Line 17 is the command you ran in Step 1. The only difference is `${CLAUDE_SKILL_DIR}`: before Claude reads the file, Claude Code replaces it with the skill's folder, `E:\may10\claudecodecli_and_python\.claude\skills\converting-units`. **Why:** Claude may be working in another folder. With the full path, the command always finds the script.

- **Lines 5 and 6** are the `allowed-tools` setting. They allow Claude to run any `python ...` command with its Bash or PowerShell tool **without asking you**, while it uses this skill. **Why both:** on Windows, Claude may use either tool. A command that is not allowed makes Claude Code ask you.
- **Why the quotes** around `"${CLAUDE_SKILL_DIR}/scripts/convert.py"`: they keep the command working even if a folder in the path has a space, for example `C:\Program Files\...`.
- **Why plain python:** Windows finds `python` in the PATH that Claude Code got from your activated cmd window. So it is your venv's Python (Lesson 0).

Step 3 - Watch Claude do the same

What: you ask Claude, and watch it run the same command.

Why: to see steps 1 to 7 of the table above happen.

Run:

```
claude
```

Type in Claude: `How many miles is 100 km?`

Expected result: Claude Code shows each tool it uses. You will see, in this order:

1. the **Skill** tool with `converting-units`: Claude chose the skill (step 1);
2. the **PowerShell** (or **Bash**) tool with `python "...convert.py" convert 100 km mi`: the same command as in Step 1, with the full folder filled in (steps 2 to 6);
3. Claude's answer, about 62.14 miles, taken from the `"result"` in the JSON (step 7).

See every step: Claude Code often folds item 2 into one line like `ran 1 shell command`. Press **Ctrl+O** (Lesson 0, section 7). The transcript view shows the full command and the JSON line that the script printed (exactly the line you saw in Step 1). It also shows Claude's thinking between the steps. Press **Esc** to go back.

If you see something else: if Claude Code asks permission to run the command, answer Yes. It asks once per skill, if you didn't choose "don't ask again" before.

Type `/exit` to leave Claude and come back to cmd.

That is the whole mechanism: the skill's text contains a command, Claude runs the command, and Claude reads the JSON that the script prints. The rest of this lesson shows five ways to write such scripts, each for a different kind of problem.

Part 2 - Five patterns, each one tried out

Pattern A - one script, several commands

The problem it solves: a skill usually needs a few related actions: convert a value, and list the units. One script with several **commands** keeps them together, and SKILL.md lists each one.

Used by: `convert.py`, `csv_tool.py` and `text_stats.py`.

How it works: `argparse` sub-commands (Step 1, the code above): each `sub.add_parser("name")` is one command, and each command calls one Python function.

Step A1 - Ask the script which commands it has

What: `--help` asks the script to describe itself.

Why: when you write a SKILL.md for a script, this shows you what to put in it. Claude can run it too, if it is unsure.

Run:

```
python .claude/skills/converting-units/scripts/convert.py --help
```

Expected result:

```
usage: convert.py [-h] {convert,units} ...

Convert between units.

positional arguments:
  {convert,units}
    convert        convert VALUE FROM TO
    units          list supported units by category
```

How to read it: the script has two commands, `convert` and `units`. You didn't write this help text: `argparse` made it from the `add_parser(..., help="...")` lines.

Step A2 - Use the second command

What: the `units` command lists every unit the script knows.

Why: you may type a unit name that the script doesn't know ("nautical miles"). Then the SKILL.md tells Claude to run `units` (line 31 in Step 2) and find the right name, instead of guessing.

Run:

```
python .claude/skills/converting-units/scripts/convert.py units
```

Expected result:

```
{"length": ["mm", "cm", "m", "km", "in", "ft", "yd", "mi", "nmi"], "mass": ["mg", "g", "kg", "t",
    "oz", "lb", "st"], "volume": ["ml", "l", "m3", "tsp", "tbsp", "cup", "pt", "qt", "gal", "floz"],
    "speed": ["m/s", "km/h", "mph", "kn"], "data": ["b", "kb", "mb", "gb", "tb", "kib", "mib", "gib",
    "tib"], "temperature": ["c", "f", "k"]}
```

How to read it: the units are grouped by category. `nmi` is the nautical mile.

Step A3 - Make a mistake on purpose

What: you ask for an impossible conversion: kilograms to miles.

Why: Claude sometimes calls a script with a wrong argument. What the script prints then decides whether Claude can fix the call by itself.

Run:

```
python .claude/skills/converting-units/scripts/convert.py convert 5 kg mi
```

Expected result:

```
{"error": "Cannot convert mass (kg) to length (mi).", "hint": "Valid mass units: mg, g, kg, t, oz,
    lb, st"}
```

Now check the exit code: (`%errorlevel%` is the exit code of the last command)

```
echo %errorlevel%

1
```

How to read it: the script didn't crash. It printed the problem **and how to fix it** (`"hint"`). It ended with exit code `1`, so Claude Code marks the command as failed. Claude reads the hint, picks a valid unit, and runs the command again.

How the Python code does it: inside `convert()`, the function **returns** the error as a dictionary instead of raising an exception:

```
if src_cat != dst_cat:
    return {
        "error": f"Cannot convert {src_cat} ({from_unit}) to {dst_cat} ({to_unit}).",
        "hint": f"Valid {src_cat} units: {', '.join(list_units()[src_cat])}",
    }
```

Why not let Python crash? A crash prints a "Traceback" of 10 or more lines about Python's internals. Claude may understand it, but it costs more tokens, and it doesn't say what a **valid** call would be.

Another example, with a column name typed with a capital letter:

Run:

```
python .claude/skills/analyzing-csv/scripts/csv_tool.py top data/sales.csv --column Amount
```

Expected result:

```
{"error": "Unknown column 'Amount'.", "columns": ["date", "region", "product", "units", "amount"]}
```

How to read it: the error lists the real column names. Claude sees that the column is `amount` (small letters) and fixes its call.

Step A4 - Give a script text instead of a file

What: you send text into `text_stats.py` without a file.

Why: if you paste text into the chat, there is no file to read. Claude could save the text to a file first, but that needs a write permission and leaves a file behind. A **pipe** is simpler.

Run:

```
echo Short text. Another one here! | python .claude/skills/analyzing-text/scripts/text_stats.py stats
    -
```

How it works: `echo` prints the text, and `|` (the pipe) passes it to the script's input, called **stdin**, instead of to the screen. The `-` instead of a file name tells the script "read stdin". In `text_stats.py` this is two lines:

```
if source == "-":
    return sys.stdin.read()      # the text that came through the pipe
```

Expected result:

```
{"words": 5, "sentences": 2, "paragraphs": 1, "characters": 31, "reading_time_min": 0.0,
    "avg_sentence_words": 2.5, "flesch_reading_ease": 85.9, "longest_sentences": ["Another one
    here!", "Short text."]}
```

How to read it: the script counted 5 words in 2 sentences. `flesch_reading_ease` is a readability score: 60-70 is plain English, 90-100 very easy, below 30 very hard.

Now in Claude: type `How readable is this text: "The cat sat on the mat. It was happy."`, and watch the command that Claude runs. It sends your text into the script with a pipe (`|`), and writes `-` instead of a file name. The SKILL.md of `analyzing-text` tells it to do this for pasted text.

Pattern B - call any Python function by its name

The problem it solves: a dates skill needs many small functions: days between, add days, add working days, weekday, age, and more later. Writing an `argparse` command for each one, and listing each in SKILL.md, is repetitive. It is also easy to forget when you add a function.

Used by: `dates.py`, with the helper file `fncli.py` (about 130 lines, in the same folder).

How it works: you write normal Python functions with **type hints** (`days: int`) and a **docstring** (the `"""..."""` text under the `def` line). The last lines of `dates.py` hand them to `fncli`:

```
PUBLIC = [days_between, add_days, add_business_days, business_days_between, weekday, age]

if __name__ == "__main__":
    fncli.run(PUBLIC)
```

`fncli.run()` uses Python's `inspect` module to read each function's name, parameters, types and docstring. Then it builds one command per function, and one `--option` per parameter. So the SKILL.md only says: "run `dates.py --list` to see the functions, then call one".

Step B1 - List all functions

What: `--list` prints every function in `PUBLIC`.

Why: this is how Claude finds out which functions exist, without reading the code.

Run:

```
python .claude/skills/calculating-dates/scripts/dates.py --list
```

Expected result (the first two of six functions):

```
[
  {
    "name": "days_between",
    "signature": "days_between(start: str, end: str) -> dict",
    "doc": "Calendar days from start to end (negative if end is earlier). Dates are YYYY-MM-DD."
  },
  {
    "name": "add_days",
    "signature": "add_days(date: str, days: int) -> dict",
    "doc": "Add (or subtract, if negative) calendar days to a date."
  },
  ...
```

How to read it: each entry is one Python function in `dates.py`: its name, its parameters (`signature`), and the first line of its docstring (`doc`). Nobody wrote this list by hand.

Step B2 - See how a function became a command

What: `--help` after a function name shows the command that `fncli` built for it.

Why: to see the rule that turns Python parameters into command options.

Run:

```
python .claude/skills/calculating-dates/scripts/dates.py add_business_days --help
```

Expected result:

```
usage: dates.py add_business_days [-h] --date DATE --days DAYS
                                  [--holidays HOLIDAYS]

Add business days (Mon-Fri), skipping comma-separated holiday dates. Days must
be >= 0.
```

How to read it, next to the Python function it came from:

```
def add_business_days(date: str, days: int, holidays: str = "") -> dict:
    """Add business days (Mon-Fri), skipping comma-separated holiday dates. Days must be >= 0."""
```

In the Python function	Becomes in the command
the name `add_business_days`	the command `add_business_days`
the parameter `date: str`	the required option `--date DATE` (text)
the parameter `days: int`	the required option `--days DAYS`, turned into a whole number
the parameter `holidays: str = ""`	the **optional** option `[--holidays HOLIDAYS]`: in `[ ]` because it has a default value
the docstring	the help text

Step B3 - Call the function

What: you call `add_business_days` with a date and a number of days.

Run:

```
python .claude/skills/calculating-dates/scripts/dates.py add_business_days --date 2026-10-15 --days
    10
```

Expected result:

```
{"date": "2026-10-15", "business_days": 10, "result": "2026-10-29", "weekday": "Thursday"}
```

How to read it: 10 working days after Thursday 15 October 2026 is Thursday 29 October 2026. Weekends are skipped. `fncli` called `add_business_days(date="2026-10-15", days=10)` and printed the returned dictionary as JSON.

Try a wrong date: `python .claude/skills/calculating-dates/scripts/dates.py add_days --date 15.10.2026 --days 3`. The function raises a `ValueError`, and `fncli` turns it into JSON, with the function's signature as a reminder of the right call:

```
{"error": "date must be an ISO date like 2026-10-15, got '15.10.2026'", "function": "add_days",
    "signature": "add_days(date: str, days: int) -> dict"}
```

Step B4 - Add your own function (learn by doing)

What: you add a new function to `dates.py`, and it becomes a command at once.

Why: this is the main benefit of Pattern B. You give Claude new abilities by writing only a normal Python function.

1. Open `.claude\skills\calculating-dates\scripts\dates.py` in an editor (for example Notepad).
2. Just above the line that starts with `PUBLIC = [`, paste this function:

   ```
   def is_weekend(date: str) -> dict:
       """True if the date is a Saturday or a Sunday."""
       day = _parse(date, "date")
       return {"date": day, "weekday": WEEKDAY_NAMES[day.weekday()], "is_weekend": day.weekday() >=
       5}
   ```

   `_parse` is a helper already in the file: it turns `"2026-10-17"` into a date, or raises a clear `ValueError`. `day.weekday()` is 0 for Monday up to 6 for Sunday, so `>= 5` means Saturday or Sunday.
3. Add `is_weekend` at the end of the `PUBLIC` list, so the line becomes:

   ```
   PUBLIC = [days_between, add_days, add_business_days, business_days_between, weekday, age,
       is_weekend]
   ```

   Why: only functions in `PUBLIC` become commands. Helpers like `_parse` stay private.
4. Save the file.

Run:

```
python .claude/skills/calculating-dates/scripts/dates.py is_weekend --date 2026-10-17
```

Expected result:

```
{"date": "2026-10-17", "weekday": "Saturday", "is_weekend": true}
```

How to read it: you wrote only a normal Python function, and it is already a command. Run `dates.py --list` again: your function is now at the end of the list. Claude can find and use it, even though the skill's SKILL.md doesn't mention it.

If you see something else: `invalid choice: 'is_weekend'` means you forgot step 3 or didn't save. `IndentationError` means the function's lines don't start with exactly 4 spaces.

Now in Claude: start `claude` and type `Is 2026-10-24 a weekend?`. Watch the commands. Claude may first run `dates.py --list` to look for a fitting function, and then call `is_weekend` (or `weekday`). Either way, the answer comes from your Python code.

When you are done experimenting, you can keep `is_weekend` or delete it again. The tests don't depend on it.

Pattern C - check the data first, then calculate

The problem it solves: for money, a mistake is expensive. Two things can go wrong:

1. **The input:** Claude writes the invoice file from your chat message. It can make a mistake in it (a missing price, a quantity of 0).
2. **The math:** Python's normal numbers (`float`) can't store most decimal fractions exactly.

See problem 2 yourself:

Run:

```
python -c "print(0.1 + 0.2)"

0.30000000000000004

python -c "from decimal import Decimal; print(Decimal('0.1') + Decimal('0.2'))"

0.3
```

Why: a `float` is stored in binary, and 0.1 has no exact binary form, just like 1/3 has no exact decimal form (0.333...). Python's `Decimal` type computes with decimal digits, exactly like on paper.

Used by: `invoice.py`.

How it works:

- The script has three commands, used **in order**: `validate` checks the file, `total` calculates, `render` writes the document.
- `validate` doesn't stop at the first problem. It collects **all** of them in a list, each with its exact place. So Claude can fix everything in one round:

  ```
  for index, item in enumerate(items):
      where = f"items[{index}]"
      if not str(item.get("description", "")).strip():
          problems.append(f"{where}.description is missing or empty.")
      ...
  return {"ok": not problems, "problems": problems}
  ```

- `total` runs the same checks again, and refuses to calculate if any fails.
- All amounts are `Decimal`, rounded to cents with the usual rule (5 and up rounds up):

  ```
  CENT = Decimal("0.01")
  def money(value):
      return value.quantize(CENT, rounding=ROUND_HALF_UP)
  ```

- The SKILL.md tells Claude the order, and gives it a checklist to copy into its reply and tick off. **Why a checklist:** Claude follows a written list of steps more reliably than a paragraph. And you can see which step it is on.

Step C1 - Look at a broken invoice

What: a practice invoice with mistakes.

Run: (`type` prints a file)

```
type data\invoice_broken.json
```

Expected result:

```
{
  "number": "INV-2026-043",
  "customer": "Beta GmbH",
  "tax_pct": 119,
  "items": [
    {"description": "Support plan", "qty": 0, "unit_price": 49},
    {"description": "", "qty": 2, "unit_price": "abc"}
  ],
  "shipping": 15
}
```

Can you find the mistakes yourself? There are five.

Step C2 - Let the script check it

What: `validate` checks the invoice without calculating anything.

Run:

```
python .claude/skills/calculating-invoices/scripts/invoice.py validate data/invoice_broken.json
```

Expected result:

```
{"ok": false, "problems": ["items[0].qty must be a number greater than 0, got 0.",
    "items[1].description is missing or empty.", "items[1].unit_price must be a number >= 0, got
    'abc'.", "'tax_pct' must be between 0 and 100, got 119.", "Unknown top-level fields ['shipping'];
    allowed: number, customer, currency, discount_pct, tax_pct, items, notes."]}
```

How to read it: `"ok": false`, and all five problems, each with its exact place. `items[0]` is the first item and `items[1]` the second (Python counts from 0). `shipping` is unknown: the script doesn't know how to tax it, so it refuses instead of ignoring it silently.

Step C3 - The script refuses to calculate a broken invoice

What: you try `total` on the broken invoice, as if Claude skipped the check.

Why: to see the second safety net. Even if Claude skips `validate`, no wrong total comes out.

Run:

```
python .claude/skills/calculating-invoices/scripts/invoice.py total data/invoice_broken.json
```

Expected result: (shortened)

```
{"error": "Invoice is invalid; run validate first: items[0].qty must be a number greater than 0, got
    0. ..."}
```

Step C4 - Check and calculate a correct invoice

Run:

```
python .claude/skills/calculating-invoices/scripts/invoice.py validate data/invoice.json
```

Expected result:

```
{"ok": true, "problems": []}
```

Run:

```
python .claude/skills/calculating-invoices/scripts/invoice.py total data/invoice.json
```

Expected result: (shortened)

```
{"currency": "USD", "lines": [...], "subtotal": "1440.47", "discount": "144.05", "tax": "106.95",
    "total": "1403.37"}
```

How to read it: subtotal 1440.47, minus 10% discount (144.05), plus 8.25% tax (106.95), gives a total of 1403.37. The amounts are text in quotes (`"1403.37"`), because `json.dumps` can't write a `Decimal` as a number without turning it back into a `float`. Text keeps every digit exact.

Step C5 - Make the invoice document

What: `render` writes the invoice as a Markdown file.

Run:

```
python .claude/skills/calculating-invoices/scripts/invoice.py render data/invoice.json --out
    invoice.md

type invoice.md
```

Expected result:

```
# Invoice INV-2026-042

**Bill to:** Acme Ltd

| Description | Qty | Unit price | Amount |
|---|---:|---:|---:|
| Consulting hours | 12 | 95.00 | 1140.00 |
| Workshop materials | 3 | 19.99 | 59.97 |
| Travel | 1 | 240.50 | 240.50 |

Subtotal: 1440.47 USD
Discount (10%): -144.05 USD
Tax (8.25%): 106.95 USD
**Total: 1403.37 USD**

Payment due within 30 days.
```

Delete the practice file afterwards: `del invoice.md`

Now in Claude: type `Make me a quote: 3 chairs at 120 each and 1 desk at 450, plus 20% VAT.`

Expected result: three steps, in this order:

1. Claude **writes** a JSON file with the items. Claude Code asks your permission to create the file: answer Yes;
2. Claude runs **`validate`**, and fixes the file if a problem is reported;
3. only then does Claude run **`total`**, and reports 810.00 + 162.00 VAT = 972.00.

To see the checklist and the order that make Claude do this: `type .claude\skills\calculating-invoices\SKILL.md`

Pattern D - data that is kept, and a skill only you can start

The problem it solves: Claude remembers nothing between sessions. Close Claude Code, and your tasks are gone, unless something saves them. A Python script can save them in a **file**.

Used by: `tracking-todos` saves tasks in `data\todos.json`. `clear-todos` deletes tasks, and only **you** can start it.

How it works:

- Every command of `todo.py` loads the file, changes the list, and saves it again:

  ```
  def load(path):
      if not path.exists():
          return []                                  # no file yet: an empty list
      return json.loads(path.read_text(encoding="utf-8"))

  def save(path, todos):
      path.write_text(json.dumps(todos, indent=2), encoding="utf-8")
  ```

- Which file? This line decides it:

  ```
  return Path(explicit or os.environ.get("TODO_FILE") or DEFAULT_FILE)
  ```

  It takes the `--file` option if given, otherwise the **environment variable** `TODO_FILE` if set, otherwise `data/todos.json`. **Why:** so tests and practice can use another file, and never touch your real list.

- Deleting is a **separate** skill, `clear-todos`, which Claude can't start (Step D5). **Why:** a misunderstood sentence must never delete your data.

Step D1 - Use a practice list, so your real list is not touched

What: you set the environment variable `TODO_FILE` for this cmd window.

Run:

```
set TODO_FILE=data\practice_todos.json
```

How it works: `set NAME=value` creates an environment variable in this cmd window only. Every program you start from this window, including `todo.py`, can read it with `os.environ.get("TODO_FILE")`.

Expected result: nothing is printed. Check it with `echo %TODO_FILE%`: it prints `data\practice_todos.json`.

Step D2 - Add two tasks and list them

Run:

```
python .claude/skills/tracking-todos/scripts/todo.py add "Buy milk"

python .claude/skills/tracking-todos/scripts/todo.py add "Send the report" --due 2026-10-01 --
    priority high

python .claude/skills/tracking-todos/scripts/todo.py list
```

Expected result (the last command; your `created` date is today's date):

```
{"file": "data\\practice_todos.json", "count": 2, "tasks": [{"id": 2, "text": "Send the report",
    "priority": "high", "due": "2026-10-01", "done": false, "created": "2026-10-20", "overdue":
    true}, {"id": 1, "text": "Buy milk", "priority": "normal", "due": null, "done": false, "created":
    "2026-10-20", "overdue": false}]}
```

How to read it:

- `"file": "data\\practice_todos.json"`: the practice file was used, thanks to `TODO_FILE`. (In JSON, a `\` is written as `\\`.)
- Each task got an `id` (1 and 2). Claude uses it later for `done 1`.
- The high-priority task is listed first, and its due date has passed, so `"overdue": true`. The SKILL.md tells Claude to mention overdue tasks first.

Step D3 - Look at the saved file

What: you look at the file the script wrote.

Why: to see where the skill's "memory" is.

Run:

```
type data\practice_todos.json
```

Expected result: (first task)

```
[
  {
    "id": 1,
    "text": "Buy milk",
    "priority": "normal",
    "due": null,
    "done": false,
    "created": "2026-10-20"
  },
  ...
```

How to read it: this file is the "memory" of the skill. It stays on disk after Claude Code closes, and the next session's `list` reads it again.

Step D4 - Finish a task, then delete finished tasks

Run:

```
python .claude/skills/tracking-todos/scripts/todo.py done 1

python .claude/skills/tracking-todos/scripts/todo.py clear --done-only
```

Expected result (the last command):

```
{"removed": 1, "remaining": 1}
```

How to read it: task 1 was marked done, and `clear --done-only` removed it. Task 2 remains.

Clean up the practice file and the variable (`set TODO_FILE=` with nothing after = removes it):

```
del data\practice_todos.json

set TODO_FILE=
```

Step D5 - See why Claude can't delete your tasks by itself

What: you read the SKILL.md of `clear-todos`.

Run:

```
type .claude\skills\clear-todos\SKILL.md
```

Expected result: (the important lines)

```
---
name: clear-todos
description: Deletes tasks from data/todos.json - finished tasks only by default, or every task with
    the "all" argument.
argument-hint: "[done|all]"
disable-model-invocation: true
...
The user's argument was: "$ARGUMENTS"

1. If the argument is `all`, run:
   python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --all
   Otherwise (empty or `done`), run:
   python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --done-only
```

How to read it:

Line	What it does	Why
`disable-model-invocation: true`	Claude may **never** start this skill by itself. Only you can, by typing `/clear-todos`	deleting must be your decision (Lesson 2, Step 6)
`argument-hint: "[done|all]"`	shown next to `/clear-todos` in the `/` menu	reminds you which words the skill accepts
`$ARGUMENTS`	replaced by the words you type after the name: `/clear-todos all` makes it `all`	the instructions choose the command from your word
`${CLAUDE_SKILL_DIR}/../tracking-todos/...`	uses the script of the neighbor skill. `..` means "one folder up"	one script for both skills, so the file format lives in one place. This works only while both skill folders are next to each other

Now in Claude: start `claude` and type `Delete all my todos.`

Expected result: Claude can't use `clear-todos`. The `tracking-todos` skill tells Claude to ask you to type `/clear-todos` yourself. Then type `/clear-todos done`: now the skill runs, because you started it.

Pattern E - pip packages from your venv

The problem it solves: some work needs packages that are not part of Python, for example pandas (tables, Excel) and matplotlib (charts). They are installed with `pip`, and in this course only in your venv.

Used by: `analyzing-excel` and `plotting-charts`. Lesson 7 explains this pattern in full. Here, try it.

How it works: the script imports the packages inside a `try`. If an import fails, it prints a JSON error with the fix, instead of crashing:

```
try:
    import matplotlib
    matplotlib.use("Agg")       # draw into files only: Claude runs the script without a screen
    import matplotlib.pyplot as plt
    import pandas as pd
except ImportError as missing:
    print(json.dumps({
        "error": f"Python package '{missing.name}' is not installed in {sys.executable}",
        "hint": "Claude Code was started without the skills' virtual environment. ...",
    }))
    sys.exit(2)
```

`sys.executable` is the full path of the Python that runs the script, so the message says **which** Python lacks the package. The file `scripts\requirements.txt` lists the packages, so `python -m pip install -r ...requirements.txt` installs them all.

Step E1 - Run with the activated venv

What: `check` reports which Python runs the script, and the package versions.

Run:

```
python .claude/skills/plotting-charts/scripts/chart.py check
```

Expected result:

```
{"python": "C:\\pyenvsmy\\claudeclioct26\\Scripts\\python.exe", "pandas": "3.0.6", "matplotlib":
    "3.11.2"}
```

How to read it: `python` is your venv's Python, and pandas and matplotlib are found there.

Step E2 - Run with the system Python (as if you forgot to activate)

What: the same command, but with the full path of the system Python.

Why: to see the error that Claude would get if you started `claude` without activating.

Run:

```
C:\Python312\python.exe .claude/skills/plotting-charts/scripts/chart.py check
```

Expected result:

```
{"error": "Python package 'pandas' is not installed in C:\\Python312\\python.exe", "hint": "Claude
    Code was started without the skills' virtual environment. Activate the venv, then start claude
    again. ..."}
```

How to read it: the `try` / `except ImportError` code above ran. The system Python has no pandas, and the message says what to do.

Step E3 - Draw a chart and open it

What: you draw a bar chart of the total amount per region.

Run:

```
python .claude/skills/plotting-charts/scripts/chart.py bar data/sales.csv --x region --y amount
```

Expected result:

```
{"written": "charts/bar_amount_by_region.png", "kind": "bar", "x": "region", "y": "amount", "agg":
    "sum", "points": [{"region": "North", "amount": 2150.0}, {"region": "East", "amount": 910.0},
    {"region": "West", "amount": 900.0}, {"region": "South", "amount": 670.0}]}
```

Run: (opens the picture in your image viewer)

```
start charts\bar_amount_by_region.png
```

How to read it: the script saved the picture (`"written"`) and also printed the numbers it drew (`"points"`). **Why both:** Claude can't see a picture by running a script. It reads the exact numbers from `points` to describe the chart, and can open the PNG to check it.

Part 3 - Bonus: a command that runs before Claude reads the skill

What: a SKILL.md can contain a line like `` !`ls data` ``. When the skill starts, Claude Code runs the command **first**, and puts its **output** in place of the line. Claude sees the result, not the command.

Why: for fresh facts that the instructions need, such as today's date or the list of files. Claude gets them without spending a turn on running a command.

Step 4 - Make a small test skill

Do: create the folder `.claude\skills\showing-data-files\`, and in it a file `SKILL.md` with this text:

```
---
name: showing-data-files
description: Shows today's date and the files in the data folder. Use when the user asks which data
    files exist.
allowed-tools:
  - Bash(date *)
  - Bash(ls *)
---

Today is !`date +%Y-%m-%d`.

Files in the data folder:
!`ls data`

Tell the user today's date and list the files above. Do not run any commands.
```

How it works: the two `allowed-tools` lines are needed: they allow the `date` and `ls` commands to run when the skill starts. `date` and `ls` are Bash commands, so this needs Git for Windows (Lesson 0). `date +%Y-%m-%d` prints the date as `2026-10-20`.

Step 5 - Use it

Run: start `claude` and type `/showing-data-files`.

Expected result (a real result):

```
Today is 2026-10-20.

Files in the data folder:
- article.txt
- invoice.json
- invoice_broken.json
- sales.csv
- sales.xlsx
```

How to read it: Claude didn't run any command itself: you see no Bash tool line. Claude Code ran `date` and `ls data` when the skill started, and Claude only read the results. Keep such commands fast, and use only commands that read, never ones that change files.

Step 6 - Remove the test skill

Type `/exit`, then (`rmdir /s /q` deletes a folder with its files, without asking):

```
rmdir /s /q .claude\skills\showing-data-files
```

Summary: which pattern to use

You need	Pattern	Example skill	Try again with
A few clear actions on one kind of input	A - one script, several commands	`converting-units`	`convert.py --help`
Many small functions, and adding new ones easily	B - call any function by name	`calculating-dates`	`dates.py --list`
Results where a mistake is expensive	C - check first, then calculate	`calculating-invoices`	`invoice.py validate ...`
Data that must survive between sessions, or a dangerous action	D - kept data, manual-only skill	`tracking-todos`, `clear-todos`	`todo.py list`
Python packages such as pandas	E - pip packages from the venv	`plotting-charts`	`chart.py check`

Rules that every skill script in this project follows (you saw each one above):

Rule	Why
1. It prints its result as **JSON** (Part 1, Step 1)	Claude reads named values reliably, and tests can check them exactly
2. It prints **errors as JSON with a hint**, and ends with exit code 1 (Step A3)	Claude can fix its call by itself, instead of guessing from a crash
3. It never opens a window or asks the user a question. It prints and ends	Claude runs it without a screen and can't answer questions. A script that waits for input would hang
4. It starts printing in UTF-8 (`sys.stdout.reconfigure(encoding="utf-8")`)	characters like ° then work on Windows (Lesson 0, section 6)
5. Its logic is in normal Python functions, and `main()` only connects them to the command line	tests and other programs can import and reuse them (Pattern B, Lessons 5 and 6)

Next: Lesson 4 - Designing an agent with many skills

Lesson 4 - Designing an agent with many skills

Goal: plan many skills so that Claude never confuses them, and understand why each design rule exists. You check each rule on this project's 9 skills.

Before you start: open cmd, `C:\pyenvsmy\claudeclioct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`.

The idea in one paragraph

With one skill, Claude only decides "skill or no skill". With many, it must also decide **which** one, from the names and descriptions alone (Lesson 2). Each extra skill makes that choice harder, and adds its description to every request. So a good set of skills has clear names, one job per skill, descriptions that don't share trigger words, and a small total size. This lesson shows how to check each of these.

Step 1 - List all skill names

What: you print the `name:` line of every SKILL.md.

Why: the names are part of Claude's menu. A clear name helps Claude choose, and helps you type `/name`.

Run: (`/s` searches all subfolders, `/b` keeps lines that begin with `name:`)

```
findstr /s /b "name:" .claude\skills\SKILL.md
```

Expected result:

```
.claude\skills\analyzing-csv\SKILL.md:name: analyzing-csv
.claude\skills\analyzing-excel\SKILL.md:name: analyzing-excel
.claude\skills\analyzing-text\SKILL.md:name: analyzing-text
.claude\skills\calculating-dates\SKILL.md:name: calculating-dates
.claude\skills\calculating-invoices\SKILL.md:name: calculating-invoices
.claude\skills\clear-todos\SKILL.md:name: clear-todos
.claude\skills\converting-units\SKILL.md:name: converting-units
.claude\skills\plotting-charts\SKILL.md:name: plotting-charts
.claude\skills\tracking-todos\SKILL.md:name: tracking-todos
```

How to read it: these names follow three naming rules:

Rule	Examples	Why
Skills that Claude chooses by itself use a verb with "-ing"	`analyzing-csv`, `calculating-dates`, `converting-units`	the name reads like an ability: "Claude is good at converting units"
The skill that only you start uses a command word	`clear-todos`	you type it like a command: `/clear-todos`
The folder name and the `name:` are the same	`analyzing-csv\SKILL.md` has `name: analyzing-csv`	only one name to remember. The linter warns when they differ (Lesson 2, Step 4)

Never use vague names like `helper`, `utils` or `tools`: they say nothing about the job.

Step 2 - Give every skill one job, with its own words

What: a planning table with one row per skill. Make it **before** you write many skills.

Why: two skills that answer the same kind of request compete. Then Claude picks one of them more or less by chance. The table makes such overlaps visible on paper.

How to fill it:

- **Job:** what the skill does, in two or three words.
- **Trigger words:** the words people type when they need this job. They must appear in the description.

- **Must NOT catch:** the neighbor job that could be confused with it. The description says this in its "Not for ..." sentence.

This project's table:

Skill	Job	Trigger words people type	Must NOT catch
converting-units	unit conversion	convert, how many X in Y, km, lb, °F, GiB	money and currencies
analyzing-csv	tables in .csv files	.csv, column, total per, top 5, average	Excel files, charts, prose
analyzing-excel	Excel workbooks	.xlsx, sheet, pivot, Excel	.csv files, charts
plotting-charts	pictures of data	chart, graph, plot, bar, line	plain number answers
analyzing-text	prose	words, readable, reading time, essay, article	tables
calculating-dates	calendar math	days until, deadline, working days, weekday	saving tasks
calculating-invoices	money totals	invoice, quote, VAT, discount, bill	unit conversion
tracking-todos	the task list	remind me, add a task, my list, mark done	deleting the list
clear-todos	deleting tasks	none: only you start it	everything Claude starts by itself

Check it on two neighbors. `analyzing-csv` and `analyzing-excel` are the most similar pair: both answer questions about tables.

Run:

```
findstr /b "description:" .claude\skills\analyzing-csv\SKILL.md .claude\skills\analyzing-
    excel\SKILL.md
```

Expected result: (the ends of the two lines)

```
...analyzing-csv\SKILL.md:description: Summarizes and aggregates tabular data in .csv files - ... Not
    for .xlsx workbooks (use analyzing-excel), charts (use plotting-charts), or prose .txt documents
    (use analyzing-text).
...analyzing-excel\SKILL.md:description: Reads Excel workbooks (.xlsx) with pandas - ... Not for .csv
    files (use analyzing-csv) or charts (use plotting-charts).
```

How to read it: each one names its **file type** (`.csv`, `.xlsx`). Each one also ends with a "Not for ..." sentence that sends the other file type to the other skill. That is how two similar skills stay separate: the file type in your request decides.

When two of your skills share trigger words, you have two choices:

Choice	When	Example
Merge them into one skill	they are really one job	`csv-stats` and `csv-totals` (Lesson 2, Step 4): overlap 1.00, the same job
Separate them clearly	they are different jobs	different nouns, different file types, and a "Not for ..." sentence, like the pair above

The linter measures this for you: `python tools/skills_lint.py` warns when two descriptions share too many words.

Split a skill into two when its instructions have parts that are never needed together, or when its SKILL.md grows past about 500 lines. **Why:** after Claude chooses a skill, its whole SKILL.md is loaded (level 2). Parts that are not needed only cost tokens.

Step 3 - Keep the menu small

What: you measure the total size of all descriptions.

Why: every description is in **every** request (level 1, Lesson 1). More text in every request means more allowance used, and a longer menu for Claude to choose from.

Run:

```
python tools/skill_sizes.py
```

Expected result: (the last lines)

```
Level 1 total (in EVERY request): 2,672 characters, about 668 tokens.
```

How to read it: 9 skills cost about 668 tokens per request, which is small. But 100 skills with 350-character descriptions would cost about 35,000 characters, nearly 9,000 tokens in every request.

Ways to keep it small:

Way	How	Why it helps
Short descriptions	200 to 400 characters, with the main job in the first sentence	enough for Claude to choose, nothing more
"Only you" skills	`disable-model-invocation: true`	they are not in the menu at all, so they cost nothing (`clear-todos` shows "only you" in the output)
Rarely used skills by name only	in `.claude\settings.json`: `{"skillOverrides": {"skill-name": "name-only"}}`	the description is left out of the menu, only the name stays
Long details in a second file	a link in SKILL.md to `reference.md`	Claude opens it only when it needs it (level 3)
Check with Claude Code	`/doctor` shows how much room the menu takes, and `/skill-doctor` lists skills you never use	you find the big and the unused skills

See the second-file way: `analyzing-csv` does it. Run:

```
findstr "reference.md" .claude\skills\analyzing-csv\SKILL.md
```

Expected result:

```
For the exact JSON shape of each command, see [reference.md](reference.md).
```

Put **all** such links in SKILL.md itself, and never link from `reference.md` to yet another file. **Why:** Claude sometimes reads only the beginning of a file it reaches through a second link.

Step 4 - Share Python code between skills

What: you look at how `clear-todos` uses the script of another skill.

Why: with many skills, some need the same Python code. You must decide where that code lives.

Run:

```
findstr "CLAUDE_SKILL_DIR" .claude\skills\clear-todos\SKILL.md
```

Expected result:

```
    python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --all
    python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --done-only
```

How it works: `clear-todos` has no script of its own. `${CLAUDE_SKILL_DIR}` is `.claude\skills\clear-todos`. `..` goes one folder up, to `.claude\skills\`, and from there into `tracking-todos`. This works only while both folders sit side by side.

Your options:

Option	How	Good for	The catch
Each skill has its own copy (the normal case)	every skill folder has its own `scripts\`	skills you give away one by one	a fix must be made in every copy
Use the neighbor's script	`${CLAUDE_SKILL_DIR}/../other-skill/scripts/x.py`, as above	skills that always stay together in one project	breaks if you copy one skill alone
An installed package	`pip install` your own package into the venv, and the skill runs `python -m mypackage ...`	the same code in many projects	needs the venv, like Pattern E

Whichever you choose, keep the logic in **functions** (Lesson 3, the rules at the end). The tests in `tests\` and the programs in `agent\` import exactly the functions that the skills run.

Step 5 - Big reading jobs go to a subagent

The problem: if a skill must read many files or long logs, all that text goes into your conversation. Claude re-reads the conversation for every new message, so every later message gets slower and more expensive.

The solution: a **subagent**, a helper Claude with its own context window. It does the reading there, and returns only a short summary to your conversation.

How: two lines in the frontmatter:

```
context: fork        # run this skill in a separate helper
agent: Explore       # the built-in helper that can read files but not change them
```

The helper doesn't see your conversation, so the skill's instructions must contain everything it needs.

Try it: Lesson 9, exercise 6 gives a complete skill, and shows how `/context` proves that your conversation stayed small.

Step 6 - Rules that must never be skipped

The problem: Claude follows a skill with judgment, and can skip a step, for example when it thinks a step is not needed.

The solution: some rules must hold **every** time, for example "never write outside `data\`". For such a rule, use a **hook**: a small program that Claude Code runs automatically before an action. If the hook says no, the action doesn't happen, and Claude can't skip it.

Try it: Lesson 9, exercise 8 has a tested hook.

Also: put the most important instructions **at the top** of a SKILL.md. **Why:** in very long conversations, Claude Code shortens old content and keeps only the first part of each loaded skill (about 5,000 tokens).

Step 7 - Use your skills in more places

What: where a skill folder can live, and what changes there.

Why: a good skill is useful in more than one project.

Run:

```
dir /b %USERPROFILE%\.claude\skills
```

How to read it: this is your personal folder (Lesson 1, Step 7). To use one of your skills in **every** project, copy its folder there.

Where	How	Note
This project	`.claude\skills\` in the project folder	goes with the project when you copy it
All your projects	`C:\Users\<you>\.claude\skills\`	only on this computer
Other people	package the skills as a **plugin**	started as `/plugin-name:skill-name`
claude.ai, Claude API	upload the skill folder	only 6 frontmatter fields are allowed there: `name`, `description`, `license`, `compatibility`, `metadata`, `allowed-tools`

Fields that only Claude Code understands (`disable-model-invocation`, `argument-hint`, `effort`, `context`, `paths`, ...) cause an error on claude.ai and the API. The linter lists them.

Run:

```
python tools/skills_lint.py
```

Expected result: look at the `clear-todos` lines:

```
[ok] clear-todos  (/command only, description 107 chars)
   INFO  Claude Code-only keys ['argument-hint', 'disable-model-invocation', 'effort'] (fine here;
    remove before uploading to claude.ai/API).
```

How to read it: `INFO` is not a problem here. It only reminds you to remove these three lines if you upload this skill to claude.ai.

Checklist before you add a skill

Check	Why	How to check
Its job doesn't overlap another skill's job	overlapping skills compete	your Step 2 table, and no overlap warning from `skills_lint.py`
Its description says what it does, has "Use when ...", and ends with "Not for ..." if a similar skill exists	the description is all Claude has for choosing	`skills_lint.py` warns when "Use when" is missing
Its name uses "-ing" if Claude may choose it, or a command word if only you start it	a clear name helps Claude and you	Step 1
If it deletes, publishes or costs money, it has `disable-model-invocation: true`	such actions must be your decision	Lesson 2, Step 6
Its SKILL.md is under 500 lines; long details are in a linked `reference.md`	the whole SKILL.md is loaded when chosen	`skill_sizes.py`
Its script prints JSON, gives errors with a hint, and has tests	Claude repeats what the script prints	`python -m unittest discover -s tests` (Lesson 5)
Its SKILL.md sets `effort:` `low` if the script does all the work, `high` if Claude must interpret the results	the same model thinks only as much as the job needs	`python agent\effort_probe.py --model opus "..."` (Lesson 6, Part 6)
`"${CLAUDE_SKILL_DIR}"` is in quotes, and `allowed-tools` is set	paths with spaces, and no permission question per run	`skills_lint.py`
You added test questions for it, and the routing test passes	only a real test shows what Claude chooses	`python tools/routing_eval.py` (Lesson 5)

Next: Lesson 5 - Testing skills that run Python

Lesson 5 - Testing skills that run Python

Goal: learn what can go wrong in a skill with Python code, and which test catches each problem. Then learn how to run each test, and how to read its result, including a failure.

Before you start: open cmd, `C:\pyenvsmy\claudeclioct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`.

Why test at all?

A skill with Python code has four parts, and each one can go wrong in its own way:

Part	What can go wrong	What you notice in Claude	The test that catches it	Cost
The Python script	it computes a wrong number	Claude tells you the wrong number, confidently	**1. Unit tests**	free
The `SKILL.md` file	a wrong name, a missing description, a script path that doesn't exist	the skill is never chosen, or fails, without a clear message	**2. Linter**	free
The description's words	it lacks the words people type	Claude doesn't choose the skill	**3. Preview** (a rough guess)	free
Claude's choice	Claude chooses another skill, or none	the wrong skill answers	**4. Routing test**	a little Pro allowance

Run the free tests 1 to 3 first, every time you change something. Run test 4 when they all pass.

Test 1 - Unit tests: does the Python code compute correctly?

Step 1 - Run all unit tests

What: `tests\test_skill_scripts.py` contains 74 unit tests. Each one runs one function of a skill script with a known input, and checks that the result is exactly the expected one.

Why: Claude trusts what a script prints. The unit tests make sure the scripts are right, so that Claude repeats right answers. They don't contact Claude, so they are free.

Run:

```
python -m unittest discover -s tests
```

How it works: `-m unittest` starts Python's built-in test runner. `discover` searches a folder for test files, and `-s tests` says which folder: `tests`. The runner opens every file named `test_*.py` there, and runs every function whose name starts with `test_`.

Expected result:

```
.....................................................................
----------------------------------------------------------------------
Ran 74 tests in 31.351s

OK
```

How to read it:

- Each `.` is one passed test. While the tests run, you see the dots appear one by one.
- `Ran 74 tests in 31.351s`: how many tests ran, and how long they took. The time doesn't matter.
- `OK`: **every** test passed.

If you see something else:

You see	Meaning	What to do
`F` among the dots, `FAILED (failures=1)`	a test got a wrong result	read the `FAIL:` block (Step 2)
`E` among the dots, `FAILED (errors=1)`	a test or a script crashed	read the `ERROR:` block. Its last line is the error message
`OK (skipped=6)`	the 6 tests that need pandas (Excel, charts, the sample workbook) were skipped	activate the venv (Lesson 0) and run again

Step 2 - Learn to read a failure (do it yourself)

What: you break the conversion script on purpose, see the tests fail, and read the failure.

Why: a failure report looks frightening the first time. After reading one carefully, you can read any of them.

Do:

1. Open `.claude\skills\converting-units\scripts\convert.py` in Notepad.
2. Find line 88. Among other units it contains `"mi": 1609.344` (one mile is 1609.344 meters).
3. Change only that number to `1600.0` (a wrong value), so the text becomes `"mi": 1600.0`. Save the file.

Run: (`-k ConvertingUnits` runs only the tests of the conversion script; Step 4 explains `-k`)

```
python -m unittest discover -s tests -k ConvertingUnits
```

Expected result (a real run):

```
..F.F
======================================================================
FAIL: test_cli (test_skill_scripts.ConvertingUnits.test_cli)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "E:\may10\claudecodecli_and_python\tests\test_skill_scripts.py", line 136, in test_cli
    self.assertEqual((code, out["result"]), (0, 1.609344))
AssertionError: Tuples differ: (0, 1.6) != (0, 1.609344)
...
======================================================================
FAIL: test_length (test_skill_scripts.ConvertingUnits.test_length)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "E:\may10\claudecodecli_and_python\tests\test_skill_scripts.py", line 119, in test_length
    self.assertAlmostEqual(self.convert.convert(100, "km", "mi")["result"], 62.137119, places=5)
AssertionError: 62.5 != 62.137119 within 5 places (0.36288100000000156 difference)

----------------------------------------------------------------------
Ran 5 tests in 0.152s

FAILED (failures=2)
```

How to read it, from top to bottom:

1. **`..F.F`:** 5 tests ran. Tests 3 and 5 failed (`F`), and the others passed (`.`).
2. **`FAIL: test_length (...ConvertingUnits.test_length)`:** the name of the failed test.
3. **`File "...test_skill_scripts.py", line 119`:** where the failed check is. The line below shows it: the test converts 100 km to miles and expects `62.137119`.
4. **`AssertionError: 62.5 != 62.137119`:** the most important line. The script returned **62.5**, but the right answer is **62.137119**. "Assertion" means "a check that must be true".
5. **`test_cli`** failed too. It runs the script **as a command**, exactly as Claude does. The command printed `1.6` instead of `1.609344` for 1 mile in km.

So the report tells you **which** test failed, **where** the check is, and **what** came out instead of what. The cause is your wrong mile value.

Now fix it: change the number back to `1609.344`, save, and run the same command again.

Expected result:

```
.....
----------------------------------------------------------------------
Ran 5 tests in 0.318s

OK
```

A second real example. On the course machine, one test once failed like this:

```
FAIL: test_skill_sizes_reports_levels
    (test_skill_scripts.TutorialTools.test_skill_sizes_reports_levels)
  File "E:\may10\claudecodecli_and_python\tests\test_skill_scripts.py", line 291, in
    test_skill_sizes_reports_levels
    self.assertEqual(proc.returncode, 0)
AssertionError: 1 != 0
```

Read it the same way. The test runs `tools\skill_sizes.py` and expects **exit code 0** (success). The program ended with **exit code 1** (an error). The cause: Python had saved compiled files (`__pycache__\*.pyc`) in the skill folders, and the tool tried to read them as text. The tool was fixed to skip them. The test now also prints the tool's error message when it fails, so the cause is visible directly in the report.

Step 3 - Look inside a test

What: two of the conversion tests, from `tests\test_skill_scripts.py`.

Why: when you write your own skill script, you write tests like these for it.

```
class ConvertingUnits(unittest.TestCase):
    convert = load("converting-units", "convert.py")      # import the skill's script

    def test_length(self):
        # call the function directly, and check the result
        self.assertAlmostEqual(self.convert.convert(100, "km", "mi")["result"], 62.137119, places=5)

    def test_cli(self):
        # run the script as a command, exactly as Claude does, and check the JSON it prints
        code, out = cli("converting-units", "convert.py", "convert", "1", "mi", "km")
        self.assertEqual((code, out["result"]), (0, 1.609344))
```

How it works:

- `class ConvertingUnits(unittest.TestCase)` is a group of tests for one script.
- `load(...)` imports the skill script, so the test can call its functions.
- `test_length` calls the function `convert(100, "km", "mi")` and checks the result. `assertAlmostEqual(..., places=5)` accepts tiny rounding differences after the 5th decimal place.
- `test_cli` runs `python convert.py convert 1 mi km` as a separate program, reads the JSON it prints, and checks the exit code (`0`) and the result (`1.609344`). This tests the command **exactly the way Claude uses it**.

Write both kinds for every skill script: one that tests the function, and one that tests the command.

Step 4 - Run only some tests, and see their names

What: `-k` runs only the tests whose name contains a word, and `-v` ("verbose") prints one line per test.

Why: when you work on one script, you only need its tests. That is faster, and the names show you what is being checked.

Run:

```
python -m unittest discover -s tests -k ConvertingUnits -v
```

Expected result:

```
test_aliases_and_temperature (test_skill_scripts.ConvertingUnits.test_aliases_and_temperature) ... ok
test_binary_data_units (test_skill_scripts.ConvertingUnits.test_binary_data_units) ... ok
test_cli (test_skill_scripts.ConvertingUnits.test_cli) ... ok
test_errors_explain_the_fix (test_skill_scripts.ConvertingUnits.test_errors_explain_the_fix) ... ok
test_length (test_skill_scripts.ConvertingUnits.test_length) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.318s

OK
```

How to read it: each line is `test name (file.group.test name) ... result`. The names say what is checked. For example, `test_errors_explain_the_fix` checks that a wrong unit gives an error message with a hint, the way Claude needs it.

The test groups in the file, to use with `-k`:

`-k` word	What it tests	Tests
`ConvertingUnits`	`convert.py`: lengths, temperatures, data sizes, error hints, the command	5
`AnalyzingCsv`	`csv_tool.py`: summary, totals per group, top rows, wrong column names	4
`CalculatingDates`	`dates.py`: working days, ages, calling functions by name, wrong dates	5
`AnalyzingText`	`text_stats.py`: counting words and sentences, frequent words, text from a pipe	4
`CalculatingInvoices`	`invoice.py`: totals with cent rounding, the 5 problems of the broken invoice, the document	3
`TrackingTodos`	`todo.py`: add, list, done, clear, wrong priority (with a temporary list, never your real one)	2
`PackageSkills`	`excel_tool.py`, `chart.py` and `tools\make_sample_xlsx.py` (need the venv)	6
`WrongInterpreter`	the helpful message when the venv is not active	1
`TutorialTools`	the tools in `tools\`: linter, preview, skill sizes, `skilllib.py`, the hook of Lesson 9, `show_sdk_command.py`	10
`CalculatingMath`	`examples\skills\calculating-math\scripts\calc.py`: the strict calculator of Lesson 8	5
`AgentGuard`	`agent\guard.py`: which tool calls `guarded_agent.py` allows (Lesson 6, Part 4)	5
`AskClaudeCommand`	`agent\ask_claude.py`: the `claude -p` command it builds, `--effort`, thinking tokens, the flags of `--strict` (Lesson 6, Part 5); `tools\run_cost.py`: the steps of a run (Part 6)	5
`BothWays`	the `claude -p` way (`--cli`) of the agent programs: their commands, the hook scripts `guard_hook.py` and `effort_hook.py`, and the tool server `tutor_mcp_server.py`. Also the routing rules that both ways of `routing_eval.py` share, the SDK options of `run_cost.py`, the modes of `tools\model_mode_compare.py`, and the profiles of `tools\harness_profile_compare.py`. One test blocks every import of the SDK, to prove that the `--cli` ways don't need it (Lesson 6, Part 4)	19

Every `.py` file names its own tests. The `HOW TO TEST` section at the top of each file gives the exact `-k` words, with the expected output. It also gives a check by hand, and one with Claude. For example, `convert.py` says `-k ConvertingUnits`, and `agent\guard_hook.py` says `-k guard_hook`. Open any script in Notepad and read its first lines.

Step 5 - See what happens without the venv

Run:

```
C:\Python312\python.exe -m unittest discover -s tests
```

Expected result (the end):

```
Ran 74 tests in 11.987s

OK (skipped=6)
```

How to read it: the system Python has no pandas, so the 6 `PackageSkills` tests can't run. They are **skipped**, not failed, because a missing pandas is a setup question, not a bug. With `-v` you see the reason next to each one: `skipped "activate the skills' venv (pandas, openpyxl, matplotlib) to run these tests"`. Where the Agent SDK is missing too, 2 more tests are skipped (`skipped=8`). They check `tools\show_sdk_command.py` and the SDK way of `tools\run_cost.py`.

Test 2 - The linter: are the SKILL.md files correct?

What: `tools\skills_lint.py` reads every `SKILL.md` and checks it against the rules of this course.

Why: Claude Code doesn't warn you about most mistakes in a `SKILL.md`. The skill just isn't chosen, or its command fails. The linter finds the cause in a second.

Run:

```
python tools/skills_lint.py
```

Expected result (shortened):

```
[ok] analyzing-csv  (auto + /command, description 372 chars)
   INFO  Claude Code-only keys ['effort'] (fine here; remove before uploading to claude.ai/API).
...
[ok] clear-todos  (/command only, description 107 chars)
   INFO  Claude Code-only keys ['argument-hint', 'disable-model-invocation', 'effort'] (fine here;
    remove before uploading to claude.ai/API).
...
Description overlap (pairs that may compete for the same request):
   none above 0.30 - descriptions are well separated

9 skills, 0 error(s).
```

How to read it:

- One block per skill. `[ok]` means no problem. `auto + /command`: Claude and you can start it. `/command only`: only you can.
- `ERROR` lines must be fixed. `WARN` lines are advice. `INFO` lines are only information.
- "Description overlap" compares every pair of descriptions. If two share too many words, you get a `WARN` line, because Claude could mix them up.

What it checks: the name format, and that a description exists and is not too long. It checks that the description is written about the skill (not "I can..."), and that it has a "Use when ..." sentence. It also checks the quotes around `"${CLAUDE_SKILL_DIR}"`, `$1`-style placeholders, that every script and file mentioned exists, and that every `.py` file is valid Python.

Try it on broken skills: `python tools/skills_lint.py tools/demo_bad_skills` prints 3 errors and an overlap warning. Lesson 2, Step 4 explains each one.

Try breaking a real skill: open `.claude\skills\converting-units\SKILL.md`. Remove the two " around `${CLAUDE_SKILL_DIR}/scripts/convert.py` on line 17, save, and run the linter. You get a `WARN ... quote the path` line. Put the quotes back, and run it again to see `0 error(s)`.

Test 3 - The preview: a free, rough guess of the choice

What: `tools\route_preview.py` guesses which skill fits a request by counting shared words.

Why: it gives a quick warning before you spend allowance. If a typical request shares **no** words with your skill's description, the description probably lacks the words people type.

Run:

```
python tools/route_preview.py "How many km is a marathon?"
```

Expected result:

```
Request: How many km is a marathon?

   3.55  converting-units       shared words: km, many
   1.24  calculating-dates      shared words: many
   0.00  tracking-todos         shared words: -

  (not scored, /command only: clear-todos)
```

How to read it: the number is a score: higher means more shared words. `converting-units` shares "km" and "many", so it is first. `clear-todos` is not scored, because Claude never chooses it by itself (only you start it, with `/clear-todos`).

Its limit: it only counts words. Claude understands meaning, so the preview can be wrong where Claude is right (Lesson 2, Step 3 shows a case). The preview also ignores the "Not for ..." sentence, because a word counter can't understand "not". That is why that sentence must be the **last** sentence of a description: the tool skips everything from "Not for" to the end.

Test 4 - The routing test: what does Claude really choose?

Step 6 - Look at the test questions

What: `tools\routing_cases.json` is a list of questions, each with the skill you expect.

Run:

```
type tools\routing_cases.json
```

Expected result (4 of the 20 lines):

```
  {"prompt": "Convert 26.2 miles to kilometers.", "expect": "converting-units"},
  {"prompt": "Which region sold the most in data/sales.csv?", "expect": "analyzing-csv"},
  {"prompt": "Delete all my todos.", "expect": ["tracking-todos", null], "note": "clear-todos is
    manual-only, so Claude must not pick it"},
  {"prompt": "What is the capital of Australia?", "expect": null},
```

How to read it: `prompt` is the question, and `expect` is the skill Claude should choose. `null` means "no skill". A list like `["tracking-todos", null]` means "either is fine".

Why these kinds of questions:

1. **Normal questions** check that each skill is found.
2. **Borderline questions** check that similar skills are told apart. For example, "How many bytes are in 3 GiB?" belongs to unit conversion, not data analysis. Questions for "only you" skills check that Claude does **not** choose them.
3. **"No skill" questions** check that descriptions don't grab too much.

Step 7 - See what will run, for free

Run:

```
python tools/routing_eval.py --dry-run --limit 2
```

Expected result:

```
C:\Users\Workstation\.local\bin\claude.EXE -p "Convert 26.2 miles to kilometers." --output-format
    stream-json --verbose --setting-sources project --no-session-persistence
C:\Users\Workstation\.local\bin\claude.EXE -p "How many pounds is 70 kg?" --output-format stream-json
    --verbose --setting-sources project --no-session-persistence
```

How it works: `--dry-run` only prints the commands; it runs nothing. Each question becomes one `claude` command in headless mode:

Part	Meaning
`-p "..."`	answer this one question without the interactive window
`--output-format stream-json`	report each step as it happens, as one JSON line
`--verbose`	report all steps (needed for `stream-json`)
`--setting-sources project`	load only this project's skills, so your personal skills don't change the test
`--no-session-persistence`	don't save the test runs in your session list

The script reads the JSON lines. As soon as one says that Claude used the **Skill** tool, for example `{"type": "tool_use", "name": "Skill", "input": {"skill": "converting-units"}}`, it notes the skill and **stops Claude**. Claude doesn't need to finish its answer, so each question is cheap. If Claude answers without the Skill tool, the result is "(no skill)".

Step 8 - Run it for real

Why: only this test shows what **Claude** really chooses.

Run: (it uses a little of your Pro allowance)

```
python tools/routing_eval.py --limit 6
```

Expected result (a real run of 6 table and chart questions):

```
Auth: claude.ai subscription (uses your Pro/Max allowance, no API billing)
Model: claude-opus-5-5

PASS  What are the average and maximum amounts in data/sales.csv  got=analyzing-csv
    want=analyzing-csv
PASS  Which region sold the most in data/sales.csv?               got=analyzing-csv
    want=analyzing-csv
PASS  Which sheets and columns are in data/sales.xlsx?            got=analyzing-excel
    want=analyzing-excel
PASS  Build a pivot of amount by region and product from data/sa  got=analyzing-excel
    want=analyzing-excel
PASS  Make a bar chart of total amount per region from data/sale  got=plotting-charts
    want=plotting-charts
PASS  Plot the monthly sales amount from data/sales.xlsx as a li  got=plotting-charts
    want=plotting-charts

6/6 routed as expected in 32s.
Details: tools\routing_report.json
```

How to read it:

- `Auth: claude.ai subscription`: your Pro plan is used, not an API key.
- Each line: `PASS` or `FAIL`, the question (shortened), the skill Claude chose (`got=`), and the skill you expected (`want=`).
- `6/6 routed as expected`: all matched. The details of every run are saved in `tools\routing_report.json`.

Other forms: `--model haiku` runs on the small Haiku model, which uses less of your allowance. Without `--limit`, all 20 questions run.

The same test through the Agent SDK: add `--sdk`. Each question then gets its own `ClaudeSDKClient`, with Claude Code's own instructions (`system_prompt={"type": "preset", "preset": "claude_code"}`). Without this option, the SDK sends no instructions, and the choice could differ. Both ways use the same stop rules, in `read_event()`. In a real run of the first 3 questions on Haiku, the SDK way passed 3 of 3, and `claude -p` 2 of 3. Once, Haiku answered "How many bytes are in 3 GiB?" without the skill. One run each can't name a winner. When a question is close, run it a few times.

What to do when a routing question fails: a real example

1. Read the `FAIL` line: which skill was expected, and which was chosen.
2. Change **one** description (Lesson 2, Step 5).
3. Run **all** questions again, because a change can fix one question and break another.

The first real run of this course gave 4 of 5:

```
FAIL  Convert 26.2 miles to kilometers.              got=(no skill)        want=converting-units
PASS  Which region sold the most in data/sales.csv?  got=analyzing-csv     want=analyzing-csv
PASS  If a job starts on 2026-10-15, what date is... got=calculating-dates want=calculating-dates
PASS  Delete all my todos.                           got=tracking-todos    want=tracking-todos | (no
    skill)
PASS  What is the capital of Australia?              got=(no skill)        want=(no skill)
```

Why it failed: the question is easy, so Claude answered from memory without the skill. The description only said *"Use when the user asks to convert a number..."*.

The fix, one changed sentence, which says the skill is for easy cases too, and why:

```
- ... MB to GiB). Use when the user asks to convert a number from one unit to another or asks "how
    many X in Y".
+ ... MB to GiB) with exact factors. Use for every unit conversion request, even simple ones like
+ miles to kilometers, and whenever the user asks "how many X in Y".
```

(In a "diff", `-` marks the old line and + the new lines.)

The result: all 3 unit questions passed. The "no skill" question still passed too. A description that said "use for everything" would have broken that one.

Next: Lesson 6 - Your own agent program with the Claude Agent SDK

Lesson 6 - Your own agent program with the Claude Agent SDK

Goal: run Claude Code, with its skills, from your own Python program. You will also give Claude your own Python functions as tools, and ask Claude questions from Python, with only your Pro login. Part 4 does the same jobs with `claude -p` and with the SDK, so you see when each one is the better choice. Part 5 shows how to set the reasoning effort, to use less of your allowance. It also translates a typical Anthropic API call into Pro code (Step 20). Part 6 shows how each skill sets its own effort, with the same model.

Before you start: open cmd, `C:\pyenvsmy\claudeclioct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`. Run every command of this lesson in that window.

The idea in one paragraph

Until now **you** typed into Claude Code. In this lesson **your Python program** does it. There is no second way to reach Claude here. With only a Pro plan (no API key), every request goes through the Claude Code program, which holds your login. Your Python program starts Claude Code **in the background**, sends it a question, and reads back each step. The **Claude Agent SDK** is a Python package that does this for you. `claude -p` is the same thing without the package.

The big idea: Claude Code instead of a direct model call

What: your Python agent sends its requests for the model to Claude Code (`claude -p` or the Agent SDK) instead of to the model's API. That is more than a way to spend your Pro allowance on every request, not only on coding. Claude Code is a **harness** around the model, so the same request can come back **more capable**. And you can **trim** the harness for each request, so a simple one stays cheap.

Why: you get three gains in one, and each is measured in this lesson:

Gain	What your agent gets	Measured in this lesson
1. The door	your Pro login for every model request of your programs, coding or not, within your plan's usage limits (Anthropic Help Center (https://support.claude.com/en/articles/15036540-use-the-claude-agent-sdk-with-your-claude-plan)); the same model names; with an API key at work, the same programs run unchanged	Part 3: `ask_claude.py` answers from Python with only Pro
2. The harness: stronger answers to hard requests	the agent loop (act, check the result, act again), tools (read the files itself, compute exactly with Python), your skills, context management, rules and sessions	a 3,000-row table: a plain call, adding up in its head, was right 1 time out of 4, in about 2 minutes on average. Claude Code ran a few lines of Python. It was right 4 times out of 4, in 17 seconds, at a tenth of the cost per right answer (below)
3. The dial: only as much harness as the request needs	switch off what a request doesn't use: your account's MCP connectors, the built-in tools, settings and memory files, extra turns	one plain question: 39,143 input tokens with everything on, 849 with everything off; one computing job: 70 to 80% lower cost per right answer than everything on, with no fewer right answers

The harder the request, the larger the gain. This test asks the same two questions as "Three questions", below: the total per region to the cent, and the number of orders over 500. It asks them on longer tables of made-up orders, on Haiku: three runs per question at 300 rows, two at 1,000 and 3,000:

```
python tools/harness_profile_compare.py --model haiku --runs 2 --tasks big --rows 3000 --profiles
    model,lean_py,full
```

Rows	Plain call	Claude Code, everything on	Claude Code, lean, with one compute rule
300	right 5 of 6, 26 s, $0.0046 per right answer	6 of 6, 12 s, $0.0043	6 of 6, 10 s, $0.0012
1,000	3 of 4, 55 s, $0.0118	4 of 4, 13 s, $0.0056	4 of 4, 11 s, $0.0023
3,000	1 of 4, 117 s, $0.0858	4 of 4, 17 s, $0.0083	3 of 4, 18 s, $0.0095

How to read it:

- **The plain call gets slower, dearer and less right as the table grows.** At 3,000 rows, one set of totals took it 5.6 minutes and about 112,000 output tokens, and was still wrong. Another time it answered "About 1,800" instead of 1,346. Twice, at 1,000 and 3,000 rows, it said honestly that it couldn't add up that many rows by hand.
- **Claude Code stays fast and cheap,** because Python adds up 3,000 rows as quickly as 300: 10 to 18 seconds, under a cent per right answer. At 3,000 rows, everything on was right every time at a tenth of the plain call's cost per right answer.
- **At 3,000 rows, everything on beat the lean profile.** The lean profile lost one run to its own restriction: Claude wrote a command that the python-only rule refused. Trim most for easy requests. For hard ones, keep what the job may need.
- **A long question needs stdin on Windows.** A command line holds at most 32,767 characters, so a 3,000-row table pasted into `claude -p "..."` fails with "The filename or extension is too long". The tool sends every question longer than 8,000 characters through stdin, as Step 12 does with a pipe.

The warning: don't fit cheap tires to an expensive car. The harness is designed and tested as a whole. Taking parts out can cost more than it saves:

- **In this course:** allowed to run only python commands, Claude Code wrote `awk` and PowerShell instead, which the rule refused. It was right 3 times out of 6, not 6 ("Three questions", below).
- **In Hindsight**, an open-source memory system for agents: its one-turn calls allowed no tools but didn't remove them, so they failed now and then. Without a pinned model, Claude Code ran a different model than Hindsight's logs showed ("What others found when they switched parts off", below).
- **In Missy**, another agent: with the tools off but Claude Code's prompt kept, the model tried to call tools that didn't exist ("What others found when they switched parts off", below).
- **At Anthropic itself:** three changes in the harness, not in the model, lowered Claude Code's quality (the postmortem, below).

The rule: switch off only what the request doesn't use, and keep what it does. For example, Sigil, an agent framework, answers its model calls with no tools, but gives its web searches exactly one tool, WebSearch. Tell Claude in one sentence what it should use ("compute every number with Python"), and pin the model. Then measure right answers and cost per right answer, before and after, on your own requests.

Not a strange idea: an established practice.

- **Anthropic built its own agents this way:** it renamed the Claude Code SDK to the Agent SDK, because the harness behind Claude Code can power other agents. It runs almost all of its major agent loops on it (sources below).
- **The Agent SDK is downloaded tens of millions of times a month.** The Python package `claude-agent-sdk` had 14.2 million downloads in May 2026 and 29.9 million in September. That is about one for every five downloads of Anthropic's plain API package (`anthropic`, 146.4 million in September) (PyPI Stats: claude-agent-sdk (https://pypistats.org/packages/claude-agent-sdk), anthropic (https://pypistats.org/packages/anthropic)). The npm package `@anthropic-ai/claude-agent-sdk` had 44.9 million in the month to 22 October 2026 (npm

(https://www.npmjs.com/package/@anthropic-ai/claude-agent-sdk). Downloads count installs, also by automatic builds, not people. But the growth is clear.

- **Well-known open-source projects plug Claude Code in where a model API would go:** OpenClaw (about 393,000 GitHub stars), Hindsight (about 54,000), promptfoo (about 26,000), OpenHarness (about 16,000), the Vercel AI SDK through a community provider, and more. The list, with what each keeps of the harness, is in "Who else builds agents this way", below.
- **OpenAI does the same with its coding agent:** the Codex SDK runs Codex from your programs.

What this doesn't prove: that a harness wins on every request. On small, easy requests a plain call was just as right, and cheaper ("Three questions", below). Use the harness where the request is hard, and the dial where it is easy.

Why go through Claude Code at all: the harness around the model

What: an API call gives you **the model**: text in, text out, one answer per request. Claude Code gives you the same model **inside a harness**. The harness is the program around the model: it runs the agent loop, gives the model tools, manages its context, and enforces your rules. With `claude -p` or the SDK, your program gets that whole harness, not only the model.

Why it matters: with the API alone, you would build each of these parts yourself, and test them yourself. With Claude Code, they are already built, maintained, and used every day by a very large number of developers.

A plain API call gives you	Claude Code adds around the same model	Where you used it
one answer to one request	the **agent loop**: gather context, act with a tool, check the result, repeat until the job is done	Lesson 6, Step 4 (5 turns for one question)
tools only if you define and run them yourself	**built-in tools**: read, write, edit and search files, run commands	Lesson 3, Part 1
no skills	**skills**: found on disk, offered by their descriptions, loaded only when chosen	Lesson 1, Step 5
the whole history sent again; you trim it yourself	**context management**: prompt caching, compaction near the limit, `CLAUDE.md` memory, files read only when needed, subagents with a clean context	Lesson 4, Step 5; Lesson 6, Step 28
no safety layer	**permissions and hooks**: allowed tools, refused calls, your rules enforced outside the model	Lesson 6, Step 15
no memory between calls	**sessions** that your program can resume	Lesson 6, Step 14
integrations that you write	**MCP**: tool servers plugged in with one option	Lesson 6, Step 8
a token count	**observability**: every step as JSON, the transcript (Ctrl+O), hook events	Lesson 0, section 7

The evidence, from Anthropic and from public benchmarks:

- **Anthropic builds its own agents on this harness.** In September 2025, it renamed the Claude Code SDK to the Claude Agent SDK. The reason: "the agent harness that powers Claude Code" can power many other kinds of agents. Anthropic also reported that this harness already runs almost all of its major agent loops (Building agents with the Claude Agent SDK (https://claude.com/blog/building-agents-with-the-claude-agent-sdk), 29 September 2025).
- **It is used at a very large scale.** Claude Code passed 500 million dollars in run-rate revenue within months of its full launch. Its usage grew more than tenfold in three months (Anthropic, 2 September 2025 (https://www.anthropic.com/news/anthropic-raises-series-f-at-usd183b-post-money-valuation)).

- **Anthropic's engineers explain why the model alone is not enough.** Context is a limited resource: the more a model must hold, the worse it recalls. A good harness keeps the context small and useful. Claude Code compacts a long conversation, and loads files with Glob and Grep only when they are needed. It reads `CLAUDE.md` as memory, and gives side jobs to subagents (Effective context engineering for AI agents (https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents), 29 September 2025). For work that spans many sessions, they add a progress file, a feature list and git on top of the harness (Effective harnesses for long-running agents (https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents), 26 November 2025).
- **The harness changes results as much as a model does.** On Terminal-Bench 2.0, third-party reports compiled from the public leaderboard put the same model, Claude Opus 4.6, between about 58% and 82%. Only the harness around it was different (Harness performance on Terminal-Bench (https://codex.danielvaughan.com/2026/04/09/harness-performance-terminal-bench/)). A test of one model in four harnesses found a 13-point spread (Same model, different harness (https://kimchi.dev/blog/same-model-different-harness-terminal-bench), September 2026, by a harness vendor). A June 2026 research survey concludes that an agent's quality comes from the model and the harness together (arXiv 2606.20683 (https://arxiv.org/abs/2606.20683)).

Be fair about it:

- On those benchmarks, Claude Code was **not** the best harness for every model: specialized harnesses scored higher. Its strength is that it is general, mature, and tested by daily use. It is also available to you with a Pro login, ready for your own skills, rules and tools.
- The harness costs context. Even a plain question carried 4,309 input tokens of Claude Code's own text (Step 20). A short skill run read 110,413 tokens of context again (Step 28). For a pure question and answer, use the "plain LLM" settings of `agent/ask_claude.py`: no tools and a short system prompt. With `--strict`, a question costs about 850 tokens instead of 4,300 ("Using Claude Code only as the model", below).

What it opens up: a single API call can only answer. Through the harness, your program can hand Claude a **job**, for example "build the pivot of this workbook and save the report". Claude finds the file, chooses the skill, runs your Python, checks the result, and stays inside your rules. Meanwhile, your program watches every step. That is the difference between a model and an agent. It is also why every program in this lesson goes through Claude Code.

When the harness is the right choice, and when it isn't

What: this section shows who else calls Claude Code (or a similar agent program) instead of the model's API, and the two very different ways to do it. It also answers whether Claude Code is only for code, when a plain API call is better, and what Anthropic's terms allow with a Pro login.

Why: a harness is not free: it costs context, time and, at work, a license. You should know when it pays off, and that you are not the only one who builds agents this way.

Who else builds agents this way

Open-source projects that call Claude Code where a model API would go (GitHub stars on 26 October 2026; "harness kept" says how much of Claude Code each one uses):

Project	What it is	How it uses Claude Code	Harness kept
OpenClaw (https://docs.openclaw.ai/gateway/cli-backends), about 393,000 stars	an agent gateway	runs its agent turns through a local Claude Code or Gemini CLI, as the main runtime or as a fallback. OpenClaw keeps the sessions, and gives Claude Code its own tools through an MCP bridge (`mcp__openclaw__<name>`, like the `mcp__tutor__` tools of Step 8)	the loop
Hindsight (https://github.com/vectorize-io/hindsight), about 54,000	memory for agents	the Agent SDK as the model of its own memory work, with a Pro or Max login: one turn, no tools, no settings files	model only
promptfoo (https://www.promptfoo.dev/docs/providers/claude-agent-sdk), about 26,000	a tool for testing LLM applications	the provider `anthropic:claude-agent-sdk`, alias `anthropic:claude-code`: no tools by default, read-only file tools when you give it a working folder. It can run on a Claude Code plan	both
OpenHarness (https://github.com/HKUDS/OpenHarness), about 16,000	an agent framework	its personal assistant, ohmo, runs on an existing Claude Code or Codex subscription through a CLI bridge, with no API key	the runtime
NodeTool (https://github.com/nodetool-ai/nodetool), about 600	a visual builder for AI workflows	`ClaudeAgentProvider` has both ways: a "pure LLM" call (one turn, tools and settings off, its own prompt), and a loop mode in which Claude Code runs its own agent loop over NodeTool's tools	both
ai-sdk-provider-claude-code (https://ai-sdk.dev/v5/providers/community-providers/claude-code), about 370; 193,000 npm downloads a month	a community provider listed in the Vercel AI SDK documentation	Claude models through the Agent SDK and the Claude Code CLI, with a Pro or Max login or an API key	the model
pi-claude-cli (https://github.com/rchern/pi-claude-cli), about 110	a model provider for the pi agent	every request is a `claude -p` run; Claude asks for tools, and pi runs them	the model
dsh-claude-cli (https://github.com/katsos/dsh-claude-cli) and dsh-oh-my-claude (https://github.com/lcestou/dsh-oh-my-claude), small	two providers for the same agent, DeepSeek Harness	the first switches off Claude Code's loop, tools, settings, memory files and MCP; the second keeps its sessions, hooks, CLAUDE.md files and MCP servers	opposite choices
Agentry (https://github.com/aweussom/agentry), small	an OpenAI-compatible API on your own computer	your OpenAI client code talks to a persistent `claude -p` process, with the built-in tools and MCP off	the model

Project	What it is	How it uses Claude Code	Harness kept
Sigil (https://sigilagent.com/reference/claude-code.html), small	an agent framework that compiles skills into procedures	a headless `claude -p` run on your subscription answers every model tier (compiler, executor, router, chat), with no API key. A model answer gets one turn and no tools; a web search gets exactly one tool, WebSearch	the model, plus one tool
Build A Harness (https://github.com/3IVIS/buildaharness), small	a canvas for agent governance, with a personal assistant, Aielia	`ASSISTANT_LLM_BACKEND=claude-cli` uses a `claude` login instead of an API key	the model
Voicetest (https://dev.to/pld/using-claude-code-as-a-free-llm-backend-for-voice-agent-testing-40bg)	a tool for testing voice agents	`claude -p --output-format json` as a plain LLM backend for simulations and judges, on a Pro or Max plan. Its author notes that one call runs at a time, so large parallel test suites are better on an API	the model

How to read it: the large projects show that the pattern is used, not only discussed. They also show both ways. Some keep only the model and run everything else themselves; others hand Claude Code the loop. The two DeepSeek Harness providers make opposite choices for the same parent agent. They are a good pair to compare.

The vendors themselves:

- **Anthropic** lists finance agents, personal assistants, customer support agents and deep research agents as examples of what to build with the Agent SDK (Building agents with the Claude Agent SDK (https://claude.com/blog/building-agents-with-the-claude-agent-sdk)). Its documentation tells programs in other languages to run `claude -p` as a subprocess with JSON output (Agent SDK overview (https://code.claude.com/docs/en/agent-sdk/overview)). It now also sells the harness itself. **Managed Agents** is a hosted agent harness. Anthropic offers it next to the Messages API, which gives "direct model prompting access" for your own agent loop (Claude Managed Agents (https://platform.claude.com/docs/en/managed-agents/overview)). And it built **Cowork**, its desktop agent for non-coding knowledge work, with the same agentic approach as Claude Code (Claude Cowork (https://claude.com/blog/cowork-research-preview)).
- **OpenAI** does the same with its own coding agent. The Codex SDK controls local Codex agents from your program, for internal tools, workflows and your own applications. Its Python version ships with the Codex CLI inside (Codex SDK (https://learn.chatgpt.com/docs/codex-sdk)). Like Anthropic, OpenAI presents it mainly for engineering work.

Independent voices:

- **Simon Willison**, an independent developer with a widely read blog about LLM tools, wrote in October 2025 that Claude Code is poorly named. In Willison's view, it is not only a coding tool, but a tool for general computer automation (Claude Skills are awesome, maybe a bigger deal than MCP (https://simonwillison.net/2025/Oct/16/claude-skills/)). Barry Zhang of Anthropic said that skills came out of a prototype that showed Claude Code is a general-purpose agent (quoted by Simon Willison (https://simonwillison.net/2025/Oct/16/barry-zhang/)).
- **An independent comparison** (MindStudio, May 2026) gives the same rule as this lesson. Use the plain API for tasks that finish in one call, with low overhead and full control. Use the Agent SDK for agents that act through a number of steps that nobody knows in advance (What is the Claude Agent SDK? (https://www.mindstudio.ai/blog/what-is-claude-agent-sdk-vs-claude-api)).

What nobody has shown yet: a controlled study in which calling Claude Code beats calling the API for every job. The evidence shows that the pattern works, that large projects and both vendors use it, and that the harness changes results. This course's own measurement shows where it wins and where it doesn't ("Three questions", below).

Two ways to use Claude Code from your agent: as the model, or as the runtime

	As the model	As the runtime
What you send	one question	a job: "find the files, compute, check, report"
Who runs the loop	your Python	Claude Code
Tools, skills, context management, hooks	switched off, or not used	they do the work
In this course	`agent/ask_claude.py`, `agent/api_equivalent.py`	`agent/run_agent.py`, `custom_tools_agent.py`, `guarded_agent.py`
What you gain over the API	mostly the door: your Pro login, nothing else to install	the whole harness

The large gain is in the second column. It is biggest when your program **can't know the steps in advance**. Claude Code then finds what it needs, chooses the next step, looks at the result, and changes its plan.

Decide who owns the loop. Your Python program may already plan, call tools and retry. If it hands each small step to a Claude Code run that also plans and calls tools, it builds a loop inside a loop. That costs time and tokens. Choose one: keep your loop and use Claude Code as the model, or hand Claude Code a whole sub-job and let your program supervise it.

For a general-purpose agent, adapt three things:

1. **Your own instructions,** not Claude Code's default coding persona: `system_prompt=` in the SDK, or `--system-prompt` in `claude -p` (Step 2, Step 10).
2. **Your business tools, through a controlled door:** an MCP server (`tutor_mcp_server.py`, Step 8) for calendars, databases or tickets, because files and commands alone don't reach them. And a hook (Step 15) for what may never happen.
3. **Skills for the know-how,** with scripts for everything that must be exact (Lesson 3).

Is Claude Code only a harness for writing code?

Its **defaults** are made for software work. Anthropic describes the Claude Code product as built for software engineering, and its long built-in instructions are about code. The **harness** underneath is general: the loop, the file and command tools, skills, hooks, sessions and MCP. You use it for other work by giving it your own instructions and tools. This course does exactly that: none of its 9 skills writes code. They analyze sales data, compute dates and invoices, keep a to-do list, and draw charts.

The real limit is a different one: the harness works **through files and commands on a computer**. Skills need a file system and a place to run code, a point Simon Willison makes too. A job that is only text in and text out doesn't need any of it.

When to use which

Your job	Better choice	Why
Answer a question, write a short email from full instructions, translate, classify, choose an action	a plain call: Claude Code as the model, or the API	no loop is needed. Even with the tools off, Claude Code adds about 4,300 tokens of its own text (about 850 with `ask_claude.py --strict`), and a few seconds to start
A database update that is fully specified in advance	your own Python, without a model	nothing to decide, so nothing for a model to do
Thousands of small requests, or answers for hundreds of users at once	the API, with an API key	every `claude -p` run starts a program and carries Claude Code's own text, one call at a time, under your plan's limits
Analyze several spreadsheets, find what doesn't match, and write a report	Claude Code as the runtime	exploring, running scripts and checking results is what the loop is for
Research across many documents	the runtime, with search tools	it reads only what it needs, and keeps notes
A multi-step office workflow across business systems	the runtime, with MCP tools and hooks	your tools do the actions; the hooks keep the rules
A job that continues later, or a conversation	the runtime, with sessions	no history handling of your own (Step 14)
A fixed pipeline, the same steps every time	`claude -p` or the SDK in a program	Anthropic calls these "static workflows"; the programs of this course are such workflows
A product for other people, or a company's production jobs	the runtime or the API, **with an API key**, or Managed Agents	see the terms below

Anthropic's own advice for the harness points the same way: spend extra steps, subagents and parallel work only where they earn their cost (A harness for every task (https://claude.dev/blog/a-harness-for-every-task-dynamic-workflows-in-claude-code/), 2 June 2026).

Pro login or API key: what Anthropic's terms allow

- **Your Pro login is for ordinary, individual use.** Anthropic's documentation says the usage limits of Pro and Max assume ordinary, individual use of Claude Code and the Agent SDK. Learning with this course is such use, and so are your own programs on your own computer.
- **Products and services need an API key.** Developers who build products or services with Claude, including with the Agent SDK, should use an API key, under Anthropic's Commercial Terms. The key comes from the Claude Console or a supported cloud provider (Amazon Bedrock, Google Vertex AI, Microsoft Foundry). You may not offer a Claude login in your own application, or send other people's requests through your Pro plan. A person may sign in to the unmodified Claude Code program with their own plan. This is allowed also when another product starts that program, as OpenClaw does. Anthropic may enforce these rules without notice (Legal and compliance (https://code.claude.com/docs/en/legal-and-compliance)).
- **So, at work:** keep the same programs, and run them with your company's API key. Set it in the environment variable `ANTHROPIC_API_KEY` before the program starts (Lesson 0, Step 8 shows how Claude Code then uses it). The `cost≈$` numbers become real costs. Or move the job to Managed Agents, where Anthropic hosts the harness. When you are unsure which way fits, Anthropic's documentation suggests asking its sales team.

Using Claude Code only as the model: switching the harness off

What: some developers use Claude Code as nothing but the model of their own agent. They switch off its loop, its tools, its settings, its memory files and its MCP tools, and keep everything else in their own code. This section shows how, and measures on this machine what each switch removes. It also tests whether Claude Code's own instructions make the model answer better.

Why: it is the opposite of the harness idea above. It fits when your program owns the loop and the tools, and only needs answers from the model.

Who does it:

- **dsh-claude-cli** makes a Claude Code login the model backend of another agent harness. In its own words, the harness stays the agent. It switches off the CLI's loop, tools, settings, memory files and MCP servers (dsh-claude-cli on GitHub (https://github.com/katsos/dsh-claude-cli), a small project).
- **pi-claude-cli** registers Claude Code as a model provider of the pi agent: every request is a `claude -p` run, Claude proposes tool calls, and pi runs them (pi-claude-cli (https://cdn.jsdelivr.net/npm/pi-claude-cli@0.3.1/README.md)).
- **Voicetest**, above, uses it as a plain LLM backend for its tests.
- **Hindsight, Sigil, NodeTool, Agentry and promptfoo** do it too, for some or all of their calls (the table in "Who else builds agents this way", above).

How, switch by switch (checked with `claude --help` on Claude Code 2.1.308):

Switch	What it keeps out
`--tools ""`	the built-in tools: Read, Write, Edit, Bash, PowerShell, ...
`--strict-mcp-config`	MCP tools, also the connectors of your claude.ai account, which `--tools ""` leaves in
`--system-prompt "..."`	Claude Code's own instructions (one line about the Agent SDK stays)
`--setting-sources ""`	your user, project and local settings files
`--safe-mode`	CLAUDE.md, skills, plugins, hooks and custom agents; your login keeps working
`--disable-slash-commands`	skills started with `/name`
`--no-session-persistence`	the saved session on disk
`--permission-mode dontAsk`	permission questions; anything not allowed is refused
`CLAUDE_CODE_DISABLE_CLAUDE_MDS=1`, `CLAUDE_CODE_DISABLE_ATTACHMENTS=1`	every CLAUDE.md file; the skill list and similar notes (Anthropic: modifying system prompts (https://code.claude.com/docs/en/agent-sdk/modifying-system-prompts))

Not this one: `--bare` sounds right ("minimal mode"), but it never reads your Pro login: it accepts only an `ANTHROPIC_API_KEY`.

There is no switch for the loop. With no tools, a `claude -p` run ends after one answer anyway. When the parent agent needs tool requests, dsh-claude-cli declares the parent's tools through an MCP server that never runs them. It ends the request as soon as the model's message ends. So Claude asks for a tool, and the parent agent runs it.

All of it in one program: `agent/ask_claude.py --strict` (both ways, with `--sdk` too).

What really reaches the model

Claude Code can write every request it sends to a folder: set `OTEL_LOG_RAW_API_BODIES=file:<folder>` before `claude -p` (Anthropic: see what Claude received (https://code.claude.com/docs/en/agent-sdk/modifying-system-prompts)). These are the requests of one question, on Haiku, from the user folder:

Setup	Tools sent	System prompt	Input tokens
Claude Code's defaults	16	11,387 characters	39,143
tools off, Claude Code's prompt kept	10, the connector tools	9,971 characters	7,332
tools off, own prompt (`ask_claude.py`)	10, the connector tools	339 characters	4,297
the same, plus `--strict-mcp-config`	0	339 characters	871
every switch (`ask_claude.py --strict`)	0	339 characters	849

How to read it:

- One switch did most of the work: `--strict-mcp-config` took 4,297 down to 871 tokens. The reason: `--tools ""` had left the 10 MCP tools of the account's claude.ai connectors in every request.
- The other switches changed almost nothing here, because these calls ran from the user folder, where there is no CLAUDE.md and no skill. From a project folder, they matter.
- Some text stays even with every switch: the line about the Agent SDK in the system prompt, a short reminder with account context, and an environment block (working folder, platform). And every run starts a program: about 5 seconds in total, of which the model needed about 1.

What others found when they switched parts off

- **Keeping Claude Code's prompt with the tools off went wrong** in Missy, an open-source agent. Its first adapter appended Missy's instructions to Claude Code's agentic system prompt. With the built-in tools off, the model still tried to call file tools that didn't exist, in 43 of 201 failing test cases. In 39 more, safety notices and a silent switch to an older model showed up in the answer text. The fix: replace the prompt, use `--max-turns 1`, block MCP tools, and check that the pinned model answered (Missy, pull request 170 (https://github.com/MissyLabs/missy/pull/170), October 2026).
- **Switch a part off completely, or not at all.** Hindsight, which uses Claude Code as the model of its memory work, documents three traps in its code. Allowing no tools (`allowed_tools=[]`) left them loaded. So a one-turn call sometimes spent its only turn loading tools, and failed with "Reached maximum number of turns (1)". Removing them (`tools=[]`) fixed it. Without a pinned model, Claude Code used its own default, an Opus-class model on a Pro or Max login. Hindsight's logs still showed the configured model (issue 2881). And the settings files of the computer brought in hooks that fed Hindsight's own calls back into its memory: a loop. Now it runs Claude Code in an empty folder, and loads no settings files (issues 1751 and 5542) (Hindsight: claude_code_llm.py (https://github.com/vectorize-io/hindsight/blob/main/hindsight-api-slim/hindsight_api/engine/providers/claude_code_llm.py)).
- **Keep one tool, not all or none.** Sigil answers its model calls with one turn and no tools. Otherwise, Claude Code would reach for its own tools instead of writing the tool-call text that Sigil reads. Its web search gets exactly one tool, WebSearch, on Haiku. Its documentation reports about 9,000 input tokens per call, even with all tools off. With `--system-prompt`, `--tools ""` and `--strict-mcp-config`, this course measured about 850 ("What really reaches the model", above). So measure the flags you use (Sigil: Claude Code (https://sigilagent.com/reference/claude-code.html)).
- **Text answers work, tool work doesn't**, in clio-agent's Claude Code provider with the tools off (clio-agent, issue 717 (https://github.com/iowarp/clio-agent/issues/717)).

- **Fewer tools, same success, lower cost:** restricting Claude Code and Codex to one `execute_code` tool was cheaper than their full tool sets, or tied with them, in 3 of 4 settings. The pass rates were statistically tied (arXiv 2607.10569 (https://arxiv.org/abs/2607.10569), July 2026).
- **A big harness costs more per success:** a preregistered study compared Claude Code 2.1.220 with the small PI.DEV harness. Claude Code sent a 12 to 15 times larger fixed prefix and took 2 to 7 times more turns. So its cost per successful task was 5 to 30 times higher, at comparable success rates (mostly open models, plus Claude Sonnet 5) (Weinberger and Hozez, arXiv 2608.01347 (https://arxiv.org/abs/2608.01347), August 2026).
- **The layer itself changes quality, both ways:** Anthropic traced a drop in Claude Code's quality to three changes in the harness, not in the model. They were a lower default effort, old thinking cleared on every turn, and a line in the system prompt that limited answer length. It says the API was not affected (Anthropic postmortem (https://anthropic.com/engineering/april-23-postmortem), 23 April 2026). So set the model and the effort yourself, and test again after a Claude Code update.
- **An anecdote to read with care:** one developer's code-review tool found 7 issues, all real, through `claude -p`, and 84 through the API on Sonnet, about 69 of them false. But it was one diff and one run, and the CLI looked at fewer files. The API version also had four integration bugs, which the author fixed on the way. It shows how much the integration matters, not that the CLI is smarter (Claude CLI vs API for code review (https://dev.to/fole/claude-cli-vs-api-for-code-review-same-model-wildly-different-results-1oai), March 2026).

Does Claude Code's own prompt make the model answer better?

That is the real question behind keeping Claude Code's instructions: maybe its engineering helps the model even without tools. `tools/model_mode_compare.py` tests it. It asks the same 12 questions (a time, a date, arithmetic, a ticket label, JSON extraction, a tool choice, logic, the order of steps, an exact reply) with the same model, in different modes. A real run on Haiku:

```
python tools/model_mode_compare.py --model haiku

mode           right  input tokens  output  thinking  seconds  model s
harness       11/12         39,151      66        54      5.5      1.4
cc_prompt     11/12          3,917       8         0      4.7      0.9
own_prompt    10/12            906      40        32      5.1      1.2
strict        10/12            882      29        23      4.6      0.9
```

Then the comparison that matters, repeated: Claude Code's own prompt (`cc_prompt`) against a short prompt of your own (`own_prompt`), both with the tools off. There are three runs on Haiku and one on Sonnet:

```
python tools/model_mode_compare.py --model haiku --modes cc_prompt,own_prompt --runs 3

Model: haiku, 12 questions, 3 run(s) per mode
mode           right  input tokens  output  thinking  seconds  model s
cc_prompt     31/36          3,917       7         0      4.9      0.9
own_prompt    33/36            905      34        26      4.9      1.2

Model: sonnet, 12 questions, 1 run(s) per mode
mode           right  input tokens  output  thinking  seconds  model s
cc_prompt     12/12          3,092      12         0      5.2      1.4
own_prompt    12/12            883      22        16      5.0      1.4
```

How to read it:

- **Claude Code's prompt did not make the answers better.** On Haiku, it was right 31 times out of 36, and your own prompt 33 times. One of those "wrong" answers was right, but not in the format asked for. On Sonnet, both were right every time. With this few questions, the difference on Haiku is within chance. So the honest result is: no sign of a gain.
- **It cost more:** 4.3 times the input tokens on Haiku (3,917 against 905), 3.5 times on Sonnet.
- **The model mattered more than the prompt.** Haiku's mistakes were simple (a time, a date, a price), and the date question failed in all three `cc_prompt` runs. Sonnet made none in either mode.

- **No answer tried to use a tool,** because these questions need none. Missy's agent, which asked about files, did run into that problem (above).
- **Anthropic's advice points the same way:** start from Claude Code's prompt for coding tools where a person watches and steers. Write your own prompt for an agent with a different job, identity or permission model (Anthropic: modifying system prompts (https://code.claude.com/docs/en/agent-sdk/modifying-system-prompts)).

So: when to switch the harness off

- **Switch it off** when your program owns the loop and the tools, and only needs answers: `python agent/ask_claude.py --strict "..."`. You keep the door (your Pro login, the model names) and send about 850 input tokens instead of about 4,300, or 39,000 with every default on. Each call still costs a few seconds to start the program.
- **Keep it on** when you want what this lesson is about: a job done by the loop, the tools, the skills and the rules. Switching all of that off throws away the reason to use Claude Code.
- **For high volume, or many users at once,** use neither: call the API with an API key.

Test it on your own agent: put 50 to 100 real questions of your agent into a file, as a JSON list of `{"prompt": "...", "expect": "<regular expression>"}`. Then run `python tools/model_mode_compare.py --questions my_questions.json --runs 3`. With an API key, you can add the direct API call as a third baseline.

Three questions: when, how, and how cheaply Claude Code beats a plain model call

What: the three questions that matter if you want Claude Code instead of a direct model call in your own agent, for work that is not coding:

1. When does Claude Code, with its tools, beat a plain model call?
2. How do you make it beat a plain model call?
3. How do you cut its tokens and still keep that advantage?

Why: the sections above show that the harness matters, and that a model-only call gains nothing from Claude Code's prompt. Your agent needs the next step: which jobs to give to the harness, how to set it up, and at what cost.

What others have measured

- **The same model, different harnesses, coding tasks:** in the HarnessTax study (UC Berkeley and Arena, September 2026), each model solved about as many tasks in Claude Code as in Codex CLI and the small Pi harness. But Claude Code cost about twice as much as Pi (HarnessTax (https://arena.ai/blog/coding-agents-harness-tax/)). A smaller test shows where the difference starts: about 27,000 input tokens per request in Claude Code, against about 2,600 in Pi. On a simple task, 24 of Claude Code's 28 tools were never used (Portkey: The Harness Tax (https://portkey.ai/blog/the-harness-tax/), April 2026).
- **Fewer tools, same success, lower cost**, and **a big harness costs more per success**: the tool-restriction paper and the preregistered study in "What others found when they switched parts off", above.
- **Anthropic's own advice for fewer tokens** in Claude Code: MCP tool definitions are deferred until used, and unused servers should be switched off. Prefer command-line tools to MCP servers. Let a hook or a script pre-process data, so the model sees hundreds of tokens instead of tens of thousands. Give know-how through skills, which load only when used, and keep CLAUDE.md under 200 lines. Lower the effort for simple work, and write specific prompts (Anthropic: manage costs effectively (https://code.claude.com/docs/en/costs)).
- **What nobody has published:** the same model, on work that is not coding, as a plain call against Claude Code with its tools. So this course measured it.

What this course measured

The test: `tools/harness_profile_compare.py` asks the same questions in five profiles of `claude -p`, all on Haiku, the cheapest model, where mistakes show first:

Profile	What Claude gets
`model`	no tools, a short prompt of your own, every customization off, one turn. The file's text is pasted into the question, as your program would do with a plain API call
`lean`	Claude Code's own prompt and loop, with only `Read` and python commands allowed; no skills, no CLAUDE.md, no MCP tools
`lean_py`	`lean`, plus one sentence appended to Claude Code's prompt: compute every number with Python
`skills`	the agent of this course: the project's skills and CLAUDE.md, no MCP tools
`full`	everything Claude Code loads by default, also your user settings and your account's connectors

An answer is right when it matches the true value exactly. "Ran code" counts the runs that called Bash, PowerShell or a skill. The cost is Claude Code's API-price estimate (`total_cost_usd`). With Pro, you don't pay it, but it shows how much of your allowance a profile uses.

Round 1: small files. Seven tasks: the total per region, an average, the product with the most units, an invoice total, a word count, working days, and a logic question as a control. Every file is under 1 KB, small enough to paste into a question. Two runs per task:

```
python tools/harness_profile_compare.py --model haiku --runs 2 --tasks small

Model: haiku, 7 tasks, 2 run(s) per profile

profile    right  ran code  input tokens      cost  cost per right  turns  seconds
model     14/14      0/14          1,062 $  0.0003         $0.0003    1.0      6.2
lean      14/14      0/14         16,695 $  0.0006         $0.0006    1.7      6.8
lean_py   13/14     12/14         24,082 $  0.0007         $0.0007    2.4      8.8
skills    12/14      5/14         77,749 $  0.0022         $0.0025    2.4      8.6
full      13/14      6/14         84,911 $  0.0031         $0.0034    2.1      9.9
```

How to read it:

- **The plain call was as right as any profile, and the cheapest.** A first round without `lean_py` gave the same picture. Over both rounds, `model` was right 27 times out of 28, and `lean`, `skills` and `full` 26 times each. With this few runs, a difference of one or two is chance.
- **Three of the "wrong" answers had the right value in the wrong form.** Twice, in `skills` and `full`, Haiku answered "Carl" with an explanation instead of the name only. Once `lean_py` wrote its Python command in a form that the allow rule didn't cover. The command was refused, and Claude said so.
- **Unrestricted Claude Code cost about 10 times as much per right answer** ($0.0034 against $0.0003), with 80 times the input tokens, and took 3.7 seconds longer.
- **Tools alone don't make Claude compute.** `lean` was allowed to run Python, but ran it in none of its 14 runs: it read each file and worked in its head. One appended sentence made `lean_py` run Python in 12 of 14.
- **The skills were called once in 14 runs.** For these one-line computations, Claude read the file or wrote its own Python.

Round 2: a table too long to add up in your head. The tool writes 300 made-up orders (order, region, amount). The questions: the total per region to the cent, and the number of orders over 500. Three runs per task:

```
python tools/harness_profile_compare.py --model haiku --runs 3 --tasks big
```

```
Model: haiku, 2 tasks, 3 run(s) per profile

profile    right  ran code  input tokens      cost  cost per right  turns  seconds
model      5/6       0/6          3,610 $  0.0038         $0.0046    1.0     25.6
lean       3/6       6/6         50,184 $  0.0043         $0.0086    4.0     26.9
lean_py    6/6       6/6         33,914 $  0.0012         $0.0012    3.0     10.3
skills     6/6       6/6        112,646 $  0.0030         $0.0030    3.0     11.0
full       6/6       6/6        135,196 $  0.0043         $0.0043    3.3     12.4
```

How to read it:

- **The plain call had to work hard, and slipped.** Working in its head, Haiku wrote about 3,300 output tokens for each count, and 7,000 to 13,600 for each set of totals. It took 26 seconds on average, and one of its totals was one cent off, given as exact.
- **A few lines of Python were cheaper than the hand arithmetic.** `lean_py` read the file, ran Python, and was right 6 times out of 6, in 10 seconds. It cost $0.0012 per right answer: a quarter of the plain call's cost, with 9 times its input tokens. Its answers needed about 420 output tokens. The plain call's cost was mostly its long output.
- **Unrestricted Claude Code was right every time,** at about the same cost per right answer as the plain call ($0.0043 against $0.0046), in half the time. `skills` and `full` ran Python in every run. Both read the project's CLAUDE.md, which names `python`.
- **`lean` failed for a reason you can fix.** It tried `awk` and PowerShell's `Import-Csv`, not python. The allow rule `Bash(python *)` refused both, and `--permission-mode dontAsk` doesn't ask. Then it was honest: twice it gave no totals rather than unchecked ones, and once it added a note to the right count. It gave no wrong number, but half its runs gave no usable answer.

The answers

1. When does Claude Code, unrestricted, beat a direct model call on work that is not coding? When the job needs more than the model's head: exact arithmetic on more than a few lines of data, files it must find and read itself, data too big to paste, several steps. On the 300-row table, it was right 6 times out of 6 against 5 out of 6, at the same cost per right answer and twice as fast. On 3,000 rows, it was right 4 times out of 4 against once, at a tenth of the cost per right answer ("The big idea", above). When the data is small enough to paste and the question is easy, it does not beat a plain call. It gives the same right answers, at about 10 times the cost.

2. How do you make it beat a direct call?

- **Give it a way to compute, and tell it to use it**, in words that match your allow rule. One appended sentence did more than any tool or skill: `lean_py` was right 6 times out of 6 on the big table, the cheapest and fastest profile.
- **Check the form of every answer** with a pattern, or ask for JSON with `--json-schema`. Claude Code's longer prompt made Haiku add explanations.
- **Put rules that must be exactly yours into skills:** an invoice's rounding, your company's holidays (Lessons 2 to 4). For a one-line sum, Claude writes its own Python.

3. How do you cut its tokens and keep the advantage?

- **Switch off what the job doesn't use, and keep Claude Code's prompt and loop:** `--strict-mcp-config` (your account's connectors), `--safe-mode` (CLAUDE.md, skills, plugins, hooks), `--tools "Read,Bash,PowerShell"` with an allow rule for python only, and `--disable-slash-commands`. Against unrestricted Claude Code, that meant about 70 to 75% fewer input tokens, and a 70 to 80% lower cost per right answer. It gave no fewer right answers (big table: $0.0012 against $0.0043; small files: $0.0007 against $0.0034).
- **Count the output tokens too:** a short Python run cost less than long reasoning in the head.

- **Route each job:** send small, easy questions to the plain call (`ask_claude.py --strict`), and jobs that compute on files to the lean runtime. One program can choose per job.
- **Don't trim what a hard job may need:** on the 3,000-row table, the lean runtime lost one run to its python-only rule. Everything on was right every time ("The big idea", above).

What this test doesn't show: Haiku only, two or three runs per task, made-up data, one computer. A larger model may slip less in its head. So repeat the test with your model and your own jobs, and compare the cost per right answer.

The three profiles, ready to copy

These are the profiles of the test, with `--max-budget-usd` added. Haiku is the model measured here. Use `sonnet` for harder jobs.

Model only (a plain call, with your data in the question):

```
claude -p "<question with the data>" --model haiku --output-format json --tools "" --system-prompt
    "<your role>" --strict-mcp-config --setting-sources "" --safe-mode --max-turns 1 --no-session-
    persistence
```

Lean runtime with the compute rule (`lean_py`: Claude Code's prompt and loop, reading files and computing with Python, nothing else):

```
claude -p "<job>" --model haiku --output-format json --tools "Read,Bash,PowerShell" --allowedTools
    "Read,Bash(python *),PowerShell(python *)" --append-system-prompt "Compute every number with
    Python (python -c or a short script) and answer from its output; never count, add or round by
    hand." --permission-mode dontAsk --strict-mcp-config --safe-mode --disable-slash-commands --max-
    turns 6 --max-budget-usd 0.50 --no-session-persistence
```

Skills runtime (the agent of this course: your tested scripts as skills):

```
claude -p "<job>" --model haiku --output-format json --allowedTools "Skill,Read,Bash(python
    *),PowerShell(python *)" --permission-mode dontAsk --setting-sources project --strict-mcp-config
    --max-turns 8 --max-budget-usd 0.50 --no-session-persistence
```

Option	What it does for cost and safety
`--max-turns N`	stops a job that goes in circles
`--max-budget-usd 0.50`	stops a job when its API-price estimate reaches the amount (only with `claude -p`)
`--permission-mode dontAsk`	refuses every tool that is not allowed, instead of waiting for an answer
`--exclude-dynamic-system-prompt-sections`	moves per-machine details out of Claude Code's own system prompt. Then many runs on different machines share one prompt cache (only with Claude Code's prompt)

Test them on your own jobs: `python tools/harness_profile_compare.py --model haiku --runs 2` runs both rounds. Add your agent's real jobs to its `tasks()`, each with a regular expression for the right answer. Then run them with `--tasks <name>`.

Step 1 - Check that the Agent SDK is installed

What: the **Claude Agent SDK** is a Python package (`claude-agent-sdk`). It lets a Python program use Claude Code: the same skills, the same tools, and the same login.

Why: every program in Parts 0 to 2 imports it.

Run:

```
python -c "import claude_agent_sdk; print(claude_agent_sdk.__version__)"
```

Expected result:

```
0.2.168
```

How to read it: the SDK is installed in your venv (this lesson was tested with version 0.2.168).

If you see something else: `ModuleNotFoundError: No module named 'claude_agent_sdk'` means it is not installed in **this** Python. Install it once, in the activated window:

```
python -m pip install -r agent\requirements.txt
```

Part 0 - What the SDK does when your program asks Claude

Step 2 - See which program the SDK starts

What: `tools\show_sdk_command.py` prints what the SDK would start for `agent\run_agent.py`, without starting it. It costs nothing.

Why: the SDK doesn't contain its own AI. It **starts the Claude Code program in the background** and talks to it. Seeing the exact program and options takes the mystery out of it.

Run:

```
python tools/show_sdk_command.py --model haiku
```

Expected result:

```
The SDK would start this program:
    C:\pyenvsmy\claudeclioct26\Lib\site-packages\claude_agent_sdk\_bundled\claude.exe
in this folder (cwd):
    E:\may10\claudecodecli_and_python
with these options:
    --output-format
    stream-json
    --verbose
    --system-prompt

    --allowedTools
    Skill,Read,Bash(python *),PowerShell(python *)
    --max-turns
    15
    --model
    haiku
    --setting-sources=project
    --input-format
    stream-json
```

How to read it:

Part	Meaning
The program	a `claude.exe` that came **inside the pip package** (`_bundled`). It is a second copy of Claude Code. It reads the same login file as your normal `claude`, so it uses your Pro plan
The folder	the project folder. That is where Claude Code finds `.claude\skills\`
`--output-format stream-json`, `--input-format stream-json`	the SDK and Claude Code talk in JSON lines: one line per message, in both directions
`--allowedTools ...`	these tools run without permission questions
`--max-turns 15`	the most turns allowed
`--model haiku`	the model
`--setting-sources=project`	load only this project's skills and settings

Part	Meaning
The empty line after `--system-prompt`	an empty system prompt. The system prompt is the long set of instructions that normally tells Claude how to behave as Claude Code. An SDK agent gets only the tools, the skills and your question. To get the full normal behavior, add `system_prompt={"type": "preset", "preset": "claude_code"}` to the options

Your question is not in the list. The SDK sends it separately, as a JSON line on the program's input (stdin). It reads Claude's steps back as JSON lines from its output (stdout). This works exactly like the pipe in Lesson 3, Step A4.

Step 3 - Compare with your normal Claude Code

What: `where claude` shows which `claude.exe` cmd starts when you type `claude`.

Why: to see that the SDK uses a different copy of the same program, and that both share one login.

Run:

```
where claude
```

Expected result:

```
C:\Users\Workstation\.local\bin\claude.exe
```

How to read it: your normal `claude` is a different file from the SDK's copy in Step 2. Both read your login from `C:\Users\<you>\.claude\.credentials.json`. The SDK prefers its own copy. To make it use your normal one, add `cli_path=r"C:\Users\Workstation\.local\bin\claude.exe"` to `ClaudeAgentOptions`.

So this is **headless mode**, the same as `claude -p`: no window, and nobody to answer permission questions. That is why a program must allow its tools in advance with `allowed_tools`.

	`claude` (normal window)	**`claude -p "..."`**	**Agent SDK**
Who starts `claude.exe`	you	you, or a program	the SDK, in the background
Which `claude.exe`	your normal one	your normal one	the copy inside the pip package
Your question goes in through	the keyboard	the command line	a JSON line on stdin
The answer comes back as	text on the screen	text, or JSON	Python objects
Permission questions	you answer them	nobody can, so allow tools in advance	nobody can: `allowed_tools`, or a Python function `can_use_tool`

Part 1 - An agent program that uses the skills

Step 4 - Run the agent program

What: `agent\run_agent.py` asks Claude one question through the SDK, and prints each step.

Why: to see that your program gets the same skill choice and script runs as the normal window, step by step.

Run: (`--model haiku` uses the small model, which saves your allowance)

```
python agent\run_agent.py --model haiku "Which region sold the most in data/sales.csv?"
```

Expected result (a real run, shortened; your answer will be worded a little differently):

```
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5
[init] skills discovered: ['analyzing-csv', 'analyzing-excel', 'analyzing-text', ...]

[skill] Claude picked: analyzing-csv
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" summary data/sales.csv
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" groupby data/sales.csv --by region --value amount --agg sum
[claude] North sold the most in `data/sales.csv`, with total `amount` of 2,150.0. The other regions
    were East at 910.0, West at 900.0 and South at 670.0.

[done] success, turns=5, cost≈$0.0028 (API list-price estimate)
```

(Without `--model haiku`, the default model, Opus, is used: the same steps cost about $0.20. Claude may also use the PowerShell tool instead of Bash; both are allowed.)

How to read it: the program prints a label before each step:

Label	Meaning
`[init]`	The session started: which login (your Pro plan), which model, and which skills were found
`[skill]`	Claude chose a skill
`[tool]`	Claude ran a tool; here Bash ran the skill's script
`[claude]`	Claude's answer
`[done]`	The end: success or error, the number of turns, and the cost estimate (not a charge on Pro)

Claude followed the skill's instructions in order: first `summary`, then `groupby`. This is exactly as in the normal `claude` window, but now your own program receives each step.

If you see something else:

- `[init] auth: ANTHROPIC_API_KEY - billed per token to that API account!`: an API key is set, and that account would pay. Remove it (Lesson 0, section 1).
- A `[refused]` line: a tool call failed, for example a tool that is not in `allowed_tools`, or a skill that is not offered (Step 6). The text after it says why.

Step 5 - Read the important part of the program

What: the core of `agent\run_agent.py`.

Why: you will copy this for your own agents.

```
options = ClaudeAgentOptions(
    cwd=str(PROJECT_ROOT),          # the project folder; skills are read from its .claude/skills
    setting_sources=["project"],    # load only this project's skills and settings
    skills="all",                   # or a list, for example ["converting-units", "calculating-
    dates"]
    allowed_tools=["Skill", "Read", "Bash(python *)", "PowerShell(python *)"],
    max_turns=15,
)

async for message in query(prompt=prompt, options=options):
    if isinstance(message, SystemMessage) and message.subtype == "init":
        print(message.data["skills"])                # -> the [init] line
    elif isinstance(message, AssistantMessage):
        for block in message.content:
            if isinstance(block, ToolUseBlock) and block.name == "Skill":
                print("picked:", block.input["skill"])    # -> the [skill] line
```

How it works:

Line	What it does	Why
`cwd=str(PROJECT_ROOT)`	starts Claude Code in the project folder	Claude Code finds skills in `.claude\skills\` of the folder it starts in
`setting_sources=["project"]`	loads the project's settings and skills, not your personal ones	the agent behaves the same on every computer
`allowed_tools=[...]`	the tools that may run without a question	nobody can answer permission questions. Without `"PowerShell(python *)"`, the run would stop on Windows, where Claude often uses PowerShell
`max_turns=15`	stops after 15 turns	a run that goes wrong can't use up your allowance
`query(prompt=..., options=...)`	sends the question	
`async for message in ...`	receives the steps one by one, **while** Claude works	you can print each step at once
`SystemMessage`, `AssistantMessage`, `ResultMessage`	the kinds of steps: the session start, Claude used a tool or wrote text, and the end	you check the kind with `isinstance`
`ToolUseBlock` with `block.name == "Skill"`	Claude called the Skill tool. `block.input["skill"]` is the skill's name	the same Skill tool call you see in the normal window (Lesson 2)

The full program is `agent\run_agent.py`. Open it with `notepad agent\run_agent.py`.

Step 6 - Make an agent that knows only some skills

What: `--skills` sets the `skills` option to a list.

Why: a specialized agent should be offered only the skills it needs. Fewer skills mean an easier choice and a smaller menu.

Run:

```
python agent\run_agent.py --model haiku --skills converting-units,calculating-dates "How many km is a
    marathon?"
```

Expected result (a real run):

```
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5
[init] skills discovered: ['analyzing-csv', 'analyzing-excel', ...]

[skill] Claude picked: converting-units
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/convert.py" convert 26.2 mi km
[claude] A marathon is **42.195 km** (about 26.2 miles). ...
[done] success, turns=4, cost≈$0.0045 (API list-price estimate)
```

How to read it: Claude is **offered** only these two skills. The `[init]` line still lists all skills found on disk, because the list only filters what Claude is offered. With different `--skills` lists, you can make several specialized agents from one set of skills.

The skills list is not a lock. A test with only `converting-units` allowed asked: "Use the calculating-dates skill to tell me what weekday 2026-12-25 is." Two runs, two behaviors:

- Run 1: Claude tried the Skill tool. Claude Code **refused** (the skill is not in the list), and Claude worked out the answer without it.

- Run 2: Claude found `.claude\skills\calculating-dates\SKILL.md` with its Read tool and ran `dates.py` itself, because `allowed_tools` allows `python` commands.

So the list only decides what Claude is offered. To really keep an agent away from a skill, give that agent its own project folder (`cwd`) that contains only its skills.

Part 2 - Your own Python functions as tools (no skills)

Step 7 - Run the tools program

What: `agent\custom_tools_agent.py` gives Claude three Python functions as **tools**, and no skills and no shell.

Why: a skill makes Claude run a **command** and read printed text. A tool lets Claude call your **function directly**, with checked parameters. This is the other way to connect Claude to your Python.

Run:

```
python agent\custom_tools_agent.py "How many km is 26.2 miles, and what weekday is 2026-12-25?"
```

Expected result (a real run):

```
[tool] mcp__tutor__convert_units({'value': 26.2, 'from_unit': 'mile', 'to_unit': 'km'})
[tool] mcp__tutor__weekday({'date': '2026-12-25'})

- 26.2 miles is about 42.16 km. ...
- 2026-12-25 is a Friday. It falls in ISO week 52.
```

How to read it: Claude **called two Python functions directly**. The `[tool]` lines show the function names and, in parentheses, the values Claude chose for the parameters. Claude wrote `'mile'` instead of `'mi'`, and it still worked, because the `convert()` function knows such alternative names.

Step 8 - Read how a function becomes a tool

What: the parts of `agent\custom_tools_agent.py` that make a tool.

Why: to write tools for your own functions.

```
@tool("convert_units", "Convert a number between units of length, mass, ...",
      {"value": float, "from_unit": str, "to_unit": str})
async def convert_units(args):
    return as_result(convert_module.convert(args["value"], args["from_unit"], args["to_unit"]))
```

How it works:

- `@tool(...)` turns the function below it into a tool. It takes three things. The first is the **name** (`convert_units`). The second is a **description**: Claude reads it to decide when to use the tool, like a skill's description. The third is the **parameters**, with their types (a number and two texts).
- When Claude uses the tool, the function receives `args`, a dictionary such as `{"value": 26.2, "from_unit": "mile", "to_unit": "km"}`. The types are checked before your function runs.
- It calls the **same `convert()` function** that the `converting-units` skill runs. `as_result(...)` wraps the returned dictionary in the form the SDK expects.

```
def sdk_tool_server():
    ...                                  # the three @tool functions, like the one above
    return create_sdk_mcp_server(name="tutor", version="1.0.0",
                                 tools=[convert_units, weekday, add_business_days])

options = ClaudeAgentOptions(
    mcp_servers={"tutor": sdk_tool_server()},
    allowed_tools=["mcp__tutor__*"],
    tools=[],
    setting_sources=[],
)
```

Line	What it does
`create_sdk_mcp_server(...)`	collects the three tools into a "tool server" named `tutor`. It runs inside your Python program. (MCP is the standard way to give Claude extra tools)
`allowed_tools=["mcp__tutor__*"]`	Claude Code names these tools `mcp__<server>__<tool>`, as you saw in the `[tool]` lines. The `*` allows all tools of the `tutor` server
`tools=[]`	removes the built-in tools (Read, Bash, ...), so Claude can use **only** your functions
`setting_sources=[]`	loads no skills from disk

The same tools with `claude -p`: add `--cli`.

```
python agent\custom_tools_agent.py --cli "How many km is 26.2 miles, and what weekday is 2026-12-25?"
```

Expected result (a real run, 8 seconds):

```
[init] auth: claude.ai subscription
[tool] mcp__tutor__convert_units({'value': 26.2, 'from_unit': 'mi', 'to_unit': 'km'})
[tool] mcp__tutor__weekday({'date': '2026-12-25'})

- **26.2 miles is about 42.16 km** (42.164813). That's roughly the length of a marathon.
- **2026-12-25 is a Friday**, in ISO week 52.
```

How it works: `claude -p` can't call functions inside your Python program. So with `--cli`, a separate program serves the same three tools: `agent\tutor_mcp_server.py`. Claude Code starts it itself (`--mcp-config` says how). That program shows what MCP really is: it reads one JSON message per line on stdin, and answers each one on stdout:

Message from Claude Code	The server's answer
`initialize`	its name, and "I have tools"
`tools/list`	each tool's name, description, and parameters (as JSON Schema)
`tools/call`	runs one function with Claude's values, and returns its result as JSON text

It uses only the standard library: no SDK, no `mcp` package. A stand-alone tool server has a bonus: the interactive `claude` window can use it too, after `claude mcp add tutor -- python agent/tutor_mcp_server.py` (`claude mcp remove tutor` removes it again).

Skills or tools? Use both:

	Skill + script	`@tool` function
Can carry step-by-step know-how (checklists, rules)	**yes**: the whole SKILL.md	only a short description
How the code runs	Claude runs a shell command and reads the printed JSON	Claude calls the function directly with checked parameters
Works in the normal `claude` window, without a program	**yes**	only as a separate tool server program, like `tutor_mcp_server.py`
Best for	jobs with several steps, files, team rules	single exact actions: look up, convert, call a web service

Part 3 - Ask Claude from Python with only Pro (no API key)

The problem: with an API key, a Python program would use the `anthropic` package. **Pro gives you no API key**, so that package can't log in.

The solution: your program asks **Claude Code**, which has your login, with `claude -p`.

Step 9 - Ask with `claude -p` directly in cmd

What: `-p` is headless mode: Claude Code answers one question, prints the answer, and ends.

Why: first see it by hand. Step 10 then does exactly the same from Python.

Run:

```
claude -p "What is 17 * 23? Answer with the number only." --model haiku
```

Expected result:

```
391
```

How to read it: no window, no conversation: one question, one answer, with your Pro login.

Now ask for the answer as JSON, which is easy for a program to read:

Run:

```
claude -p "What is 17 * 23? Answer with the number only." --model haiku --output-format json
```

Expected result (the important fields of a real run; the answer was worded differently this time):

```
{
 "type": "result",
 "subtype": "success",
 "is_error": false,
 "result": "17 * 23 = 391",
 "num_turns": 1,
 "duration_ms": 3660,
 "total_cost_usd": 0.00236645
}
```

How to read it: `result` is the answer, `is_error` tells whether it worked, and `total_cost_usd` is the cost estimate (not a charge on Pro). A Python program can run this command and read `result`.

Step 10 - The same from a Python program

What: `agent\ask_claude.py` does Step 9 with Python's `subprocess` module. It needs only the standard library.

Why: this is how you use Claude as a normal "question in, answer out" LLM in your own Python code, with only Pro.

Run:

```
python agent\ask_claude.py "Explain a Python decorator in two sentences."
```

Expected result (a real run):

```
A Python decorator is a function that takes another function as input, wraps it with extra behavior,
    and returns the modified function, letting you change or extend its behavior without editing its
    code. You apply it with the `@` syntax above a function definition, ...

--- {"is_error": false, "model_usage": ["claude-haiku-5-5"], "thinking_tokens": 0,
    "cost_estimate_usd": 0.00039922, "duration_ms": 4288} (cost is an API list-price estimate; on Pro
    nothing is charged)
```

Run: (structured output: you describe the shape of the JSON, and the answer follows it)

```
python agent\ask_claude.py --schema "List three European capitals."
```

Expected result:

```
{
  "capitals": [
    {"country": "France", "capital": "Paris"},
    {"country": "Germany", "capital": "Berlin"},
    {"country": "Italy", "capital": "Rome"}
  ]
}
```

How it works, the important part of the program:

```
cmd = [
    claude, "-p", question,
    "--output-format", "json",      # print one JSON object: answer, usage, cost estimate
    "--model", model,
    "--system-prompt", system,      # your own instructions instead of Claude Code's coding
    instructions
    "--tools", "",                  # no tools at all: just a question and an answer
    "--disable-slash-commands",     # no skills either
    "--no-session-persistence",     # don't save it in the session list
]
data = json.loads(subprocess.run(cmd, capture_output=True, text=True, encoding="utf-8").stdout)
answer = data["result"]
```

- `subprocess.run` runs the command like typing it in cmd, and `capture_output=True` collects what it prints instead of showing it.
- `json.loads` turns the printed JSON into a Python dictionary. `data["result"]` is the answer.
- `--schema` adds `--json-schema` with a description of the wanted JSON shape. The answer is then in `data["structured_output"]`.

Rules:

Rule	Why
For your own use only	a program for other people must use an API key (Lesson 0, section 5)
Never copy the login token out of `.credentials.json` into your own code	it is a password to your account. Always go through `claude` or the SDK
Don't add `--bare`	that mode ignores your Pro login and expects an API key
Watch for `ANTHROPIC_API_KEY`	if it is set, `claude -p` uses it and bills that API account. `ask_claude.py` warns you about this

Part 4 - `claude -p` or the SDK: the same jobs both ways

The idea: both ways run the **same** Claude Code program, with the same login, the same skills, the same models and the same allowance. The SDK even starts `claude.exe` itself (Step 2). So the question is never "which one is smarter", only "**how much does my Python program need to take part while Claude works?**". This part shows it with real runs. First comes one job done both ways, then jobs where `claude -p` is simpler, then jobs where the SDK is simpler. Those are done both ways too: every agent program of the course has a `claude -p` way. The option `--cli` switches it on (Step 16 lists them).

Every run below used Haiku and was copied from the course machine. Times are wall-clock times, measured in cmd.

Step 11 - The same job both ways

What: one skills job, `Build a pivot of amount by region and product from data/sales.xlsx.`, first with `claude -p`, then with `run_agent.py`.

Why: to compare what you get back, when the job and the result are the same.

Run, way 1: `claude -p` (one command, nothing to install)

```
claude -p "Build a pivot of amount by region and product from data/sales.xlsx." --model haiku --
    allowedTools "Skill,Read,Bash(python *),PowerShell(python *)"
```

How it works: `--allowedTools` lists the tools that may run without a question, the same list as `allowed_tools` in `run_agent.py`. Without `Skill` in it, the skill would not load (Lesson 7, Step 10).

Expected result (a real run, 22 seconds):

```
Here is the sum of `amount` by region (rows) and product (columns) from the `Sales` sheet of
    `data/sales.xlsx`:

| region | Gadget | Gizmo | Widget | Total |
|--------|-------:|------:|-------:|------:|
| East   | 160    | 600   | 150    | 910   |
| North  | 0      | 1650  | 500    | 2150  |
| South  | 320    | 150   | 200    | 670   |
| West   | 400    | 0     | 500    | 900   |
| **Total** | **880** | **2400** | **1350** | **4630** |

Gizmo is the largest product at 2400, and North is the largest region at 2150. I used the `Sales`
    sheet; the `Targets` sheet was not included.
```

Run, way 2: the SDK program

```
python agent\run_agent.py --model haiku "Build a pivot of amount by region and product from
    data/sales.xlsx."
```

Expected result (a real run, 18 seconds; the `skills discovered` line is left out):

```
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5

[skill] Claude picked: analyzing-excel
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    excel/scripts/excel_tool.py" sheets data/sales.xlsx
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    excel/scripts/excel_tool.py" pivot data/sales.xlsx --sheet Sales --index region --columns product
    --values amount --agg sum
[claude] Here is the sum of `amount` by region (rows) and product (columns), from the `Sales` sheet
    of `data/sales.xlsx`:
...the same table...
[done] success, turns=5, cost≈$0.0019 (API list-price estimate)
```

How to read it:

	`claude -p`	**`run_agent.py` (SDK)**
The answer	the same table	the same table
What you get back	only the final text	every step: the skill, each command, the answer, turns and cost
Time	about 20 seconds	about 20 seconds: the SDK starts the same `claude.exe`
What you needed	one command	the SDK (about 270 MB) and a 100-line Python program

Good to know: `claude -p` can show the steps too. With `--output-format stream-json --verbose`, it prints one JSON line per step, and your program reads them. `tools\routing_eval.py` does exactly that (Lesson 5, Test 4). The SDK reads the same lines for you, and turns them into Python objects. So for one job, the SDK is more convenient, not more capable.

Run, way 3: the same program, the `claude -p` way

```
python agent\run_agent.py --cli --model haiku "Build a pivot of amount by region and product from
    data/sales.xlsx."
```

Expected result (a real run, 18 seconds; the `skills discovered` line is left out):

```
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5

[skill] Claude picked: analyzing-excel
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    excel/scripts/excel_tool.py" sheets data/sales.xlsx
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    excel/scripts/excel_tool.py" pivot data/sales.xlsx --sheet Sales --index region --columns product
    --values amount --agg sum
[claude] Here is the sum of `amount` by region and product from the `Sales` sheet of
    `data/sales.xlsx`:
...the same table...
[done] success, turns=5, cost≈$0.0036 (API list-price estimate)
```

How it works: with `--cli`, `run_agent.py` doesn't use the SDK at all:

Function	What it does
`build_cli_command()`	writes the SDK options as the flags the SDK itself passes to `claude.exe` (Step 2): `--system-prompt ""`, `--allowedTools "Skill,Read,Bash(python *),PowerShell(python *)"`, `--max-turns 15`, `--setting-sources project`, plus `-p "<question>" --output-format stream-json --verbose`
`run_cli()`	starts that command with `subprocess`, and reads its output line by line, while Claude works
`print_event()`	prints each JSON line with the same labels as `print_message()` does for the SDK's objects

The steps and the answer are the same, and the `--cli` way needs only the standard library.

The one difference: `--skills`. The SDK tells `claude.exe` in its start-up message to offer only the listed skills. `claude -p` has no flag for that. With `--cli --skills converting-units`, Claude still sees all skills. Only the listed ones are pre-approved (`Skill(converting-units)`), and Claude Code refuses the others.

Step 12 - Where `claude -p` wins: text in, text out, from any command line

What: you send a file into Claude through a pipe, and save the answer in a file. No Python at all.

Why: this is the most common small job: summarize, translate, check a text. With `claude -p`, it is one line in cmd, a `.bat` file, or any program that can start a command.

Run:

```
type data\article.txt | claude -p "Summarize this text in 3 short bullet points." --model haiku >
    summary.txt

type summary.txt
```

How it works:

Part	What it does
`type data\article.txt`	prints the article
`|`	sends that text into `claude -p` as stdin, instead of to the screen. Claude gets the text together with your prompt
`> summary.txt`	saves what `claude -p` prints in a file, instead of showing it

Expected result (a real run, 7 seconds):

```
- Language models get exact tasks like adding a column of numbers approximately right, which isn't
    good enough for invoices, deadlines, or doses. Pair them with small, well-tested tools and tell
    them when to use each one.
- A skill divides the work: its description says when it applies, its body explains how to do the
    job, and its script handles the part that must be exact. The model picks the skill, runs the
    script, and explains the result.
- Keeping the logic in a testable, reviewable file keeps prompts short, saves context space, and
    stops answers from drifting. Start with one skill, watch where the model hesitates, and write the
    next one there.
```

Delete the practice file afterwards: `del summary.txt`

Step 13 - Where `claude -p` wins: a scheduled job without Python

What: `agent\daily_report.bat`, a job for Windows Task Scheduler. It asks one question with the Excel skill, and saves the answer in `reports\daily_report.txt`.

Why: a scheduled job runs when nobody is at the computer. It must be small and need nothing but Claude Code. Here there is no Python program and no SDK.

Run: (`type` shows the file, `call` runs it)

```
type agent\daily_report.bat

call agent\daily_report.bat
```

The file:

```
@echo off
call C:\pyenvsmy\claudeclioct26\Scripts\activate.bat
cd /d "%~dp0.."
if not exist reports mkdir reports
claude -p "Which region has the highest total amount in data/sales.xlsx? Answer in one sentence." --
    model haiku --allowedTools "Skill,Read,Bash(python *),PowerShell(python *)" --no-session-
    persistence > reports\daily_report.txt
type reports\daily_report.txt
```

How it works, line by line:

Line	Why it is there
`call ...\activate.bat`	a scheduled task has no prepared cmd window, so the job activates the venv itself. Without it, the Excel skill gets the "pandas is not installed" message (Lesson 0, "What happens if you forget to activate")
`cd /d "%~dp0.."`	`%~dp0` is the folder of the `.bat` file (`agent\`), and `..` is one folder up: the project folder, where the skills are
`if not exist reports mkdir reports`	creates the folder for the answer, the first time
`claude -p ... --no-session-persistence > reports\daily_report.txt`	asks the question and saves the answer. `--no-session-persistence` keeps the daily runs out of your session list
`type reports\daily_report.txt`	shows the answer, for when you run the job by hand

Expected result (a real run, 17 seconds):

```
North has the highest total amount in data/sales.xlsx, at 2,150 across the Sales sheet (ahead of West
    at 900, East at 910 and South at 670).
```

To run it every morning at 8:00, Windows Task Scheduler can start it. This command creates such a task (it was not run on the course machine, because it really creates a task):

```
schtasks /create /tn "Sales report" /tr "E:\may10\claudecodecli_and_python\agent\daily_report.bat"
    /sc daily /st 08:00
```

To remove the task again: `schtasks /delete /tn "Sales report" /f`. The task runs as you, with your saved login. For a task that runs while you are logged out, see `claude setup-token` in Lesson 0, section 5.

Step 14 - Where the SDK wins: a conversation with follow-up questions

What: `agent\chat_agent.py` asks two questions in **one** conversation. The second one, "And which region sold the least?", only makes sense together with the first.

Why: a chat program, or any program that asks follow-up questions, needs Claude to remember the earlier questions and answers.

Run:

```
python agent\chat_agent.py --model haiku
```

Expected result (a real run; the `skills discovered` line is left out):

```
[you]   Which region sold the most in data/sales.csv?
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5

[tool]  Read: E:\may10\claudecodecli_and_python\data\sales.csv
[claude] **North** sold the most in `data/sales.csv`, with a total amount of **2,150** across four
    orders.
...a table of all four regions...

[done] success, turns=2, cost≈$0.0011 (API list-price estimate)
[time]  question 1 took 6.2s

[you]   And which region sold the least?
[claude] **South** sold the least by amount, with a total of 670. By units, **East** sold the least,
    with 12 against South's 13.

[done] success, turns=1, cost≈$0.0015 (API list-price estimate)
[time]  question 2 took 1.2s
```

How to read it:

- Claude understood "the least" without being told the file again: the first question and answer are still in the conversation.
- The second question took **1.2 seconds**, because `claude.exe` was already running.
- In this run, Claude read the small CSV file directly, instead of using the skill. `Read` is in `allowed_tools`, so this is allowed. With a bigger file, or the Excel file, Claude uses the skill.

How it works, the core of `agent\chat_agent.py`:

```
async with ClaudeSDKClient(options=build_options("all", model)) as client:   # starts claude.exe once
    for question in questions:
        await client.query(question)                    # same session: Claude sees the earlier
    turns
        async for message in client.receive_response():  # this question's steps, up to its
    ResultMessage
            print_message(message)
```

`ClaudeSDKClient` keeps one `claude.exe` and one conversation open until the `async with` block ends. `build_options` and `print_message` come from `run_agent.py`, so all three agent programs work the same way.
Try it: `python agent\chat_agent.py --model haiku --interactive` lets you type your own questions. An empty line ends the chat.

The same program with `claude -p`: add `--cli`.

```
python agent\chat_agent.py --cli --model haiku
```

Expected result (a real run; the `[init]` lines are left out):

```
[you]   Which region sold the most in data/sales.csv?
[tool]  Read: E:\may10\claudecodecli_and_python\data\sales.csv
[claude] **North** sold the most, with total sales of **2,150** (by `amount`).
...a table of all four regions...
[done] success, turns=2, cost≈$0.0024 (API list-price estimate)
[time]  question 1 took 8.4s

[you]   And which region sold the least?
[claude] **South** sold the least, with total sales of **670** (by `amount`).
...
[done] success, turns=1, cost≈$0.0028 (API list-price estimate)
[time]  question 2 took 5.6s
```

How to read it: Claude remembered the first question here too. But each question started a new `claude` process, which loaded the saved conversation from disk again. That took **5.6 seconds** for a one-turn follow-up. The SDK way, run the same day on the same machine, took **0.9 seconds** for it.

How `--cli` does it, in `chat_cli()`: the last JSON line of each answer (the `result`) contains a `session_id`. The program keeps it, and passes it to the next question with `--resume`:

```
result = run_cli(build_cli_command(claude, question, "all", model, resume=session_id))
session_id = result.get("session_id") or session_id   # the next question continues this conversation
```

The same by hand, in cmd: ask the first question with `--output-format json`:

```
claude -p "Build a pivot of amount by region and product from data/sales.xlsx." --model haiku --
    allowedTools "Skill,Read,Bash(python *),PowerShell(python *)" --output-format json
```

Expected result (the important fields of a real run):

```
"result": "Here is the sum of `amount` by region and product, ...",
"num_turns": 5,
"duration_ms": 16128,
"session_id": "da79f480-1a45-4aad-a6b9-71243cdf4e3a"
```

Then pass **your** `session_id` (yours is different) to `--resume`:

```
claude -p "Which product sold the most in that pivot?" --resume da79f480-1a45-4aad-a6b9-71243cdf4e3a
    --model haiku --allowedTools "Skill,Read,Bash(python *),PowerShell(python *)" --output-format
    json
```

Expected result (a real run, 5.8 seconds):

```
"result": "Gizmo sold the most, with 2400 in total `amount`. That is 52% of the 4630 grand total.
    Widget is second at 1350 and Gadget is third at 880.",
"num_turns": 1,
"session_id": "da79f480-1a45-4aad-a6b9-71243cdf4e3a"
```

How to read it: Claude remembered the pivot. But each question started a new `claude.exe`, which loaded the saved conversation again. That took 5.8 seconds for a one-turn follow-up, against 1.2 seconds with `ClaudeSDKClient`. And your program had to read `session_id` from the first answer, and pass it on. (`--continue` instead of `--resume <id>` continues the most recent conversation in this folder. That is shorter, but if you used `claude` in between, it continues the wrong one.)

Step 15 - Where the SDK wins: your Python decides about every tool call

What: `agent\guarded_agent.py` runs the skills, but a Python function in the program approves or refuses **every** shell command and every file change, before it happens.

Why: `allowed_tools` and `--allowedTools` are fixed lists, written before Claude starts. Real programs often need rules with judgment and memory: "only skill scripts", "only files in `data\`", "at most 6 commands per run", "write each decision to a log".

Run:

```
python agent\guarded_agent.py --model haiku "Which region sold the most in data/sales.csv? Save the
    answer in report.txt."
```

Expected result (a real run, 18 seconds; the `[init]` lines are left out, and long paths in the `[guard]` lines are shortened):

```
[skill] Claude picked: analyzing-csv
[tool]  Bash: head -n 5 data/sales.csv; echo; wc -l data/sales.csv
[guard] DENY  Bash: head -n 5 data/sales.csv; echo; wc -l data/sales.csv  (only python commands that
    run a skill script are allowed; use a skill)
[refused] PreToolUse:Bash hook error: only python commands that run a skill script are allowed; use a
    skill
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" summary data/sales.csv
[guard] ALLOW Bash: python "E:/.../analyzing-csv/scripts/csv_tool.py" summary data/sales.csv  (skill
    script, command 1 of 6)
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" groupby data/sales.csv --by region --value amount --agg sum
[guard] ALLOW Bash: python "E:/.../analyzing-csv/scripts/csv_tool.py" groupby data/sales.csv ...
    (skill script, command 2 of 6)
[tool]  Write: E:\may10\claudecodecli_and_python\report.txt
[guard] DENY  Write: E:\may10\claudecodecli_and_python\report.txt  (files may only be written inside
    data/, not E:\may10\claudecodecli_and_python\report.txt)
[refused] PreToolUse:Write hook error: files may only be written inside data/, not
    E:\may10\claudecodecli_and_python\report.txt
[tool]  Write: E:\may10\claudecodecli_and_python\data\report.txt
[guard] ALLOW Write: E:\may10\claudecodecli_and_python\data\report.txt  (writing inside data/ is
    allowed)
[claude] **North** sold the most in `data/sales.csv`, with a total amount of 2150.0. ...

I couldn't save the answer as `report.txt` in the project root. A file-write hook only allows writes
    inside `data/`, so I saved it as **`data/report.txt`** instead. ...

[done] success, turns=8, cost≈$0.0064 (API list-price estimate)
```

How to read it: five decisions, each with its reason:

#	Planned call	Decision	What happened next
1	`head ...; wc -l ...`	**denied**: not a skill script	Claude read the reason, and used the skill's script instead
2	the skill's `summary` command	**allowed**, command 1 of 6	it ran
3	the skill's `groupby` command	**allowed**, command 2 of 6	it ran
4	Write `report.txt` in the project folder	**denied**: outside `data\`	Claude chose another place
5	Write `data\report.txt`	**allowed**	Claude saved the answer there, and told you why

Delete the practice file afterwards: `del data\report.txt`

How it works: the rules are plain Python in `agent\guard.py`, so the tests can check them without Claude (`python -m unittest discover -s tests -k AgentGuard -v`, 5 tests). `guarded_agent.py` connects them to Claude Code as a **PreToolUse hook**, a Python function inside your program:

```
async def check_tool(hook_input, tool_use_id, context):
    allowed, reason = guard.check(hook_input["tool_name"], hook_input["tool_input"])
    return {"hookSpecificOutput": {"hookEventName": "PreToolUse",
                                   "permissionDecision": "allow" if allowed else "deny",
                                   "permissionDecisionReason": reason}}

options.hooks = {"PreToolUse": [HookMatcher(matcher="Bash|PowerShell|Write|Edit",
    hooks=[check_tool])]}
```

Part	What it does
`HookMatcher(matcher="Bash|PowerShell|Write|Edit", ...)`	Claude Code calls `check_tool` before every call of these four tools

Part	What it does
`guard.check(...)`	your rules. `guard` is one object for the whole run, so it can count commands
`"permissionDecision": "deny"`	the call doesn't run. Claude gets `permissionDecisionReason` as the reason, and can try another way

Why a hook, and not `can_use_tool`? The first version of this program used the SDK's `can_use_tool` function. In its real run, the guard never saw the skill's `python` commands. Claude Code asks `can_use_tool` only about calls that **no allow rule** covers, and the skill's `allowed-tools: Bash(python *)` is such a rule. A PreToolUse hook runs before every matching call, whatever the rules say.

The same program with `claude -p`: add `--cli`.

```
python agent\guarded_agent.py --cli --model haiku "Which region sold the most in data/sales.csv? Save
    the answer in report.txt."
```

Expected result (a real run, 15 seconds; the `[init]` lines are left out, the answer is shortened):

```
[tool]  Read: E:\may10\claudecodecli_and_python\data\sales.csv
[tool]  Write: E:\may10\claudecodecli_and_python\report.txt
[guard] DENY  Write: E:\may10\claudecodecli_and_python\report.txt  (files may only be written inside
    data/, not E:\may10\claudecodecli_and_python\report.txt)
[refused] PreToolUse:Write hook error: files may only be written inside data/, not
    E:\may10\claudecodecli_and_python\report.txt
[claude] The write hook only allows files inside `data/`, so I'll save the report there instead.
[tool]  Write: E:\may10\claudecodecli_and_python\data\report.txt
[guard] ALLOW Write: E:\may10\claudecodecli_and_python\data\report.txt  (writing inside data/ is
    allowed)
[claude] North sold the most, with a total amount of 2150 across four rows. ...
[done] success, turns=4, cost≈$0.0086 (API list-price estimate)
```

This time, Claude read the small CSV file with `Read`, which the guard allows, so there was no shell command to judge. The same rules decided about both writes. Delete `data\report.txt` again.

How `--cli` does it: with `claude -p`, the hook can't be a function inside your program. It is a separate program, `agent\guard_hook.py`, which Claude Code starts before each matching tool call. It uses the same rules from `guard.py`:

	SDK way	`--cli` way
Where the hook is	`check_tool()`, a function in the program	`agent\guard_hook.py`, its own program
How it is switched on	`options.hooks = {"PreToolUse": [HookMatcher(...)]}`	`--settings` with the same hook as JSON, made by `hook_settings()` in `run_agent.py`
How it gets the planned call	a Python dictionary	the same data as JSON on stdin
How it answers	returns the dictionary	prints the same dictionary as JSON on stdout
The command counter	a number in the `Guard` object, in memory	a temporary file named by the environment variable `GUARD_STATE`: each call is a new process, which remembers nothing
The `[guard]` lines	printed by the function	written to stderr; `--include-hook-events` makes Claude Code pass them back as JSON lines

A hook script like this also works in `.claude\settings.json`, for the interactive `claude` window (Lesson 9, Exercise 8). Then it applies to every Claude Code session in the project.

Step 16 - Which one to choose

Every job in this part can be done both ways. The difference is how much of the work your program does itself. Here is the whole picture:

Your situation	Simpler with	Why	Shown in
One question from cmd, a `.bat` file, or another program	`claude -p`	one command, nothing to install	Steps 9, 11
Summarize, translate or check a file, and save the answer	`claude -p` with | and >	text in, text out, no program needed	Step 12
A scheduled job	`claude -p` in a `.bat` file	small, no Python program, no SDK	Step 13
A computer where you can't install the SDK	`claude -p`	it needs only Claude Code	
Claude as an LLM inside your Python program, one question at a time	`claude -p` through `subprocess`	standard library only	Step 10
Show each step while it happens	either	the SDK gives Python objects; with `claude -p` you read JSON lines (`run_agent.py --cli`)	Steps 4, 11
Many test questions, stop each run as soon as Claude chose a skill	either	`routing_eval.py`, with `--sdk` for the SDK way	Lesson 5
A chat, or many quick follow-up questions	the SDK (`ClaudeSDKClient`)	one process stays open: 0.9 seconds per follow-up, against 5.6 seconds with `--resume`	Step 14
Your own rules decide about each tool call	the SDK (a Python hook)	the rule is a function with your program's memory; with `claude -p` it is a hook script with a state file	Step 15
Your Python functions as tools	the SDK (`@tool`)	the functions run inside your program; with `claude -p` you write a separate tool server program	Steps 7, 8

Rule of thumb: start with `claude -p`. Switch to the SDK as soon as your Python program must **take part while Claude works**. For example, it must hold a conversation, decide about tool calls with its own data, or offer its own functions. All of it works with `claude -p` too, but then you write the extra pieces yourself. You keep a session id, write a hook script, and write a tool server.

Every program of the course, both ways:

Program	Default	Other way	What the other way needs
`agent\ask_claude.py`	`claude -p`	`--sdk`	the SDK
`agent\api_equivalent.py`	`claude -p`	`--sdk`	the SDK
`agent\run_agent.py`	the SDK	`--cli`	nothing: the same flags the SDK passes (only `--skills` works differently, Step 11)
`agent\chat_agent.py`	the SDK	`--cli`	`--resume <session_id>` for each follow-up
`agent\guarded_agent.py`	the SDK	`--cli`	the hook script `agent\guard_hook.py`
`agent\custom_tools_agent.py`	the SDK	`--cli`	the tool server `agent\tutor_mcp_server.py` (Step 8)
`agent\effort_probe.py`	the SDK	`--cli`	the hook script `agent\effort_hook.py` (Step 24)
`tools\run_cost.py`	`claude -p`	`--sdk`	the SDK (Step 28)
`tools\routing_eval.py`	`claude -p`	`--sdk`	the SDK (Lesson 5, Step 8)

The `--cli` ways need only the standard library. One unit test proves it: it blocks every import of the SDK, then loads all `--cli` parts (`python -m unittest discover -s tests -k need_no_sdk -v`). All tests of both ways: `-k BothWays` (19 tests).

Part 5 - Reasoning effort: how much Claude thinks

The idea: before it answers, Claude **thinks**: it reasons in tokens that you don't see by default (Lesson 0, section 7). Thinking tokens count as output tokens, so they use your Pro allowance. **Effort** tells Claude how much to think. With an API key, you would pass it in your request. With Pro, you pass it to Claude Code, and Claude Code sends it for you. It works the same in `claude -p` and in the SDK.

Step 17 - Where to set the effort

Where	How	Example
`claude -p`	the option `--effort`	`claude -p "..." --model sonnet --effort low`
Agent SDK	the option `effort=`	`ClaudeAgentOptions(model="sonnet", effort="low")`
this course's programs	the option `--effort`	`python agent\ask_claude.py --effort low "..."`, `python agent\run_agent.py --effort low "..."`
the interactive `claude` window	the command `/effort`	`/effort low`
default for every session	the setting `effortLevel` in `settings.json`, or the environment variable `CLAUDE_CODE_EFFORT_LEVEL`	`"effortLevel": "low"`
one skill, while it runs	`effort:` in its SKILL.md frontmatter	`effort: low` (Part 6)

The levels: `low`, `medium`, `high`, `xhigh`, `max`. Without a setting, the model's default is used. The Claude Code documentation says:

Fact	What it means for you
Opus 5.5, Sonnet 5.5 and Haiku 5.5 already default to `medium`	only `low` can save anything compared with the default
You can't turn thinking off on these models. `MAX_THINKING_TOKENS=0` and the SDK's `thinking={"type": "disabled"}` have no effect on them	effort is the only control
Effort is a hint, not a limit	Claude still decides per question how much to think. An easy question may get no thinking at all
"You are charged for all thinking tokens generated, even when collapsed or redacted"	hidden thinking still uses your allowance

Step 18 - Measure it yourself

What: you ask the same question with `--effort low` and with `--effort high`, and compare the thinking tokens.

Why: effort helps more on some models and questions than on others. Measuring is the only way to know for your own jobs.

Run: (each run uses a little allowance)

```
python agent\ask_claude.py --model sonnet --effort low "How many integers from 1 to 10000 are
    divisible by 3 or by 5, but not by 7? Answer with the number only."

python agent\ask_claude.py --model sonnet --effort high "How many integers from 1 to 10000 are
    divisible by 3 or by 5, but not by 7? Answer with the number only."
```

Expected result (real runs):

```
4001

--- {"is_error": false, "model_usage": ["claude-sonnet-5-5"], "thinking_tokens": 98,
    "cost_estimate_usd": 0.0066911, "duration_ms": 4817} ...

4001

--- {"is_error": false, "model_usage": ["claude-sonnet-5-5"], "thinking_tokens": 140,
    "cost_estimate_usd": 0.0071031, "duration_ms": 5018} ...
```

How to read it: both answers are right (4001). `thinking_tokens` is how much Claude thought: 98 with `low`, 140 with `high`. `ask_claude.py` reads it from `modelUsage` in the JSON that `claude -p --output-format json` prints. Without `ask_claude.py`, you find it there yourself.

More real measurements, 3 runs per level, the same question:

Model	`--effort low`	`--effort high`	Answers
Sonnet 5.5	94, 88, 91 thinking tokens (median 91)	120, 103, 171 (median 120)	all 4001
Haiku 5.5	354, 280, 321 (median 321)	345, 298, 317 (median 317)	all 4001

What the measurements show	Why it matters
On Sonnet, `low` thought about 25% less than `high`	effort works, but the saving on one short question is small
On Haiku, effort made no measurable difference	the small model decides its thinking itself; don't expect savings there
The model matters much more. In these runs, one question cost about **$0.0004 to $0.001 on Haiku**, and **about $0.007 on Sonnet** (API-price estimates)	choosing the model is the biggest lever for your allowance
But a cheaper model also makes more mistakes. In Lesson 0, section 4, Haiku chose no skill in 3 of 20 routing tests, and Opus passed all 20	test a cheaper model on your own jobs before you rely on it

Don't compare the cost of single runs. The first run in a series also pays for writing the cache (the reusable start of the request). So it can look more expensive, although it thought less. Compare `thinking_tokens`.

Step 19 - Effort in your agent programs

What: `run_agent.py` and `ask_claude.py` both accept `--effort`.

Run:

```
python agent\run_agent.py --model haiku --effort low "Which region sold the most in data/sales.csv?"
```

Expected result (a real run; the `skills discovered` line is left out):

```
[init] auth: claude.ai subscription (Pro/Max usage, no API billing)
[init] model: claude-haiku-5-5

[skill] Claude picked: analyzing-csv
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" summary data/sales.csv
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/analyzing-
    csv/scripts/csv_tool.py" groupby data/sales.csv --by region --value amount --agg sum
[claude] North sold the most in `data/sales.csv`, with a total amount of 2,150.0. The other regions
    were East at 910.0, West at 900.0 and South at 670.0.

[done] success, turns=5, cost≈$0.0055 (API list-price estimate)
```

How it works: one more option in `build_options()` in `run_agent.py`:

```
return ClaudeAgentOptions(
    ...
    model=model,      # None = your Claude Code default; "haiku" uses less of your plan
    effort=effort,    # None = the model's default; "low" thinks less, "high" more
)
```

In `ask_claude.py`, the `claude -p` way adds `["--effort", effort]` to the command, and the SDK way passes `effort=effort`. Try the SDK way with the question from Step 18:

```
python agent\ask_claude.py --sdk --model sonnet --effort low "How many integers from 1 to 10000 are
    divisible by 3 or by 5, but not by 7? Answer with the number only."
```

Expected result (a real run):

```
4001

--- {"is_error": false, "thinking_tokens": 85, "cost_estimate_usd": 0.0056171, "duration_ms": 1936}
    ...
```

85 thinking tokens: the same range as the `claude -p` runs with `low` in Step 18.

Also good to know: `claude -p` has `--max-budget-usd`, which stops a run above an estimated cost. It was not tested with a Pro login while writing this course.

Step 20 - Your API code, translated to Pro

What: `agent\api_equivalent.py` does what this typical API program does, with your Pro login instead of an API key. It has the same settings at the top: `MODEL`, `EFFORT`, `MAX_TOKENS`, `PROMPT`.

```
import anthropic

client = anthropic.Anthropic()          # needs ANTHROPIC_API_KEY; billed separately from Pro
response = client.messages.create(
    model="claude-sonnet-5-5",
    max_tokens=4096,
    thinking={"type": "adaptive"},
    output_config={"effort": "low"},
    messages=[{"role": "user", "content": "Explain Python decorators briefly."}],
)
print(response.content[0].text)
print("Input tokens:", response.usage.input_tokens)
print("Output tokens:", response.usage.output_tokens)
```

Why: you know the API way. This step shows each part's Pro counterpart, so you can move your own API code over.

Each part, translated:

API code	Pro with `claude -p`	Pro with the Agent SDK
`client = anthropic.Anthropic()`	no client: the `claude` program holds your login	`query(...)` starts Claude Code with your login
`model="claude-sonnet-5-5"`	`--model sonnet` (or the full ID)	`model="sonnet"`
`output_config={"effort": "low"}`	`--effort low`	`effort="low"`
`thinking={"type": "adaptive"}`	nothing to set: the 5.5 models always think adaptively	the same
`max_tokens=4096`	the environment variable `CLAUDE_CODE_MAX_OUTPUT_TOKENS=4096`	`env={"CLAUDE_CODE_MAX_OUTPUT_TOKENS": "4096"}`
`system="..."`	`--system-prompt "..."`	`system_prompt="..."`
`messages=[{"role": "user", "content": ...}]`	the text after `-p`	`prompt=...`
no tools	`--tools ""`	`tools=[]`

API code	Pro with `claude -p`	Pro with the Agent SDK
`response.content[0].text`	`data["result"]` from `--output-format json`	`ResultMessage.result`
`response.usage`	`data["usage"]`	`ResultMessage.usage`
billed per token to the API account	uses your Pro allowance	the same

Not every API parameter has a Claude Code option, for example `temperature`.

How it works, the core of the `claude -p` way:

```
cmd = [
    claude, "-p", prompt,
    "--model", model,                   # model=MODEL
    "--effort", effort,                 # output_config={"effort": EFFORT}
    "--system-prompt", SYSTEM,          # system=...
    "--output-format", "json",          # one JSON object: the answer and the usage
    "--tools", "",                      # no tools: a plain question and answer, like the API call
    "--disable-slash-commands",         # no skills either
    "--no-session-persistence",         # don't save it as a resumable session
]
env = {**os.environ, "CLAUDE_CODE_MAX_OUTPUT_TOKENS": str(MAX_TOKENS)}     # max_tokens=MAX_TOKENS
data = json.loads(subprocess.run(cmd, capture_output=True, text=True, encoding="utf-8",
    env=env).stdout)
print(data["result"])
```

Run: (with the settings at the top of the file: Sonnet, `low`)

```
python agent\api_equivalent.py
```

Expected result (a real run, the answer shortened):

```
A decorator is a function that takes another function and returns a modified version of it, letting
    you add behavior without changing the original code.
...
Input tokens: 830 (new 2, written to cache 828, read from cache 0)
Output tokens: 343 (of which thinking 0)
```

Now change only the effort, on Opus, with a question that needs reasoning:

```
python agent\api_equivalent.py --model opus --effort low --prompt "How many integers from 1 to 10000
    are divisible by 3 or by 5, but not by 7? Answer with the number only."

python agent\api_equivalent.py --model opus --effort high --prompt "How many integers from 1 to 10000
    are divisible by 3 or by 5, but not by 7? Answer with the number only."
```

Expected result (two real runs):

```
4001

Input tokens: 4309 (new 2, written to cache 4307, read from cache 0)
Output tokens: 77 (of which thinking 73)

4001

Input tokens: 4305 (new 2, written to cache 1358, read from cache 2945)
Output tokens: 100 (of which thinking 96)
```

How to read it:

Line	Meaning
`Input tokens: 4309`	everything sent to the model. Your prompt is only a few tokens. Claude Code adds its own text (settings, environment), even with `--system-prompt` and `--tools ""`. In these runs, it was 830 to 4,309 tokens. With the API, only your system prompt and message would be sent
`new 2, written to cache ..., read from cache ...`	Claude Code caches the text it sends; reading from the cache is cheaper. The second run re-used 2,945 tokens from the first

Line	Meaning
`Output tokens: 77 (of which thinking 73)`	what Claude wrote. Thinking counts as output, as with the API
`low` 73 thinking against `high` 96	the same model with a different effort, the same right answer (4001). The decorator question had 0 thinking at both levels: easy questions get no thinking

The same through the SDK: `python agent\api_equivalent.py --sdk`. A real run printed an answer of the same kind, and `Input tokens: 4026 (new 2, written to cache 1107, read from cache 2917)`, `Output tokens: 467 (of which thinking 0)`.

Part 6 - Control the effort of each skill, with the same model

The idea: you keep one model, for example Opus, for everything. But a lookup skill that runs one Python command doesn't need deep thinking, while a skill that must interpret results does. So you set the effort for the whole session, and you let each skill set its own effort while it runs. Everything in this part was measured with Opus 5.5.

Step 21 - Set the effort for a whole session

What: the ways to choose the effort for everything Claude does in a session, with the same model.

Why: the session effort is used for every step that no skill changes, for example the step where Claude chooses a skill.

Where	How	Example
when you start Claude Code	`--effort`	`claude --model opus --effort low`
inside a running session	`/effort`	`/effort low`, later `/effort high`. `/effort auto` goes back to the default
for one prompt only	the word `ultrathink` anywhere in your prompt	`ultrathink: check these invoice totals`
for every session	`"effortLevel": "low"` in `settings.json`	(Lesson 0, section 7 shows how to edit that file)
for every session started from one cmd window	the environment variable `CLAUDE_CODE_EFFORT_LEVEL`	`set CLAUDE_CODE_EFFORT_LEVEL=low`, then `claude`
`claude -p`	`--effort`	`claude -p "..." --model opus --effort low`
the Agent SDK	`effort=`	`ClaudeAgentOptions(model="opus", effort="low")`
this course's programs	`--effort`	`python agent\run_agent.py --model opus --effort low "..."`

Without any of these, Opus 5.5, Sonnet 5.5 and Haiku 5.5 use `medium` (Part 5).

Step 22 - Give each skill its own effort

What: one line in a skill's frontmatter: `effort: low` (or `medium`, `high`, `xhigh`, `max`).

Why: the Claude Code documentation says frontmatter effort "applies when that skill or subagent is active, overriding the session level". So each skill can bring the effort its job needs.

How to see it: `agent\effort_probe.py` is `run_agent.py` plus a hook that prints the effort level Claude Code reports before each tool call. (A hook gets this level in its input, as `effort.level`.)

Run 1: without an `effort:` line. Since Step 27, the project's `converting-units` has `effort: low` as the last frontmatter line. Open `.claude\skills\converting-units\SKILL.md` in Notepad, delete that line, and save. Then run:

```
python agent\effort_probe.py --model opus "Convert 26.2 miles to kilometers."
```

Expected result (a real run):

```
[skill] Claude picked: converting-units
[effort] medium before Skill: converting-units
[tool]  PowerShell: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/convert.py" convert 26.2 mi km
[effort] medium before PowerShell: python
    "E:/may10/claudecodecli_and_python/.claude/skills/converting-units/scripts/conv
[claude] 26.2 miles is about **42.16 km** (42.164813 km). ...
[done] success, turns=4, cost≈$0.1896 (API list-price estimate)
[usage] claude-opus-5-5: output 321 tokens, of which thinking 21
```

Do: put the line `effort: low` back, just before the second `---`, and save.

Run 2: the same command again.

Expected result (a real run):

```
[skill] Claude picked: converting-units
[effort] medium before Skill: converting-units
[tool]  PowerShell: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/convert.py" convert 26.2 mi km
[effort] low    before PowerShell: python
    "E:/may10/claudecodecli_and_python/.claude/skills/converting-units/scripts/conv
[claude] 26.2 miles is **42.16 km**. The exact figure is 42.164813 km, using 1 mile = 1.609344 km.
    ...
[done] success, turns=4, cost≈$0.0371 (API list-price estimate)
[usage] claude-opus-5-5: output 265 tokens, of which thinking 0
```

How to read it:

Line	What it shows
`[effort] medium before Skill`	Claude chose the skill with the **session** effort: `medium`, Opus's default. The skill wasn't active yet
`[effort] low before PowerShell`	after the skill loaded, its `effort: low` was in effect, for running the script and writing the answer
`thinking 0` (without the line: 21)	Claude didn't think at all for the rest of the job

It also works when you start the skill yourself: `python agent\effort_probe.py --model opus "/converting-units 26.2 mi km"` showed `[effort] low` before the command, and `thinking 0`.

Don't compare the `cost≈` numbers of these two runs: the first run in a series also pays for writing the cache (Part 5, Step 18). Compare the thinking tokens.

The file is now as it is in the project again.

Step 23 - Which effort wins?

What: three more real runs, with the skill's `effort:` against the session's effort.

Session	Skill	Effort before the Skill call	Effort after the skill loaded
`--effort high`	`effort: low`	high	**low**: the skill wins
`--effort low`	`effort: high`	low	**high**: the skill wins
`set CLAUDE_CODE_EFFORT_LEVEL=high`	`effort: low`	high	**high**: the environment variable wins

How to read it: this matches the documentation: frontmatter effort overrides the session level "but not the environment variable". So don't set `CLAUDE_CODE_EFFORT_LEVEL` permanently if you want your skills' effort lines to work. Use `--effort`, `/effort` or `effortLevel` for the session instead.

Step 24 - How a skill gets its own effort, request by request

The puzzle: a skill's instructions are only text that is added to Claude's context. Text can't change how much the model thinks. So how can a skill have its own effort?

The answer: the `effort:` line is not read by Claude, but by **Claude Code**, the program. A SKILL.md has text for Claude, and settings for Claude Code:

Part of the SKILL.md	Read by	What happens with it
`description:`	Claude	it is in the skill list Claude gets with every request (Lesson 2)
the instructions after the second `---`	Claude	Claude Code adds them to the conversation when the skill is loaded
`allowed-tools:`, `effort:`, `model:`, `context:`, `disable-model-invocation:`	Claude Code	settings for how Claude Code runs this skill. Claude doesn't obey them; Claude Code applies them

Every step is a new request. Claude Code doesn't hold one long call to the model. For every step, it sends a new request, with the model name, the **effort level**, its instructions and tools, and the whole conversation so far. (That is why Step 28 shows the context being re-read at every step.) So the effort is a setting of each request, like the model name: it is not text in the prompt.

The steps of a real run, where `converting-units` has `effort: low` and the session uses `medium`:

#	Who	What happens	Effort of the request
1	Claude Code	sends request 1: your question, and the skill list	**medium**: the session
2	Claude	answers: "call the Skill tool with `converting-units`"	
3	Claude Code	runs the Skill tool: adds the skill's instructions to the conversation, and notes `effort: low` for the rest of this answer	
4	Claude Code	sends request 2: the conversation, now with the instructions	**low**: the skill
5	Claude	answers: "run `python ... convert.py convert 26.2 mi km`"	
6	Claude Code	runs the Python script on your computer (no tokens), and adds the printed JSON to the conversation	
7	Claude Code	sends request 3	**low**
8	Claude	writes the answer for you	
9	you	type your next prompt: its requests use **medium** again	

It is the same agent, the same model and the same conversation all the time. Only the effort setting that Claude Code sends with the later requests changes.

See it yourself: `effort_probe.py` accepts several prompts, and asks them one after the other in the same conversation.

Run: (uses a little allowance)

```
python agent\effort_probe.py --model sonnet "Convert 26.2 miles to kilometers." "Now read the file
    data/article.txt and tell me its first sentence."
```

Expected result (a real run; the `[usage]` lines are left out):

```
[you]   Convert 26.2 miles to kilometers.
[skill] Claude picked: converting-units
[effort] medium before Skill: converting-units
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/convert.py" convert 26.2 mi km
[effort] low    before Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/conv
[claude] 26.2 miles is about **42.16 km**. That's marathon distance, and the script gave 42.164813
    km.

[done] success, turns=4, cost≈$0.0179 (API list-price estimate)

[you]   Now read the file data/article.txt and tell me its first sentence.
[tool]  Read: E:\may10\claudecodecli_and_python\data\article.txt
[effort] medium before Read: E:\may10\claudecodecli_and_python\data\article.txt
[claude] The file opens with a title, "Why small tools beat big prompts". The first sentence of the
    body text is: ...

[done] success, turns=2, cost≈$0.0258 (API list-price estimate)
```

How to read it: in the first answer, the effort was `medium` until the skill was loaded, then `low` (steps 1 to 8 of the table). The second prompt didn't use a skill: its request went out with `medium` again (step 9). The skill's effort lasted until the end of its answer, and no longer.

The same with `claude -p`: add `--cli` to the same command. The hook is then the script `agent\effort_hook.py`, and the second prompt continues the conversation with `--resume`. A real run printed the same levels: `medium` before the Skill, `low` before the script's command, and `medium` again before the `Read` of the second prompt. Both ways print the line with the same function: `effort_line()` in `effort_hook.py`.

Step 25 - A forked skill: its own agent, model and effort

What: with `context: fork` in its frontmatter, a skill doesn't run in your conversation. Claude Code starts a separate helper agent (a **subagent**) for it. The Claude Code documentation says: "The skill content becomes the subagent's prompt. The subagent doesn't see your conversation history." And with `context: fork`, the skill's `model:` "sets the forked subagent's model".

Why: in Step 22, the skill changed only the effort of the main agent's requests. A forked skill is a second agent, with its own requests. So it can have its own model too, also when Claude chose the skill by itself. (Without `context: fork`, the skill's `model:` line was not used when Claude chose the skill: Step 28.)

Do: open `.claude\skills\converting-units\SKILL.md` in Notepad. After the line `effort: low`, add two lines, so the frontmatter ends like this:

```
effort: low
context: fork
model: haiku
---
```

Save the file.

Run: the main agent uses Opus

```
python agent\effort_probe.py --model opus "Convert 26.2 miles to kilometers."
```

Expected result (a real run):

```
[skill] Claude picked: converting-units
[effort] medium before Skill: converting-units
[tool]  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
    units/scripts/convert.py" convert 26.2 mi km
[effort] low    before Bash (in subagent general-purpose): python
    "E:/may10/claudecodecli_and_python/.claude/skills/converting-units/scripts/conv
[claude] 26.2 miles is **42.164813 km**, or about **42.16 km**. That's the length of a marathon. ...
[done] success, turns=2, cost≈$0.0275 (API list-price estimate)
[usage] claude-opus-5-5: output 181 tokens, of which thinking 22
[usage] claude-haiku-5-5: output 157 tokens, of which thinking 0
```

How to read it:

Line	What it shows
`[effort] medium before Skill`	the main agent (Opus, `medium`) chose the skill
`[effort] low before Bash (in subagent general-purpose)`	the subagent ran the script, at the skill's `low` effort. `general-purpose` is the default kind of subagent (the `agent:` line can choose another)
`[usage] claude-opus-5-5 ...`	the main agent's work: choosing the skill, and writing your answer from the subagent's result
`[usage] claude-haiku-5-5 ...`	the subagent's work, on Haiku, with no thinking

Clean up: remove the two lines `context: fork` and `model: haiku` again, and save.

Ordinary or forked skill?

	Ordinary skill	**Skill with `context: fork`**
Who runs it	the same agent, in your conversation	a separate subagent, with its own context
Sees your conversation	yes	no: the instructions must contain everything it needs
Its own `effort:`	yes (Step 22)	yes
Its own `model:`	in these tests only when you start it with `/skill-name` (Step 28)	yes, also when Claude chooses it
Extra cost	none	the subagent builds its own context from the start
Good for	short jobs: run a script, report the result	jobs that need another model, or big reading jobs that would fill your conversation (Lesson 4, Step 5)

Step 26 - Low or high: prove it with two copies of one skill

What: an experiment where **only** the effort differs. You make two copies of `calculating-dates` that are identical except for two frontmatter lines, and ask the same question through each.

Why: to prove that the same agent really works with a different effort when a different skill runs. The system prompt and the model stay the same. You also see what that changes in the result.

Do:

1. Make the two copies:

   ```
   xcopy /e /i .claude\skills\calculating-dates .claude\skills\dates-low
   xcopy /e /i .claude\skills\calculating-dates .claude\skills\dates-high
   ```

   Expected result, for each copy: the copied files, and `5 File(s) copied` (your number can differ: Python's __pycache__ files are copied too, if they exist).
2. Open `.claude\skills\dates-low\SKILL.md` in Notepad. Change `name: calculating-dates` to `name: dates-low`, and `effort: high` to `effort: low`. Save.
3. Open `.claude\skills\dates-high\SKILL.md`. Change only the name: `name: dates-high`. It keeps `effort: high`. Save.

Now the two files differ only in `name:` and `effort:`. Both copies are in the project at the same time, so every run below gets **the same** system prompt and skill list.

Run: the same question through each copy, with `/name` so you choose the copy. Each command is a fresh session. Run each twice. (Each run uses some allowance, more with Opus.)

```
python agent\effort_probe.py --model opus "/dates-low How many working days are there from 2026-12-21
    to 2027-01-08, if 2026-12-24, 2026-12-25, 2026-12-31 and 2027-01-01 are holidays? Answer with the
    number and one sentence."

python agent\effort_probe.py --model opus "/dates-high How many working days are there from 2026-12-
    21 to 2027-01-08, if 2026-12-24, 2026-12-25, 2026-12-31 and 2027-01-01 are holidays? Answer with
    the number and one sentence."
```

Expected result (real runs on the course machine, round 1, shortened):

```
/dates-low
[tool]  PowerShell: python ".../dates-low/scripts/dates.py" business_days_between --start 2026-12-21
    --end 2027-01-08 --holidays ...
[effort] low    before PowerShell: ...
[tool]  Grep: {'pattern': 'def business_days_between', ...}
[effort] low    before Grep:
[claude] **11 working days** if you count both 2026-12-21 (a Monday) and 2027-01-08. The script
    reported 10 because it doesn't count the start date. ...
[usage] claude-opus-5-5: output 595 tokens, of which thinking 171

/dates-high
[tool]  PowerShell: python ".../dates-high/scripts/dates.py" business_days_between --start 2026-12-21
    --end 2027-01-08 --holidays ...
[effort] high   before PowerShell: ...
[claude] The script returned 10, but counting by hand I get 11: 3 + 3 + 5 across the three weeks.
    I'll check whether the script leaves out one of the end dates.
[tool]  PowerShell: python ".../dates-high/scripts/dates.py" --list
[effort] high   before PowerShell: ...
[claude] **11 working days.** ... The script's `business_days_between` returned **10** because it
    doesn't count the start date. ...
[usage] claude-opus-5-5: output 677 tokens, of which thinking 192
```

All four runs:

Round	Skill copy	Effort at every step	Thinking tokens	Steps after the first command	Answer
1	`dates-low`	low	171	read the function with Grep	11; the script counts from the day after the start
1	`dates-high`	high	192	said "I'll check", ran `--list`	11; the same explanation
2	`dates-low`	low	175	ran `--list`	11; the same explanation
2	`dates-high`	high	201	said "I'll check", ran `--list`	11; the same explanation

How to read it:

- **The proof:** same project, same system prompt and skill list, same model (Opus), same question, and instructions that are identical word for word. Only the `effort:` line differs. The hook reported `low` at every step of `/dates-low`, and `high` at every step of `/dates-high`. Claude Code sent each request with the skill's effort (Step 24).
- **The effect:** `high` thought more in both rounds, but only about 12% to 15% more (192 against 171, 201 against 175). Both copies checked how the script counts, and all four answers were right. `business_days_between` counts the days **after** the start date, up to and including the end date. So it gives 10 where you'd count 11.
- **An earlier single pair of runs** of this question, when Claude chose the skill itself, gave 208 thinking tokens with `low` and 373 with `high`. There, the `low` run explained the script wrongly ("doesn't count the end date"). The four runs above show that this was not the rule: `low` was right every time. One pair of runs is too little to judge an effort level.

- **So:** effort reliably changes the setting, but how much it changes the thinking depends on the question. Here the difference was small. `low` fits skills where the script does the work and Claude reports the result. For skills where Claude must check or interpret, try both levels on your own questions, several times, before you decide.

One request, one effort: the effort belongs to a whole request. Inside one request, all the text is handled with the same effort. A skill changes the effort of the requests that are sent while it is active (Step 24). It can't give one part of a request's text a different effort.

Clean up: delete both copies:

```
rmdir /s /q .claude\skills\dates-low

rmdir /s /q .claude\skills\dates-high
```

Step 27 - Plan the effort of your skills

What: the effort that each skill of this project has, as the last line of its frontmatter, with the reason. Check any of them with `effort_probe.py`, and change the line if your questions need more or less thinking.

See them all:

```
findstr /s /b "effort:" .claude\skills\SKILL.md
```

Expected result:

```
.claude\skills\analyzing-csv\SKILL.md:effort: low
.claude\skills\analyzing-excel\SKILL.md:effort: low
.claude\skills\analyzing-text\SKILL.md:effort: low
.claude\skills\calculating-dates\SKILL.md:effort: high
.claude\skills\calculating-invoices\SKILL.md:effort: high
.claude\skills\clear-todos\SKILL.md:effort: low
.claude\skills\converting-units\SKILL.md:effort: low
.claude\skills\plotting-charts\SKILL.md:effort: medium
.claude\skills\tracking-todos\SKILL.md:effort: low
```

Skill	Effort	Why
`converting-units`	low	one command; Claude only reports the result (Step 22)
`tracking-todos`, `clear-todos`	low	one command each; the script does all the work
`analyzing-text`	low	one command; the numbers come from the script
`analyzing-csv`, `analyzing-excel`	low	usually one or two commands; raise it to `medium` if your questions need several steps
`plotting-charts`	medium	Claude also looks at the PNG and checks it against the numbers
`calculating-invoices`	high	money: Claude writes the invoice JSON, and must fix every problem `validate` reports
`calculating-dates`	high	the question often needs checking how the script counts (Step 26)

What the effort does not change: the Python interpreter, its packages, and the script's result. The script runs the same and costs no tokens. Effort only changes how much Claude thinks around it.

Step 28 - Where the tokens of a skill run go

What: `tools\run_cost.py` runs one prompt with `claude -p`, and prints each step with its model, and the tokens per model.

Why: to see how big the thinking part is compared with the rest, before you expect savings.

Run: (uses a little allowance)

```
python tools/run_cost.py "Convert 26.2 miles to kilometers." --model sonnet
```

Expected result (a real run with Sonnet, made before the skill got its `effort: low` line):

```
Steps:
  1. claude-sonnet-5-5  thinks
  2. claude-sonnet-5-5  Skill converting-units
  3. claude-sonnet-5-5  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
     units/sc
  4. claude-sonnet-5-5  writes the answer

Tokens per model:
  claude-sonnet-5-5: context re-read 110,413, new context 10,929, output 269 (thinking 31),
    cost≈$0.0575
```

How to read it:

Part	Meaning
`context re-read 110,413`	at **every** step, Claude reads its whole context again: Claude Code's instructions, the tool and skill lists, your question, and the results so far. That is about 27,000 tokens per step. Re-reading from the cache is cheaper than new text, but it still counts
`new context 10,929`	text that entered the context for the first time, like the skill's instructions and the script's output
`output 269 (thinking 31)`	what Claude wrote, including its thinking

The lesson: for a one-command skill, thinking is a small part of the tokens, so `effort: low` saves only a little there. On a skill where Claude thinks more, `low` used 11% to 44% less thinking than `high` in real runs (Step 26). Fewer steps save tokens too. Let the script do the whole job in one command, and write the SKILL.md so Claude needs only that one command.

The same with the SDK: add `--sdk`. `run_cost.py` then makes the same run with `query()`. Two options make it the same run as `claude -p`. The first is `system_prompt={"type": "preset", "preset": "claude_code"}`: Claude Code's own instructions (without it, the SDK sends none, Step 2). The second is `setting_sources=["user", "project", "local"]`: the settings files that `claude -p` reads. A real run with Sonnet, the same day as a `claude -p` run:

```
Steps:
  1. claude-sonnet-5-5  thinks
  2. claude-sonnet-5-5  Skill converting-units
  3. claude-sonnet-5-5  Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/converting-
     units/sc
  4. claude-sonnet-5-5  writes the answer

Tokens per model:
  claude-sonnet-5-5: context re-read 51,514, new context 27,063, output 243 (thinking 31),
    cost≈$0.1158
```

It shows the same four steps, and almost the same output (243 against 226 tokens with `claude -p` that day). The context numbers differ, for two reasons. The SDK starts its own `claude.exe` from the pip package (Step 2). Here that was Claude Code 2.1.301, against 2.1.308 for the `claude` command, with slightly different built-in instructions. And its cache was cold, so more text was written to the cache ("new context": 27,063 against 13,634) and less read from it (51,514 against 106,979).

If you also want a cheaper model: a skill can set `model: haiku` next to `effort:`. In real `claude -p` runs, it was used only when you started the skill yourself (`/converting-units 26.2 mi km`: Haiku, $0.0029 instead of $0.0575). When Claude chose the skill, the run stayed on the session model. Check it in your window with Ctrl+O, which shows the model of each answer.

Beyond your own computer (optional)

- Your skill folders also work on claude.ai (upload them there) and with the Claude API (which needs an API key). Only 6 frontmatter fields are allowed there (Lesson 4, Step 7).

- "Managed Agents" is an Anthropic service that runs agents on Anthropic's computers. It needs an API key. You don't need it for this course.

Next: Lesson 7 - Skills that need pip packages

Lesson 7 - Skills that need pip packages (an isolated venv)

Goal: a skill whose Python script uses packages that don't come with Python, such as pandas, openpyxl or matplotlib. You check, with commands, where the packages are, why activation matters, and what happens when it is missing.

Before you start: open cmd, `C:\pyenvsmy\claudeclioct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`.

The idea in one paragraph

Python comes with a **standard library** (json, argparse, datetime, ...). Seven skills of this course use only that, so any Python can run them. Other packages, like pandas, are installed with `pip`, and they are installed **into one Python installation**. This course installs them into an isolated **venv**, `C:\pyenvsmy\claudeclioct26`, so the system Python stays clean. A skill's command just says `python`. Which Python that is depends on the PATH of the window where you started `claude` (Lesson 0). So: activate, then start `claude`, and the skills find their packages.

This project has two such skills:

Skill	Packages it needs	Script	What it does
`analyzing-excel`	pandas, openpyxl	`excel_tool.py`	sheets, statistics and pivot tables of .xlsx files
`plotting-charts`	pandas, openpyxl, matplotlib	`chart.py`	bar and line charts as PNG pictures

The venv was created with `C:\Python312\python.exe -m venv C:\pyenvsmy\claudeclioct26`.

Step 1 - See that pandas is only in the venv

What: `pip show` prints where a package is installed.

Why: to see with your own eyes that a package belongs to one Python, and not to the others.

Run: (with the activated venv's Python)

```
python -m pip show pandas
```

How it works: `python -m pip` runs pip **of this Python**. That is safer than typing just `pip`, which could belong to another Python.

Expected result: (the important lines)

```
Name: pandas
Version: 3.0.6
Location: C:\pyenvsmy\claudeclioct26\Lib\site-packages
```

Run: (with the system Python)

```
C:\Python312\python.exe -m pip show pandas
```

Expected result:

```
WARNING: Package(s) not found: pandas
```

How to read it: pandas is installed **in the venv's folder** (`Location:`), and the system Python doesn't have it. Activation decides which Python the word `python` means, for you **and** for every command that Claude runs.

Step 2 - See which packages a skill needs

What: each package skill has a `requirements.txt` file in its `scripts` folder.

Why: anyone who gets the skill folder knows what to install, and can install it with one command.

Run:

```
type .claude\skills\plotting-charts\scripts\requirements.txt
```

Expected result:

```
# Packages for chart.py. Install into the activated virtual environment:
#   python -m pip install -r requirements.txt
pandas>=2.2
openpyxl>=3.1
matplotlib>=3.8
```

How to read it: one package per line. `>=2.2` means "version 2.2 or newer". Lines starting with `#` are comments.

Step 3 - Install the packages (only once)

What: you install everything listed in both `requirements.txt` files.

Why: the scripts can't run without them.

Run: (in the activated window, from the project folder)

```
python -m pip install -r .claude\skills\analyzing-excel\scripts\requirements.txt -r
    .claude\skills\plotting-charts\scripts\requirements.txt
```

How it works: `-r file` means "install everything listed in this file". You can give `-r` more than once.

Expected result: on the course machine they are already installed, so pip only reports `Requirement already satisfied: ...` for each package. On a new venv, pip downloads and installs them.

Check it:

```
python .claude/skills/plotting-charts/scripts/chart.py check

{"python": "C:\\pyenvsmy\\claudeclioct26\\Scripts\\python.exe", "pandas": "3.0.6", "matplotlib":
    "3.11.2"}
```

How to read it: the `check` command prints which Python ran the script, and the package versions. Both are what you want.

If you see something else: if `"python"` shows `C:\\Python312\\...`, the venv is not active, and you installed into the wrong Python. Activate the venv, and install again.

Step 4 - Use the Excel script yourself

What: you run two commands of `excel_tool.py`, as Claude would.

Why: to see what the skill gives Claude before Claude uses it.

Run:

```
python .claude/skills/analyzing-excel/scripts/excel_tool.py sheets data/sales.xlsx
```

Expected result:

```
{"file": "data/sales.xlsx", "sheets": [{"sheet": "Sales", "rows": 12, "columns": ["date", "region",
    "product", "units", "amount"]}, {"sheet": "Targets", "rows": 4, "columns": ["region",
    "target"]}]}
```

How to read it: the workbook has two sheets. `Sales` has 12 rows with 5 columns, and `Targets` has 4 rows.

Run: (a pivot table: total `amount` per `region`)

```
python .claude/skills/analyzing-excel/scripts/excel_tool.py pivot data/sales.xlsx --sheet Sales --
    index region --values amount
```

Expected result:

```
{"index": "region", "columns": null, "values": "amount", "agg": "sum", "rows": [{"region": "East",
    "amount": 910}, {"region": "North", "amount": 2150}, {"region": "South", "amount": 670},
    {"region": "West", "amount": 900}, {"region": "Total", "amount": 4630}]}
```

How to read it: pandas summed `amount` for each region and added a `Total` row (4630).

How it works: the script opens the workbook with pandas (`pd.ExcelFile(path, engine="openpyxl")`). pandas uses openpyxl to read .xlsx files. Then the script builds the table with `pd.pivot_table(...)`. These few lines of pandas do the work of many lines of plain Python. That is why a skill uses packages.

Step 5 - Draw a chart and look at it

What: you draw a line chart of the monthly sales.

Run:

```
python .claude/skills/plotting-charts/scripts/chart.py line data/sales.xlsx --sheet Sales --x date --
    y amount --freq M
```

Expected result:

```
{"written": "charts/line_amount_by_date.png", "kind": "line", "x": "date", "y": "amount", "agg":
    "sum", "freq": "M", "points": [{"date": "2026-01", "amount": 720.0}, {"date": "2026-02",
    "amount": 1050.0}, {"date": "2026-03", "amount": 2260.0}, {"date": "2026-04", "amount": 600.0}]}
```

Run: (opens the picture)

```
start charts\line_amount_by_date.png
```

How to read it: `--freq M` grouped the dates by month. The script did two things:

1. It saved the picture: `"written": "charts/line_amount_by_date.png"`.
2. It printed the numbers it drew: `"points"`. **Why:** Claude reads the exact values from here, so it doesn't have to estimate them from the picture. The skill also tells Claude to open the PNG with its Read tool. Claude can look at images, and check that the chart is right.

Step 6 - The skill's text is as simple as any other

What: you look at the commands in the chart skill's SKILL.md.

Why: to see that a package skill needs no special setting: activation chooses the venv, not the skill.

Run:

```
findstr /n "python" .claude\skills\plotting-charts\SKILL.md
```

Expected result:

```
5:  - Bash(python *)
6:  - PowerShell(python *)
17:python "${CLAUDE_SKILL_DIR}/scripts/chart.py" bar data/sales.csv --x region --y amount --agg sum
23:python "${CLAUDE_SKILL_DIR}/scripts/chart.py" line data/sales.csv --x date --y amount --agg sum --
    freq M
```

How to read it: the commands start with plain `python`, exactly like the standard-library skills. Nothing in the skill says where the venv is. It works because you activated the venv before you started `claude`.

The skill also contains one sentence for Claude: if a package is missing, **don't install anything**. Instead, ask the user to activate the venv and restart `claude`. **Why:** installing software is your decision, and Claude would install into whatever Python it has, possibly the system Python.

Step 7 - See what happens without the venv

What: you run the Excel script with the system Python, as if you forgot to activate.

Why: to see the message Claude gets in that case, and the code that produces it.

Run:

```
C:\Python312\python.exe .claude/skills/analyzing-excel/scripts/excel_tool.py sheets data/sales.xlsx
```

Expected result:

```
{"error": "Python package 'pandas' is not installed in C:\\Python312\\python.exe", "hint": "Claude
    Code was started without the skills' virtual environment. Activate the venv, then start claude
    again. Missing packages: python -m pip install -r
    \"E:\\may10\\claudecodecli_and_python\\.claude\\skills\\analyzing-
    excel\\scripts\\requirements.txt\""}
```

How to read it: instead of a long Python error report, the script prints a short JSON message: **what** is missing, **which** Python was used, and **how** to fix it.

How it works: these lines at the top of the script:

```
try:
    import pandas as pd
    import openpyxl
except ImportError as missing:
    print(json.dumps({
        "error": f"Python package '{missing.name}' is not installed in {sys.executable}",
        "hint": "Claude Code was started without the skills' virtual environment. Activate the venv,
    ...",
    }))
    sys.exit(2)
```

Part	Meaning
`try: ... except ImportError as missing:`	try the imports. If a package is missing, Python raises `ImportError`, and the `except` part runs instead of a crash
`missing.name`	the missing package's name, here `pandas`
`sys.executable`	the full path of the Python that is running, here `C:\Python312\python.exe`
`sys.exit(2)`	ends with exit code 2, so Claude Code marks the command as failed

Use the same guard in your own package scripts.

`chart.py` also has the line `matplotlib.use("Agg")` before it imports `pyplot`. "Agg" makes matplotlib draw into files only. **Why:** Claude runs scripts without a screen, so a script must never try to open a window. A window would wait for someone to close it, and the command would never end.

Step 8 - Use the skills in Claude

What: you ask Claude four questions that need the package skills.

Why: to see the whole chain: activated window, `claude`, skill, `python` of the venv, pandas.

Run: `claude` (in the activated window), then type these one at a time.

Expected result: the skill Claude chose on the course machine is in brackets:

```
Which sheets and columns are in data/sales.xlsx?                    [analyzing-excel]
Build a pivot of amount by region and product from data/sales.xlsx.  [analyzing-excel]
Make a bar chart of total amount per region from data/sales.csv.     [plotting-charts]
Plot the monthly sales amount from data/sales.xlsx as a line chart.  [plotting-charts]
```

Watch for, with the bar chart request (a real test run):

1. the **Skill** tool with `plotting-charts`;

2. a **Bash** or **PowerShell** command: `python ".../plotting-charts/scripts/chart.py" bar data/sales.csv --x region --y amount --agg sum`;
3. the **Read** tool with `charts\bar_amount_by_region.png`: Claude looks at the picture;
4. the answer: "North 2,150, East 910, West 900, South 670 ... checked the image visually against these numbers."

If items 2 and 3 are folded into one line like `Read 1 file, ran 1 shell command`, press **Ctrl+O** to see them, with the JSON that `chart.py` printed (Lesson 0, section 7). Press **Esc** to go back.

If you see something else: `Python package 'pandas' is not installed` means you started `claude` without activating. Type `/exit`, activate, and start `claude` again.

Type `/exit` when you are done.

Step 9 - Run the tests both ways

What: you run the unit tests with the venv's Python, and then with the system Python.

Why: to see how the tests handle a missing package: they skip, they don't fail.

Run: (in the activated window)

```
python -m unittest discover -s tests
```

Expected result: (the end)

```
Ran 74 tests in 31.351s

OK
```

Run: (with the system Python)

```
C:\Python312\python.exe -m unittest discover -s tests
```

Expected result: (the end)

```
Ran 74 tests in 11.987s

OK (skipped=6)
```

How to read it: two groups of tests belong to this lesson:

Group	Tests	What it checks	With the system Python
`PackageSkills`	6	runs `excel_tool.py`, `chart.py` and `tools\make_sample_xlsx.py` with the same Python that runs the tests	**skipped**, not failed, because pandas is missing. The tests start with `@unittest.skipUnless(has_packages("pandas", "openpyxl", "matplotlib"), ...)`
`WrongInterpreter`	1	the message from Step 7	runs. It starts the scripts with `python -S`, which tells Python not to load installed packages. So pandas is "missing" even in the venv. It checks that the script prints the helpful message and doesn't crash

The system Python run is faster (12.0 instead of 31.4 seconds). The 6 skipped tests are the slow ones: loading pandas and drawing charts takes time.

Step 10 - The same skills from a program

What: you run a package skill with `claude -p`, without the window.

Why: in a program, nobody can answer Claude Code's permission questions (Lesson 6).

The problem: in the normal `claude` window, Claude Code may ask you "use this skill?", and you answer. In a program (`claude -p`, or the Agent SDK), nobody can answer, so the answer is automatically "no". A real test showed this: the skill didn't load, so its `allowed-tools` didn't apply. Every `python` command was refused too.

The fix: allow the Skill tool in advance.

Run: (uses a little allowance)

```
claude -p "Make a bar chart of total amount per region from data/sales.csv" --model haiku --
    allowedTools Skill
```

Expected result: Claude's answer, for example with the four region totals, and a new `charts\bar_amount_by_region.png`. Once the skill is loaded, its own `allowed-tools` allows the `python` commands.

With the Agent SDK, put `"Skill"` in `allowed_tools`, as `agent\run_agent.py` does (Lesson 6).

Another way, not used here: write the venv's path into the skill

A SKILL.md could call the venv's Python by its full path instead of plain `python`:

```
C:/pyenvsmy/claudeclioct26/Scripts/python.exe "${CLAUDE_SKILL_DIR}/scripts/chart.py" bar ...
```

This is called "pinning the interpreter".

	Activation (this course)	**Pinned path**
Works if you forget to activate	no: you get the Step 7 message	yes
The commands in SKILL.md	short: `python ...`	long, and `allowed-tools` must name the same path
Works on another computer	yes, with any venv that has the packages	only if the venv is in exactly that folder

That is why this course uses activation. If you pin a path in your own skills, `python tools/skills_lint.py` checks that the `python.exe` exists, and warns if it doesn't.

Try it yourself: add a package

You learn: the whole cycle for a new package, from `pip install` to Claude using it.

1. Pick a small library, for example `holidays` (public holidays of many countries).
2. In the activated window: `python -m pip install holidays`.
3. Check: `python -m pip show holidays`. `Location:` must be the venv's folder (`C:\pyenvsmy\claudeclioct26\Lib\site-packages`).
4. Add a line `holidays` to the `requirements.txt` of the skill that will use it.
5. Write a function that uses it, with the same `try ... except ImportError` guard as in Step 7.
6. Run your script yourself in cmd first, and then ask Claude.
7. Add a test, and run `python -m unittest discover -s tests` (Lesson 5).

Next: Lesson 8 - Messy input: Claude reads, Python computes

Lesson 8 - Messy input: Claude reads, Python computes

Goal: a skill whose Python script needs clean, exact input. But people type requests in any order, with numbers as words, and with typos. You see how the SKILL.md makes Claude turn messy text into one clean command. You also learn exactly how the placeholders `$ARGUMENTS`, `$0`, `$1`, ... work, and when to use them.

Before you start: open cmd, `C:\pyenvsmy\claudecliоct26\Scripts\activate`, and `cd /d "E:\may10\claudecodecli_and_python"`.

The idea in one paragraph

Each side does what it is good at. **Claude** understands language: "seven and five add", "4 radnias sinus" and "squera root 67" are clear to it, in any order, with typos. **Python** computes exactly, but only from clean input. So the script accepts one strict form, `calc.py <operation> <numbers> [--unit ...]`, and the SKILL.md teaches Claude to translate the request into that form. If Claude gets it wrong, the script answers with a JSON error and a hint, and Claude fixes its call (Lesson 3, Step A3).

Step 1 - Install the two example skills

What: the folder `examples\skills\` has two skills: `calculating-math` (the calculator) and `showing-arguments` (a small demo for Step 6). They are not in `.claude\skills\`, so Claude Code doesn't load them until you copy them there.

Why they are separate: the other lessons show outputs for the project's 9 skills. When you have finished this lesson, you remove the copies again (Step 7), or keep the calculator if you like it.

Run:

```
xcopy /e /i examples\skills\calculating-math .claude\skills\calculating-math

xcopy /e /i examples\skills\showing-arguments .claude\skills\showing-arguments
```

Expected result:

```
examples\skills\calculating-math\SKILL.md
examples\skills\calculating-math\scripts\calc.py
2 File(s) copied
examples\skills\showing-arguments\SKILL.md
1 File(s) copied
```

(If you ran the unit tests before, Python's __pycache__ files are copied too, and the count is higher.)

Check them: `python tools/skills_lint.py` now ends with `11 skills, 0 error(s).` It shows three `WARN` lines for `showing-arguments`: Step 6 explains them.

Step 2 - Run the strict script yourself

What: you call `calc.py` with clean input, and with input it doesn't accept.

Why: to see what the script expects, and what Claude gets back when its call is wrong.

Run:

```
python .claude/skills/calculating-math/scripts/calc.py add 7 5

python .claude/skills/calculating-math/scripts/calc.py cos 45 --unit degrees

python .claude/skills/calculating-math/scripts/calc.py cos 23

python .claude/skills/calculating-math/scripts/calc.py add seven 5
```

Expected result:

```
{"operation": "add", "numbers": [7.0, 5.0], "unit": null, "result": 12.0}
{"operation": "cos", "numbers": [45.0], "unit": "degrees", "result": 0.7071067812}
{"error": "'cos' needs --unit degrees or --unit radians.", "hint": "If the user gave no unit, use
    degrees and say so in the answer."}
{"error": "Not a number: ['seven', '5'].", "hint": "Write numbers as digits: 'seven' -> 7, 'twenty-
    three' -> 23."}
```

How it works: the operations are a table in the script: how many numbers each one takes, and the Python function that computes it:

```
OPERATIONS = {
    "add": (2, lambda a, b: a + b),
    "subtract": (2, lambda a, b: a - b),
    "multiply": (2, lambda a, b: a * b),
    "divide": (2, lambda a, b: a / b),
    "sqrt": (1, math.sqrt),
    "sin": (1, math.sin),
    "cos": (1, math.cos),
}
ALIASES = {"plus": "add", "times": "multiply", "sine": "sin", "sinus": "sin", "root": "sqrt", ...}
```

Why the script is strict:

Rule in the script	Why
numbers only as digits	turning "twenty-three" into 23 is language work: Claude does it better than a word list
`sin` and `cos` need `--unit`	23 degrees and 23 radians give very different results. The script never guesses, so Claude must decide, and the SKILL.md tells it how
a fixed number of numbers per operation	`add 7` is a mistake, not a request; the error says so
a few other names (`plus`, `sinus`, `root`)	cheap to accept, and it saves Claude a retry
every error has a `hint`	Claude reads the hint and fixes its call by itself

Step 3 - Read the SKILL.md: how it teaches the translation

Run:

```
type .claude\skills\calculating-math\SKILL.md
```

How to read it, part by part:

Part of the file	What it does	Why
`description:` with "in any word order, with numbers written as words ("seven") or with typos ("sinus", "squere root")"	tells Claude **when** to choose the skill: also for messy requests	without these words, a request like "4 radnias sinus" may not look like math (Lesson 2)
`effort: low`	Claude thinks little while the skill runs	the translation is easy; the script does the math (Lesson 6, Part 6)
"Never compute the result yourself"	Claude runs the script even for "7 + 5"	otherwise Claude answers easy questions from memory (Lesson 5)
`What the user typed after /calculating-math ...: $ARGUMENTS`	puts your typed text into the instructions	explained in Step 6
"Find the three parts": operation, numbers, unit	turns a vague job into three small decisions	clear steps are followed more reliably than "understand the request"
the rules for order and missing units	"subtract 5 from 7" is 7 - 5; no unit means degrees, said in the answer	these are the cases where a guess would give a wrong number
the table "The user writes / Command"	eight examples of messy text and the right command	examples teach the pattern faster than rules alone

Part of the file	What it does	Why
"say how you read the request"	the answer shows the interpretation	you see at once if Claude misread "23 cosine"

Step 4 - Messy input, started with `/calculating-math`

What: you type the skill's name and then the messy text.

Run: start `claude`, and type each line:

```
/calculating-math seven and five add
/calculating-math 4 radnias sinus
/calculating-math squera root 67
```

Expected result: an answer like `sin 4 rad = -0.7568` for the second line. Press **Ctrl+O** to see the command Claude built from your text (Lesson 0, section 7).

To demonstrate smart query processing by the LLM, we use typos on purpose in the math queries: "radnias" for radians and "squere" for square.

All nine of the examples, real runs on the course machine (with `claude -p` and Haiku, about $0.003 each as an API-price estimate):

You typed after `/calculating-math`	Command Claude ran (after `calc.py`)	Answer (first line)
`seven and five add`	`add 7 5`	7 + 5 = 12
`add seven and five`	`add 7 5`	7 + 5 = **12**
`23 cosine`	`cos 23 --unit degrees`	cos 23° = 0.9205
`cosine of 45 degrees`	`cos 45 --unit degrees`	cos 45° = 0.7071 (0.7071067812 to 10 decimal places)
`sinus 3 radians`	`sin 3 --unit radians`	sin 3 rad = 0.1411
`4 radnias sinus`	`sin 4 --unit radians`	sin 4 rad = -0.7568
`45 squere root`	`sqrt 45`	The square root of 45 is **6.7082039325**.
`squera root 67`	`sqrt 67`	The square root of 67 is **8.1853527719**.
`34 square root`	`sqrt 34`	√34 = 5.8309518948

How to read it: every request became one clean command, and every number came from the script. "23 cosine" had no unit, so Claude used degrees, as the SKILL.md says. It said so in a later line of its answer ("I read "23 cosine" as the cosine of 23 with no unit given, so I used degrees").

Step 5 - Messy input, and Claude chooses the skill

What: the same kind of requests, without `/calculating-math`. Claude must recognize that they are math, choose the skill, and translate them.

Run: in `claude`, type:

```
4 radnias sinus
```

Expected result (a real run with Opus: its steps, and the answer):

```
claude-opus-5-5    thinks
claude-opus-5-5    Skill calculating-math
claude-opus-5-5    Bash: python "E:/may10/claudecodecli_and_python/.claude/skills/calculating-
    math/sc...
claude-opus-5-5    writes the answer
answer: I took your request as the sine of 4 radians.  **sin(4 rad) = -0.7568024953**
```

The command was `calc.py sin 4 --unit radians`. In the same test, `seven and five add` and `squera root 67` also went to the skill by themselves, and gave 12 and 8.1853527719.

How to read it: the description's words "in any word order", "numbers written as words" and "typos" let Claude match these requests to the skill (Lesson 2). Without the `/name`, the run also has the step where Claude chooses the skill, so it costs a little more.

Step 6 - `$ARGUMENTS`, `$0`, `$1`: how arguments reach a skill

What: when you type `/skill-name some words`, Claude Code puts "some words" into the skill's text before Claude reads it. It is not magic, and not a hack: it is documented in the Claude Code skills documentation, in the section "Pass arguments to skills". Claude Code does it the same way as it replaces `${CLAUDE_SKILL_DIR}` (Lesson 1, Step 4).

The placeholders (from the documentation):

Placeholder	Becomes
`$ARGUMENTS`	everything you typed after the skill name, exactly as typed
`$0`	the **first** word. Counting starts at 0
`$1`, `$2`, ...	the second, third, ... word
`$ARGUMENTS[0]`, `$ARGUMENTS[1]`, ...	the same as `$0`, `$1`, ...
`$issue`	a named argument, if the frontmatter has `arguments: [issue, branch]`: `$issue` is the first word, `$branch` the second

Words are split like in a shell: text in quotes counts as **one** argument. To write a real dollar sign followed by a digit, like a price of $5, write `\$5`.

See it: `showing-arguments` only repeats the placeholders. Its SKILL.md says:

```
Reply with exactly these four lines and nothing else:

ALL: [$ARGUMENTS]
FIRST: [$0]
SECOND: [$1]
THIRD: [$2]
```

Run: in `claude`, type each line:

```
/showing-arguments seven and five add
/showing-arguments "seven and five" add
```

Expected result (two real runs):

```
ALL: [seven and five add]
FIRST: [seven]
SECOND: [and]
THIRD: [five]

ALL: ["seven and five" add]
FIRST: [seven and five]
SECOND: [add]
THIRD: [$2]
```

How to read it:

- `$ARGUMENTS` is always the whole text. `$0`, `$1`, `$2` are single words, counted from 0.
- With quotes, `"seven and five"` was one argument: `$0`.
- There was no third argument, so `$2` stayed as it was: the documentation says an indexed placeholder without an argument "stays in the content unchanged".

- The linter warns about each `$0`, `$1`, `$2` in this file (`WARN ... '$0' is replaced by word 1 typed after the skill name`). Here they are wanted. In most skills, a `$` before a digit is a mistake, like a price written as `$5` (Lesson 2, Step 4).

Which one for messy input? `$ARGUMENTS`. With `seven and five add`, `$0` is "seven", `$1` is "and" and `$2` is "five". The positions mean nothing, because people don't type in a fixed order. So the calculator gives Claude the whole text with `$ARGUMENTS`, and Claude finds the parts. Use `$0`, `$1`, ... only for commands with a fixed order, like the documentation's example `/migrate-component SearchBar JavaScript TypeScript` with `Migrate the $0 component from $1 to $2.`

And when Claude chooses the skill itself? The documentation says that both you and Claude can pass arguments, but not in detail how Claude fills them. Don't depend on them: Claude always has your message in the conversation. That is why the calculator's line says "(empty if you chose this skill yourself)", and why Step 5 worked without `/calculating-math`.

Step 7 - Clean up

What: remove the two copies, so the other lessons' outputs match again (9 skills).

Run:

```
rmdir /s /q .claude\skills\calculating-math
rmdir /s /q .claude\skills\showing-arguments
```

The originals stay in `examples\skills\`. The unit tests check `calc.py` there: run `python -m unittest discover -s tests -k CalculatingMath -v` (5 tests).

Summary

You want	Do this	Shown in
exact results from messy requests	a strict script, and a SKILL.md that teaches Claude to translate the request into one clean command	Steps 2 to 4
Claude to choose the skill for messy requests	describe the messiness in the description: any order, words for numbers, typos	Step 5
your typed text inside the instructions	`$ARGUMENTS`	Step 6
fixed-order arguments	`$0`, `$1`, ... (counted from 0), or named `arguments:`	Step 6
a dollar sign before a digit	`\$5`	Step 6

Next: Lesson 9 - Exercises

Lesson 9 - Exercises

Goal: practice everything from Lessons 1 to 8 on your own. Each exercise says what you will learn, why it matters, the steps, the result to expect, and what to do if you see something else.

Before every exercise: start the usual way (Lesson 0, "Start here"): activate the venv, and `cd /d "E:\may10\claudecodecli_and_python"`.

After every exercise, run the two free checks in cmd. They show at once if a change broke a skill file or a script (Lesson 5 explains both):

```
python tools/skills_lint.py
python -m unittest discover -s tests
```

Expected result: `9 skills, 0 error(s).` and `Ran 74 tests ... OK`. If you want, also run the real routing test. It uses a little of your Pro allowance:

```
python tools/routing_eval.py --limit 6
```

1. Watch routing happen (5 min)

You learn: that Claude chooses by meaning, not by exact words (Lesson 2).

Why: people never type a request the same way twice. A good description must work for all forms.

Steps: in `claude`, ask the same thing in three ways. Each time, note which skill appears with the **Skill** tool:

- `How long is a 10k race in miles?`
- `10 km → mi`
- `Is a 10k longer than 6 miles?`

Then ask something that needs **no** skill: `Explain what a kilometer is`.

Expected result: most likely `converting-units` for all three, with a `python ... convert.py convert 10 km mi` command, and about 6.21 miles. The last question gets **no** skill: it asks for an explanation, not a conversion.

If you see something else: if one of the three gets no skill, Claude answered from memory. Note which form it was. It is a good candidate for a test question in `tools\routing_cases.json` (Lesson 5).

2. Break a description, then fix it (10 min)

You learn: how much the choice depends on the description, and what the free tools say about a bad one.

Why: you will write descriptions yourself. Seeing a bad one fail makes the rules of Lesson 2, Step 5 concrete.

Steps:

1. Open `.claude\skills\converting-units\SKILL.md` in Notepad. Replace the whole description line with `description: Helps with numbers.` and save.
2. Run the linter:
   ```
   python tools/skills_lint.py
   ```
3. Run the preview:
   ```
   python tools/route_preview.py "convert 5 kg to lb"
   ```
4. Optional (uses allowance): `python tools/routing_eval.py --limit 3`.
5. Put the original description back (copy it from Lesson 1, Step 3), save, and run the linter again.

Expected result (real runs):

Step 2, the linter:

```
[check] converting-units  (auto + /command, description 19 chars)
   WARN  Add a trigger clause such as 'Use when the user asks ...' so Claude knows WHEN to pick it.
```

Step 3, the preview:

```
   2.77  converting-units       shared words: convert
   0.00  tracking-todos         shared words: -
   0.00  plotting-charts        shared words: -
```

How to read it: the linter warns: no "Use when ..." sentence. The preview still puts `converting-units` first, but only because of the word "convert" in the skill's **name** (`converting-units`). "kg", "lb" and every other unit are gone. A request like `How heavy is 5 kg in pounds?` would now share no word at all. Step 4 shows what Claude does. Results can vary, because the name alone still says a lot.

Your answer: in one sentence, explain why the vague description fails. (It doesn't say what the skill does, with which units, or when to use it.)

3. Create a collision on purpose (15 min)

You learn: what happens when two skills compete for the same requests, and how the linter detects it.

Why: with many skills, this is the most common problem (Lesson 4, Step 2).

Steps:

1. Create the folder `.claude\skills\summarizing-tables\`, and in it a file `SKILL.md` with this text. It is a second skill for the same job as `analyzing-csv`:

   ```
   ---
   name: summarizing-tables
   description: Summarizes tables in CSV files - totals, averages and top rows of a column. Use when
       the user asks about totals, averages or rankings in a CSV file.
   ---

   # Summarizing tables
   ```

2. Run the linter:

   ```
   python tools/skills_lint.py
   ```

3. Add a test question that both skills could take to `tools\routing_cases.json`. Add it as a new line before the last `]` (don't forget the comma at the end of the line before it):

   ```
   {"prompt": "What is the average amount in data/sales.csv?", "expect": "analyzing-csv"}
   ```

4. Optional (uses allowance): `python tools/routing_eval.py`. With the collision, Claude may choose `summarizing-tables`, which has no script.

5. Fix the collision. Here the jobs are the same, so **merge**: delete the extra skill (`rmdir /s /q .claude\skills\summarizing-tables`). Run the linter again.

Expected result (a real run of Step 2):

```
[ok] summarizing-tables  (auto + /command, description 148 chars)
...
Description overlap (pairs that may compete for the same request):
   WARN  analyzing-csv <-> summarizing-tables: 0.38 shared words ['average', 'column', 'csv', 'file',
     'ranking', 'rows', 'summarize', 'top', 'total']

10 skills, 0 error(s).
```

How to read it: the new skill itself is `[ok]`: its file is correct. The problem is the pair: 0.38 means 38% of their important words are shared (the warning starts at 0.30). After Step 5, you see `none above 0.30` and `9 skills, 0 error(s)` again. Keep the new test question: it guards against the same mistake later.

If you see something else: `9 skills` instead of `10` in Step 2 means the new SKILL.md is not in the right folder, or its first line is not `---`.

4. Add a new skill with a Python function library (30 min)

You learn: to build a complete skill with a Python script, from an empty folder to a tested skill (Lessons 3 and 5).

Why: this is the main goal of the course: your own Python functions, used by Claude.

The task: a skill that answers `What is my monthly payment for a $250,000 mortgage at 6% over 30 years?`

Steps:

1. Create the folder `.claude\skills\calculating-loans\scripts\`.
2. Copy `fncli.py` into it (Pattern B, Lesson 3):
   ```
   copy .claude\skills\calculating-dates\scripts\fncli.py .claude\skills\calculating-loans\scripts\
   ```
3. Write `scripts\loans.py` with two typed functions, and end it with `fncli.run([...])`, as in `dates.py`:
   - `monthly_payment(principal: float, annual_rate_pct: float, years: int) -> dict`
   - `total_interest(principal: float, annual_rate_pct: float, years: int) -> dict`

   The formula is `principal * r / (1 - (1 + r) ** -n)`, with `r = annual_rate_pct / 100 / 12` (the rate per month) and `n = years * 12` (the number of months).
4. Write `SKILL.md`:
   - a description: what it does, a "Use when ..." sentence, and as the last sentence "Not for invoices or quotes (use calculating-invoices)."
   - `allowed-tools` with `Bash(python *)` and `PowerShell(python *)`
   - a table of commands, with the path in quotes: `"${CLAUDE_SKILL_DIR}/scripts/loans.py"`
   - careful: write dollar amounts as `\$250,000`. A bare `$2` would be replaced by an argument (Lesson 2, Step 4).
5. Add unit tests in a new file `tests\test_loans.py`.
6. Add 3 test questions to `tools\routing_cases.json`: two that must go to `calculating-loans`, and one borderline question that must go to `calculating-invoices`, for example `Make me a quote: 2 laptops at 1200 each plus 20% VAT.`

Try it first yourself. Then compare with the solution below.

Solution, `scripts\loans.py`:

```
"""Loan arithmetic: monthly payment and total interest of a fixed-rate loan."""

import fncli


def monthly_payment(principal: float, annual_rate_pct: float, years: int) -> dict:
    """Monthly payment of a fixed-rate loan (annuity formula)."""
    if principal <= 0 or years <= 0 or annual_rate_pct < 0:
        raise ValueError("principal and years must be > 0, annual_rate_pct must be >= 0")
    r, n = annual_rate_pct / 100 / 12, years * 12
    payment = principal / n if r == 0 else principal * r / (1 - (1 + r) ** -n)
    return {"principal": principal, "annual_rate_pct": annual_rate_pct, "years": years,
            "months": n, "monthly_payment": round(payment, 2)}


def total_interest(principal: float, annual_rate_pct: float, years: int) -> dict:
    """Total interest paid over the whole loan."""
    m = monthly_payment(principal, annual_rate_pct, years)
    paid = m["monthly_payment"] * m["months"]
    return {**m, "total_paid": round(paid, 2), "total_interest": round(paid - principal, 2)}


if __name__ == "__main__":
    fncli.run([monthly_payment, total_interest])
```

Why it is written this way:

- The `ValueError` becomes a JSON error with the right call (Lesson 3, Step B3), so Claude can fix a wrong call by itself.
- `r == 0` is handled separately: the formula would divide by zero for a 0% loan.
- `total_interest` reuses `monthly_payment`, so the two can never disagree.

Solution, `SKILL.md`:

```
---
name: calculating-loans
description: Calculates fixed-rate loans and mortgages - the monthly payment, the total amount paid,
    and the total interest for a principal, an annual interest rate and a term in years. Use when the
    user asks about a mortgage, loan or car-loan payment, or how much interest a loan costs. Not for
    invoices or quotes (use calculating-invoices).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
---

# Calculating loans

Never do loan math in your head. Run the script.

| Question | Command |
|---|---|
| Monthly payment | `python "${CLAUDE_SKILL_DIR}/scripts/loans.py" monthly_payment --principal 250000
    --annual_rate_pct 6 --years 30` |
| Total interest | `python "${CLAUDE_SKILL_DIR}/scripts/loans.py" total_interest --principal 250000 -
    -annual_rate_pct 6 --years 30` |

Write amounts without `\$` and commas: 250000, not \$250,000. Report `monthly_payment` and
`total_interest` as printed, in the user's currency.
```

Solution, `tests\test_loans.py`:

```
import unittest

from test_skill_scripts import cli, load      # the helpers of the course's test file


class CalculatingLoans(unittest.TestCase):
    loans = load("calculating-loans", "loans.py")

    def test_monthly_payment(self):
        self.assertEqual(self.loans.monthly_payment(250000, 6, 30)["monthly_payment"], 1498.88)

    def test_cli_total_interest(self):
        code, out = cli("calculating-loans", "loans.py", "total_interest",
                        "--principal", "250000", "--annual_rate_pct", "6", "--years", "30")
        self.assertEqual((code, out["total_interest"]), (0, 289596.8))
```

One test calls the function, and one runs the command exactly as Claude does (Lesson 5, Step 3).

Check it, in this order:

```
python .claude/skills/calculating-loans/scripts/loans.py monthly_payment --principal 250000 --
    annual_rate_pct 6 --years 30

{"principal": 250000.0, "annual_rate_pct": 6.0, "years": 30, "months": 360, "monthly_payment":
    1498.88}

python -m unittest discover -s tests -k CalculatingLoans -v

test_cli_total_interest (test_loans.CalculatingLoans.test_cli_total_interest) ... ok
test_monthly_payment (test_loans.CalculatingLoans.test_monthly_payment) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.181s

OK

python tools/skills_lint.py
```

Expected result: `[ok] calculating-loans ...`, no overlap warning, and `10 skills, 0 error(s).` Then start `claude` and ask the mortgage question: you should see the Skill tool with `calculating-loans`, and about 1,498.88 per month.

If you see something else: `ModuleNotFoundError: No module named 'fncli'` means `fncli.py` is not in the `scripts` folder (step 2). After this exercise, the test counts in the other lessons are 2 higher (76), and the linter says 10 skills. Keep the skill, or delete the folder and `tests\test_loans.py`.

5. Make a skill file-aware (10 min)

You learn: the `paths:` setting, which limits **automatic** use of a skill to work with matching files.

Why: a skill that only makes sense for certain files then stays out of the way for everything else.

Steps:

1. Add this to the frontmatter of `.claude\skills\analyzing-csv\SKILL.md` (before the closing `---`):

   ```
   paths:
     - "**/*.csv"
   ```

   `**` means "any folder, at any depth", and `*.csv` means "any file whose name ends in .csv".
2. Start `claude` again (skills are read at the start of a session), and ask `Which region sold the most in data/sales.csv?`.
3. Think about it: which requests could now **no longer** reach the skill by themselves?
4. Remove the `paths:` lines again. This project uses descriptions only.

Expected result: in Step 2, the skill is still chosen, because a .csv file is involved. The answer to Step 3: for example CSV text pasted straight into the chat, with no file. You can always start the skill yourself with `/analyzing-csv`.

6. Run a big reading job in a subagent (15 min)

You learn: `context: fork`, which runs a skill in a helper Claude with its own context window (Lesson 4, Step 5).

Why: reading many files fills your conversation, and every later message gets slower and more expensive. A subagent keeps your conversation small.

Steps:

1. Create the file `.claude\skills\summarizing-project\SKILL.md` with this content:

```
---
name: summarizing-project
description: Reads all lesson files of this course and returns a 10-line summary. Use when the
    user asks for a summary or overview of the course lessons.
context: fork
agent: Explore
background: false
---

Read every file in the lessons folder. Then write a summary of the whole course in exactly
10 lines, one main idea per line. Return only the summary.
```

Line	What it does
`context: fork`	runs the skill in a subagent. The text below the frontmatter becomes the subagent's task
`agent: Explore`	uses the built-in helper that can read files but not change them
`background: false`	makes Claude wait for the result before it continues (needs Claude Code 2.1.218 or later; check with `claude --version`)

2. Start `claude`, and type `/context`. It shows how full the context window is and what fills it. Note the number.
3. Type `/summarizing-project`. Wait for the 10-line summary.
4. Type `/context` again.
5. Optional: start a new session, ask `Read all files in the lessons folder and summarize them` **without** the skill, and compare `/context` afterwards.

Expected result: in Step 4, the context grew only by the short summary, not by all 9 lesson files. The subagent read them in its own context window. In Step 5, the context grows much more.

Clean up: `rmdir /s /q .claude\skills\summarizing-project`.

7. Specialized agents from the same skills (15 min)

You learn: to make agents with different skill sets from one project, and the limits of the `skills` list (Lesson 6, Step 6).

Why: a real agent program usually needs only some skills.

Steps:

1. Make a "finance agent" that knows only invoices and dates:

```
python agent\run_agent.py --model haiku --skills calculating-invoices,calculating-dates "What
    weekday is 2026-12-25 and convert 5 km to miles"
```

2. Make a "data agent" that knows only CSV, text and units:

```
python agent\run_agent.py --model haiku --skills analyzing-csv,analyzing-text,converting-units
    "What weekday is 2026-12-25 and convert 5 km to miles"
```

3. Look at the `[skill]` and `[tool]` lines. Run each one twice.

Expected result: the finance agent uses `calculating-dates` for the weekday (Friday), and the data agent uses `converting-units` for the miles (about 3.11). For the part without an offered skill, the agent may answer from its own knowledge. Or it reads the hidden skill's `SKILL.md` with its Read tool, and runs the script anyway. Lesson 6, Step 6 explains why: the list is not a lock.

8. A rule that can't be skipped: a hook (stretch)

You learn: hooks, the way to enforce a rule that Claude can't skip (Lesson 4, Step 6).

Why: a sentence in a SKILL.md is advice. Claude usually follows it, but not always. Claude Code itself enforces a hook.

The goal: make it impossible for Claude's Write and Edit tools to change any file outside `data\`.

How it works: a hook is a command that Claude Code itself runs at a certain moment. Here the moment is `PreToolUse`: just **before** a tool runs. Claude Code sends the planned tool call to your command as JSON on stdin, for example `{"tool_name": "Write", "tool_input": {"file_path": "E:\\...\\notes.txt", ...}, "cwd": "E:\\may10\\claudecodecli_and_python", ...}`.

Your command ends with	Claude Code then
exit code **2**	blocks the tool call, and shows Claude the text your command printed on stderr
exit code **0**	runs the tool call normally

The hook script is ready in the project: `tools/hooks/only_data_folder.py`.

```
event = json.load(sys.stdin)                      # Claude Code sends the planned tool call as JSON
target = Path(event["tool_input"].get("file_path", ""))
allowed = Path(event["cwd"]) / "data"
try:
    target.resolve().relative_to(allowed.resolve())
except ValueError:
    print(f"Blocked by hook: files may only be written inside {allowed}", file=sys.stderr)
    sys.exit(2)                                   # exit code 2 = block the tool call
sys.exit(0)                                       # exit code 0 = allow it
```

`resolve()` turns the path into a full path. `relative_to` raises a `ValueError` when the target file is not inside the `data` folder. In that case the script blocks the call.

Steps:

1. Create the file `.claude\settings.json` in the project, with this content. It switches the hook on for the Write and Edit tools:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          { "type": "command", "command": "python tools/hooks/only_data_folder.py" }
        ]
      }
    ]
  }
}
```

   `"matcher": "Write|Edit"` means: run the hook only before the Write or the Edit tool.

2. Exit and start `claude` again, so the new setting is loaded.

3. Ask: `Create a file test.txt in the project folder with the text hi.`
4. Ask: `Create the file data/test.txt with the text hi.` Then delete `data\test.txt`.
5. Think about it:
   - Why is a hook better than the sentence "only write in data/" in a SKILL.md?
   - Which file changes does this hook **not** stop?
6. Delete `.claude\settings.json` again. Otherwise Claude also can't edit the lessons or skills for you.

Expected result (tested on the course machine): in Step 3, the hook blocks the write, and Claude reports `Blocked by hook: files may only be written inside ...\data`. In Step 4, the write is allowed.

The answers to Step 5: a hook runs every time, and the model can't skip or forget it. It doesn't stop a Python script started with the Bash or PowerShell tool. That script can still write anywhere, because the hook checks only the tools in its `matcher`.

If you see something else: if the write in Step 3 is not blocked, Claude Code didn't load the setting. Check that the file is `.claude\settings.json` (not `.txt`), and that you restarted `claude`.

What you have learned

You started as a user of Claude Code. You finish as someone who can **build the harness around an AI agent**: the tools it uses, the rules it must follow, the tests that prove it works, the program that runs it, and the numbers that show what it costs. Real companies need this work when they put agents into production. You have done each part with your own hands, on your own machine, with real outputs.

Your achievements

You can now	And you have proved it, because you
Make an agent compute **exactly**, with your Python	ran the same script Claude runs, compared outputs, and saw Claude repeat the script's numbers (Lesson 3)
Design **many skills** that don't compete	wrote descriptions with a "Not for ..." boundary, and checked overlaps with the linter (Lesson 4)
Prove which skill Claude chooses	ran 20 real routing questions, and read every PASS and FAIL (Lesson 5, Test 4)
Test an agent for free	ran 74 unit tests, broke a script on purpose, and read the failure (Lesson 5, Step 2)
Run Claude **from your own programs, with only Pro**	used `claude -p` and the Agent SDK, with the same jobs both ways (Lesson 6, Part 4)
Decide **when Claude Code beats a plain model call**	measured five setups on the same data tasks: a plain call for small questions, a lean runtime with one compute rule for tables, at a quarter of the cost (Lesson 6, "Three questions")
Move **API code to Pro**	translated model, effort, max_tokens and usage line by line (Lesson 6, Step 20)
Enforce rules the model can't skip	stopped a file write outside `data\` with a hook, and watched Claude adapt (Lesson 6, Step 15)
Give Claude **your functions as tools**	served them in-process and as a stand-alone MCP server (Lesson 6, Step 8)
Control cost with the same model	set an effort level per skill, and measured the thinking tokens (Lesson 6, Step 26)
See inside every run	read thinking and commands with Ctrl+O, the effort level with a hook, and the tokens with `run_cost.py`
Use **pip packages** safely	ran the same skill with and without the venv, and saw the helpful error instead of a crash (Lesson 7)
Handle **messy input**	let Claude translate "4 radnias sinus" into a strict command that a script checks (Lesson 8)

The parts of an agent harness, and your ready solution for each

You didn't build these parts from zero. Claude Code is a harness that is already built and used every day, and you learned to use it from your own programs. On public benchmarks, the harness alone moved the same model by more than 20 points (Lesson 6, "Why go through Claude Code at all"). What you add is your part: your skills, your rules, your tools and your tests. You are in good company. Anthropic runs almost all of its major agent loops on this harness, and its Python Agent SDK was downloaded 29.9 million times in September 2026. OpenClaw, Hindsight and promptfoo plug Claude Code in where a model API would go. And OpenAI offers its Codex agent to programs through an SDK (Lesson 6, "The big idea").

An agent in a real company is more than a prompt. It has the parts below. For each one, you have a working, tested example in the project folder.

Part	What it does in a real system	Your ready solution	Lesson
Exact tools	calculations, lookups and file work that must be right every time	skills with scripts; `@tool` functions; an MCP server	3, 6
Routing	the right tool for each request, among many	descriptions with "Use when" and "Not for"; `routing_eval.py`	2, 4, 5
Running from code	start the agent from a program, a `.bat` file or a scheduler	`claude -p` programs and SDK programs, both ways	6
Rules and safety	what the agent may never do	hooks (`guard.py`, `only_data_folder.py`), `allowed-tools`, skills only you can start	3, 6, 9
Memory	conversations and data that last	`ClaudeSDKClient`, `--resume`, the to-do list file	3, 6
Cost control	the same results for less allowance	`effort:` per skill, the model per job, `run_cost.py`	0, 6
Quality gates	catch mistakes before users do	linter, 74 unit tests, routing test, the HOW TO TEST section of each file	5
Observability	know what happened, step by step	Ctrl+O, `stream-json` steps, hook lines, token reports	0, 6

Copy-paste solutions for your work

Each row is a job you may have at work, and the folder or file to copy. Change the part in the last column. Keep the structure, the tests and the checks.

At work, use an API key. Your Pro login is for your own, individual use. For a product, a service for other people, or a company's production job, run the same programs with an API key from the Claude Console or a cloud provider. Use it under Anthropic's Commercial Terms (Lesson 6, "When the harness is the right choice"). The code doesn't change; only who pays does.

Your job needs	Copy	Change
Exact numbers in answers: prices, units, dates	`.claude\skills\converting-units\`	the functions, the description
A library of functions Claude can call by name	`calculating-dates\scripts\fncli.py` with your functions	your typed functions and their docstrings
Check the data before acting (orders, payments)	`.claude\skills\calculating-invoices\`	the checks in `validate()`
Data that lasts, plus a delete that only a person may start	`tracking-todos\` and `clear-todos\`	the fields of a record
Excel and CSV reports, charts	`analyzing-excel\`, `plotting-charts\`, with `requirements.txt`	column names, chart types
Messy text from users	`examples\skills\calculating-math\`	the operations and the "user writes / command" table
Claude as an LLM inside a Python program	`agent\ask_claude.py`	the system prompt, the JSON schema
Existing Anthropic API code, now on Pro	`agent\api_equivalent.py`	`MODEL`, `EFFORT`, `PROMPT`
An agent job from code, with each step logged	`agent\run_agent.py` (`--cli` without the SDK)	the allowed tools, `max_turns`
A chat backend with follow-up questions	`agent\chat_agent.py`	where the questions come from
Company rules for commands and files	`agent\guard.py` with `guarded_agent.py` or `guard_hook.py`	the rules in `Guard.check()`
Internal functions or APIs as tools	`agent\custom_tools_agent.py`, `agent\tutor_mcp_server.py`	the `@tool` functions, or the server's `TOOLS` table

Your job needs	Copy	Change
A nightly report with no Python program	`agent\daily_report.bat`	the prompt, the schedule
A quality gate for your skills	`tools\skills_lint.py`, the tests, `tools\routing_eval.py`	`tools\routing_cases.json`
The cost of a job, and where it goes	`tools\run_cost.py`, `agent\effort_probe.py`	the prompt

`claude -p` or the SDK: the rule you now know

The surprise of Lesson 6 is that **neither way always wins**. Both run the same Claude Code, so the answers are the same. What differs is how much your program takes part.

`claude -p` is better when	The SDK is better when	Both work equally well for
you want one command and nothing to install	the program holds a **conversation** (0.9 seconds per follow-up, against 5.6)	one question from Python
the job runs from a `.bat` file or a scheduler	your **rules** need the program's own data and memory	showing each step while it happens
text goes in through a pipe and comes out in a file	your **functions** should be tools without a second program	a routing test, a cost measurement
the computer can't get the SDK	you want each step as a Python object	

Rule of thumb: start with `claude -p`. Switch to the SDK as soon as your program must take part while Claude works. Every program of the course can do both, so you can compare them on your own job.

The numbers to remember

Number	Meaning	Lesson
about 668 tokens	all 9 skill descriptions together, in every request; the scripts cost nothing	1
3 of 20	routing questions Haiku answered without the right skill; Opus passed all 20	0
1,403.36 against 1403.37	Haiku doing invoice math by itself, against the script: one cent off	`invoice.py` header
849 against 39,143	input tokens of one plain question: every switch off, against Claude Code's defaults	6
0.9 against 5.6 seconds	a follow-up question with the SDK, against `claude -p --resume`	6
110,413	context tokens re-read during one short skill run; the thinking was 31	6
12% to 15%	how much more Opus thought at `high` than at `low`, on the same skill; all answers right	6
74	unit tests that check every script and program, for free	5

What to do next, at work

1. Pick one repetitive job that needs exact results. Write its Python function first, and test it.
2. Wrap it as a skill: a description with "Use when" and "Not for", and the command to run.
3. Write five real questions for `routing_cases.json`, and run the routing test.
4. Add a hook for the rule that must never be broken.
5. Run the job from a `.bat` file with `claude -p`, or from your program with the SDK, and measure its cost with `run_cost.py`. Lower the effort where the script does the work.

6. Before it serves other people or runs in production, switch it to your company's API key.

You now know what most Claude Code users never see: what happens inside each step, and how to make it exact, safe, tested and affordable.

Cheat sheet

Start: the venv's `activate.bat`, `cd /d E:\may10\claudecodecli_and_python`, `claude`. `set ANTHROPIC`: no API key.

In a `SKILL.md`	What it does
`description:` "Does X. Use when ... Not for Y (use Z)."	level 1: always in context; Claude chooses by it
the body: `python "${CLAUDE_SKILL_DIR}/scripts/x.py"`	level 2: read only when the skill is chosen
`scripts/*.py`: JSON out; on errors a hint, exit code 1	level 3: run, never read, so free in context
`allowed-tools: Bash(python *)`	its commands run without a permission prompt
`disable-model-invocation: true`	only you can start it, with `/name`
`effort: low` / `context: fork` with `model: haiku`	its own effort / its own subagent and model
`$ARGUMENTS`, `$0`, `$1`	all words after `/name`, the first word, the second word

Job	`claude -p`	Agent SDK
one answer, as JSON	`--output-format json`	`query()`, then `ResultMessage`
each step as it happens	`--output-format stream-json --verbose`	each message as a Python object
model and effort	`--model haiku --effort low`	`model="haiku"`, `effort="low"`
no permission prompts	`--allowedTools "Skill,Bash(python *)"`	`allowed_tools=[...]`
a plain LLM call	`--system-prompt "..." --tools ""`	`system_prompt="..."`, `tools=[]`
`max_tokens`	`set CLAUDE_CODE_MAX_OUTPUT_TOKENS=4096`	`env={...}` with the same variable
follow-up questions	`--resume <session_id>`, new process	`ClaudeSDKClient`, one process, fast
your rule on each tool call	a hook script, `--settings`	`HookMatcher` with a function
your functions as tools	an MCP server, `--mcp-config`	`@tool`, `create_sdk_mcp_server`
better for	no install, `.bat`, schedulers, pipes	chats, rules with program state, own tools

Effort: which setting wins	
1. `set CLAUDE_CODE_EFFORT_LEVEL=...`	beats everything, also the skills' lines
2. a skill's `effort:`	for the requests while the skill works
3. `--effort`, `/effort`, `effortLevel`	the session; without them, the model's default

Check and see	Command
SKILL.md mistakes (free)	`python tools/skills_lint.py`
scripts and programs (free)	`python -m unittest discover -s tests` (74 tests)
Claude's real skill choice	`python tools/routing_eval.py --limit 4 --model haiku`
size, cost and effort of a run	`tools/skill_sizes.py`, `tools/run_cost.py`, `agent/effort_probe.py`
thinking and every command	**Ctrl+O** in claude; `"showThinkingSummaries": true` in settings

Problem	Fix
"pandas is not installed", exit code 2	activate the venv, then start claude again
the wrong skill, or none	sharpen "Use when" and "Not for"; run the routing test
Claude computes by itself	start the skill with `/name`, or use a stronger model

Questions and answers

Test yourself: cover the answer, say yours, then compare. Each answer names the lesson step where you saw it happen.

Setup, Pro and seeing inside

1. Do I need an API key to use Claude from Python? No. Claude Code holds your Pro login; `claude -p` and the Agent SDK both use it (Lesson 6, Part 3).

2. What happens if the environment variable `ANTHROPIC_API_KEY` is set? Claude Code uses it instead of your Pro login, and that API account pays. In `claude -p` and the SDK, this happens without asking. `set ANTHROPIC` shows whether it is set (Lesson 0, Step 8).

3. Why must the venv be activated before I start `claude`? Skills run plain `python`, and Claude Code passes its PATH to them. Only the activated venv puts the Python with pandas and matplotlib first (Lesson 0, Step 1; Lesson 7, Step 7).

4. Why do I see dollar amounts if Pro is a flat price? They are API list-price estimates of each run. On Pro, nothing is charged. Read them as the size of a run (Lesson 0, section 3).

5. How do I see what Claude thinks and which commands it runs? Press Ctrl+O for the transcript view. To read the thinking text too, set `"showThinkingSummaries": true` once in your settings (Lesson 0, Step 9; Lesson 0, Step 10).

What a skill is, and how Claude chooses

6. What are the three loading levels of a skill? Level 1, the description, is in every request. Level 2, the SKILL.md instructions, is read only when the skill is chosen. Level 3, extra files, is read only when needed. Scripts are run, never read (Lesson 1, Step 5).

7. How much does a skill's long Python script cost in context? Nothing: Claude runs it and reads only what it prints. `convert.py` has 222 lines, but Claude sees one line of JSON (Lesson 1, Step 5).

8. How does Claude choose a skill? It compares your request with the names and descriptions of all skills, using the full language model. Then it calls the Skill tool for the best fit, or uses no skill (Lesson 2, Step 1).

9. What makes a good description? It says what the skill does, and has a "Use when ..." sentence with the words users really say. Its last sentence, "Not for ... (use ...)", sends neighboring requests to the right skill (Lesson 2, Step 5).

10. Why must "Not for ..." be the last sentence in this project? The word-counting tools (the preview, and the linter's overlap check) cut it off before they count: a word counter can't read "not". Claude reads it fine (the header of `tools/skilllib.py`; Lesson 9, Exercise 4).

11. How do I make a skill that Claude never starts by itself? `disable-model-invocation: true`. Only you can start it, with `/name`. The course uses it for `clear-todos`, which deletes data (Lesson 2, Step 6; Lesson 3, Step D5).

12. Is the free word-counting preview (`route_preview.py`) a good test of routing? No, it is a rough guess. It finds descriptions that miss the users' words, but only `routing_eval.py` shows what Claude really chooses (Lesson 2, Step 3).

Skills that run Python

13. Why let a script compute instead of Claude? Because the script is exact and testable. In a real run, Haiku did the invoice math itself and got 1,403.36. The script gives 1403.37 (Lesson 3, Pattern C, and the header of `invoice.py`).

14. What should a skill script print? One line of JSON. On an error, it prints JSON with "error" and a "hint", and ends with exit code 1. Then Claude can correct its command and try again (Lesson 3, Step A3).

15. What are the five patterns of Lesson 3? A: one script, several commands. B: call any function by name (`fncli`). C: check the data first, then calculate. D: data that is kept, and a skill only you can start. E: pip packages from your venv (Lesson 3, Part 2).

16. How does a skill find its script, wherever the project folder is? With `"${CLAUDE_SKILL_DIR}/scripts/x.py"`, in quotes, because the path may contain spaces (Lesson 3, Step 2).

17. How can Claude give a script text without writing a file? Through a pipe: `echo text | python text_stats.py stats -` (Lesson 3, Step A4).

18. What does `allowed-tools: Bash(python *)` do? While the skill is active, Claude may run `python ...` commands without a permission prompt (Lesson 3, Step 2).

Many skills

19. How many skills are too many? Every model-invocable description is in every request: 9 skills cost about 668 tokens. More skills also make the choice harder, so give each skill one job and its own words (Lesson 4, Step 3).

20. How can two skills share Python code? One skill calls the other's script by path: `clear-todos` runs `"${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py"` (Lesson 4, Step 4).

21. Where do big reading jobs go? To a subagent: it reads in its own context window and returns only a short result (Lesson 4, Step 5).

Testing

22. Which three tests does the course use, and what does each prove? Unit tests prove the scripts compute right (free). The linter proves the SKILL.md files are correct (free). The routing test proves Claude chooses the right skill (uses allowance) (Lesson 5, Test 1; Lesson 5, Test 2; Lesson 5, Test 4).

23. How do I run only the tests of one file? `-k` with a word from the test names, for example `-k ConvertingUnits` or `-k guard_hook`. Each `.py` file's header names its own words (Lesson 5, Step 4).

24. Why are some tests "skipped" with the system Python? They need pandas, which only the venv has. They are skipped, not failed, because a missing package is a setup question (Lesson 5, Step 5).

25. What do I do when a routing question fails? Read which skill Claude chose, and sharpen the descriptions of both skills. Then run the linter and the routing test again. Lesson 5 shows a real example.

Your own programs: `claude -p` and the SDK

26. What does the Agent SDK really run? Its own copy of Claude Code (`claude.exe`, bundled in the pip package), with your login and skills. It is not a different engine (Lesson 6, Step 2).

27. Which system prompt does the SDK send by default? An empty one, not Claude Code's long instructions. `system_prompt={"type": "preset", "preset": "claude_code"}` gives the normal behavior (Lesson 6, Step 2).

28. How do I get every step from `claude -p`? `--output-format stream-json --verbose`: one JSON line per step. `run_agent.py --cli` reads them and prints the same labels as the SDK way (Lesson 6, Step 11).

29. When is `claude -p` the better choice? One command with nothing to install; `.bat` files and schedulers; text in through a pipe, the answer into a file (Lesson 6, Step 12; Lesson 6, Step 13).

30. When is the SDK the better choice? Conversations (one process stays open: 0.9 seconds per follow-up, against 5.6 with `--resume`); rules that need your program's data; your functions as tools without a second program (Lesson 6, Step 16).

31. Can `claude -p` hold a conversation? Yes: read `session_id` from the last JSON line, and pass `--resume <session_id>` with the next question. Each question starts a new process (Lesson 6, Step 14).

32. Why a PreToolUse hook, and not the SDK's `can_use_tool`, for rules? Claude Code asks `can_use_tool` only about calls that no allow rule covers, and a skill's `allowed-tools` is such a rule. A hook runs before every matching call (Lesson 6, Step 15).

33. How does a hook decide with `claude -p`? It is a separate program, switched on with `--settings`. It gets the planned call as JSON on stdin. It answers with JSON on stdout (`permissionDecision`), or blocks with exit code 2 (Lesson 6, Step 15; Lesson 9, Exercise 8).

34. How do I give Claude my own functions as tools? With the SDK: `@tool` and `create_sdk_mcp_server`, inside your program. With `claude -p`: a small MCP server program, started with `--mcp-config` (Lesson 6, Step 8).

35. What is MCP, really? JSON messages, one per line: `initialize`, `tools/list`, `tools/call`. `tutor_mcp_server.py` does it in about 60 lines of standard-library Python (Lesson 6, Step 8).

36. Is there anything only the SDK can do? Offering only some skills (`skills=[...]`): the SDK says it in its start-up message, and `claude -p` has no flag for it. Everything else works both ways (Lesson 6, Step 11).

37. How do I turn my Anthropic API code into Pro code? `model` becomes `--model`, effort becomes `--effort`, `max_tokens` becomes `CLAUDE_CODE_MAX_OUTPUT_TOKENS`, `system` becomes `--system-prompt`, no tools becomes `--tools ""` (Lesson 6, Step 20).

Effort and cost

38. What is effort? How much Claude thinks before it acts or answers: low, medium, high, xhigh or max. The same model thinks less at low (Lesson 6, Step 17).

39. Can thinking be switched off? No need to try: the 5.5 models always think adaptively, as much as the question needs. Effort makes that thinking shorter or longer (Lesson 6, Step 20).

40. Which effort setting wins? The environment variable `CLAUDE_CODE_EFFORT_LEVEL` beats everything. A skill's `effort:` beats the session's `--effort`, `/effort` or `effortLevel` (Lesson 6, Step 23).

41. The skill's text joins the same conversation. How can it have its own effort? Claude Code sends each request with the effort in force at that moment. While the skill works, its requests carry the skill's effort. After its answer, the session's effort is back (Lesson 6, Step 24).

42. How do I give a skill its own model? `context: fork` with `model: haiku`: the skill runs in a subagent with that model and effort (Lesson 6, Step 25).

43. How much does low effort save? It depends on the question. In the real runs, `low` used 11% to 44% less thinking than `high`. In the careful two-copies test on Opus, the difference was small, and all four answers were right (Lesson 6, Step 26).

44. Where do the tokens of a short skill run go? Mostly into the context, read again at every step: 110,413 tokens re-read, against 31 tokens of thinking (Lesson 6, Step 28).

45. Is a cheaper model a free saving? No. Haiku answered 3 of 20 routing questions without the right skill; Opus passed all 20. Test the cheaper model on your own jobs first (Lesson 0, section 4).

pip packages and messy input

46. How does a skill use pandas without breaking on other computers? Its packages are in `requirements.txt`, installed into the venv. The script starts with an import guard: when a package is missing, it prints a helpful error and ends with exit code 2 (Lesson 7, Step 7).

47. Why should the script be strict when users type messy text? Claude is good at understanding "4 radnias sinus". Python is good at computing. Claude translates the text into a strict command, and the strict script refuses anything unclear (Lesson 8, Step 4).

48. What are `$ARGUMENTS`, `$0` and `$1` in a SKILL.md? `$ARGUMENTS` is everything typed after `/name`. `$0` is the first word, `$1` the second: counting starts at 0. Quotes make several words one argument (Lesson 8, Step 6).

49. Is `$0` a hack? No, it is documented ("Pass arguments to skills"). Write a backslash before the dollar sign when you mean a real price like $5 in a SKILL.md. The linter warns you (Lesson 8, Step 6; Lesson 2, Step 4).

Building a real harness

50. What are the parts of an agent harness? Exact tools, routing, running from code, rules and safety, memory, cost control, quality gates, and observability. The summary, "What you have learned", names the file for each.

51. Where do I start with a job from my work? Write and test the Python function first. Then wrap it in a skill, write five routing questions, add a hook for the rule that must never break, and measure the cost.

52. What does Claude Code add that a plain API call doesn't have? A harness around the same model: the agent loop, built-in tools, skills, context management (caching, compaction, memory, subagents), permissions and hooks, sessions, MCP, and every step visible (Lesson 6, "Why go through Claude Code at all").

53. Is there evidence that the harness matters? Yes. Anthropic runs almost all of its major agent loops on this harness. On Terminal-Bench 2.0, the same model scored between about 58% and 82%, depending only on the harness. Claude Code was not the top harness there. Its strength is that it is general, mature, and yours with Pro.

54. When is a plain LLM call the better choice? For one question and one answer. The harness adds thousands of context tokens, so `ask_claude.py` turns the tools off and uses a short system prompt (Lesson 6, Step 20).

55. Is Claude Code only a tool for writing code? No. Its defaults are made for software work, but its harness is general. Anthropic renamed the SDK to "Agent SDK", and built Cowork, its agent for non-coding work, the same way. None of this course's 9 skills writes code. The real limit: it works through files and commands on a computer.

56. Who outside Anthropic says that calling Claude Code is a good idea? Simon Willison called Claude Code a tool for general computer automation (October 2025). An independent comparison (MindStudio, May 2026) recommends the Agent SDK for agents with many steps, and the plain API for one-call tasks. That is the same rule as this course.

57. May I run these programs at work with my Pro login? For your own, individual use, yes. For a product, a service for other people, or a company's production jobs, Anthropic's terms ask for an API key (Claude Console, or Bedrock, Vertex AI or Foundry). The same programs run with `ANTHROPIC_API_KEY` set (Lesson 0, Step 8).

58. What is the difference between using Claude Code as the model and as the runtime? As the model, your program sends one question with the tools off, like `ask_claude.py`: you mostly gain the door (your Pro login). As the runtime, your program hands over a whole job, like `run_agent.py`. Then the harness runs the loop, the tools, the skills and the rules. Decide who owns the loop, so you don't build a loop inside a loop (Lesson 6, "When the harness is the right choice").

59. Who else uses an agent CLI as the runtime of a general agent? OpenClaw runs its agent turns through a local Claude Code or Gemini CLI, with its own tools passed in over MCP. Anthropic sells the harness itself as Managed Agents, and OpenAI offers its Codex agent to programs through the Codex SDK.

60. How do I use Claude Code only as the model, and should I? Switch off what it adds: `--tools ""`, `--strict-mcp-config` (it also removes your account's connector tools), `--system-prompt`, `--setting-sources ""` and `--safe-mode`. `ask_claude.py --strict` does all of it. Do it when your program owns the loop and the tools. You keep your Pro login and lose the harness. In the measured runs, Claude Code's own prompt didn't make the answers better (Lesson 6, "Using Claude Code only as the model").

61. When does Claude Code, with everything on, beat a plain model call on work that isn't coding? When the job needs more than the model's head: exact arithmetic on more than a few lines, files it must find itself, data too big to paste, several steps. On a 300-row table, it was right 6 times out of 6 against 5, at the same cost per right answer and twice as fast. On small files, it was no more right, at about 10 times the cost (Lesson 6, "Three questions").

62. How do I make Claude Code beat a plain model call? Give it a way to compute, and tell it to use it, in words that match your allow rule. One appended sentence, "compute every number with Python", made the lean profile right every time on the big table. With Python allowed but no such sentence, Claude worked in its head, or wrote `awk`, which the rule refused (Lesson 6, "Three questions").

63. How do I cut Claude Code's tokens and keep that advantage? Keep its prompt and loop, and switch off what the job doesn't use: `--strict-mcp-config`, `--safe-mode`, `--tools "Read,Bash,PowerShell"` with a python-only allow rule, `--disable-slash-commands`. That cut the cost per right answer by 70 to 80% against everything on. Send small, easy questions to a plain call (Lesson 6, "Three questions").

64. Why call Claude Code instead of the model's API at all? Three gains in one. The door: your Pro login for every model request of your programs, not only for coding. The harness: the loop and the tools make hard requests come back right. On a 3,000-row table, Claude Code was right 4 times out of 4 in about 17 seconds, and a plain call once, in about 2 minutes. The dial: switch off what a request doesn't use, from 39,143 down to 849 input tokens for one plain question (Lesson 6, "The big idea").

65. Can switching parts of Claude Code off make it worse? Yes, like cheap tires on an expensive car. Allowed only python commands, Claude Code wrote `awk`, which was refused: right 3 times out of 6. Hindsight's one-turn calls failed now and then because its tools were allowed none but still loaded. Switch off only what the request doesn't use, say in one sentence what to use instead, pin the model, and measure (Lesson 6, "The big idea").

66. Is calling Claude Code instead of the API an established practice? Yes. Anthropic runs almost all of its major agent loops on this harness. The Python Agent SDK was downloaded 29.9 million times in September 2026. OpenClaw, Hindsight, promptfoo, OpenHarness, NodeTool and a Vercel AI SDK provider plug Claude Code in where a model API would go. That doesn't prove a win on every request: measure yours (Lesson 6, "Who else builds agents this way").

67. How do I know a step of this course worked on my computer? Compare your output with the "Expected result": every one is a real run. And run the HOW TO TEST section of any `.py` file.

Appendix A - The nine SKILL.md files

Each file is shown exactly as it is on disk. The frontmatter between the --- lines is level 1 (always in context); the rest is level 2 (loaded when the skill is picked).

converting-units

.claude/skills/converting-units/SKILL.md

````
---
name: converting-units
description: Converts physical quantities between units of length, mass, volume, speed, temperature,
    and digital data size (km to miles, kg to lb, °F to °C, liters to gallons, mph to km/h, MB to
    GiB) with exact factors. Use for every unit conversion request, even simple ones like miles to
    kilometers, and whenever the user asks "how many X in Y".
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Converting units

Never do unit math in your head. Run the script; it is exact and handles aliases.

## Convert one value

```bash
python "${CLAUDE_SKILL_DIR}/scripts/convert.py" convert 100 km mi
```

Output is JSON:

```json
{"value": 100.0, "from": "km", "to": "mi", "result": 62.137119, "category": "length"}
```

Report `result` to the user, rounded sensibly (usually 2-4 significant decimals).

## List supported units

```bash
python "${CLAUDE_SKILL_DIR}/scripts/convert.py" units
```

Run this when the user names a unit you are not sure about, then pick the closest match.

## Errors

If the script prints `{"error": ...}`, read the message. It lists valid units for the category,
so fix the unit name and run again. Do not guess a conversion factor yourself.
````

analyzing-csv

.claude/skills/analyzing-csv/SKILL.md

````
---
name: analyzing-csv
description: Summarizes and aggregates tabular data in .csv files - column types, missing values,
    min/max/mean/median, group-by totals and averages, top-N rows by a column. Use when the user asks
    about the contents, statistics, totals, or rankings in a CSV file. Not for .xlsx workbooks (use
    analyzing-excel), charts (use plotting-charts), or prose .txt documents (use analyzing-text).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Analyzing CSV files

Use the bundled script instead of reading the whole CSV into context.
It uses only the Python standard library, so nothing needs installing.

## Workflow

1. **Always start with a summary** so you know the real column names and types:
   ```bash
   python "${CLAUDE_SKILL_DIR}/scripts/csv_tool.py" summary data/sales.csv
   ```
2. Then answer the question with one of:
   - Totals or averages per category:
     ```bash
     python "${CLAUDE_SKILL_DIR}/scripts/csv_tool.py" groupby data/sales.csv --by region --value
     amount --agg sum
     ```
     `--agg` is one of `sum`, `mean`, `count`, `min`, `max`.
   - Largest or smallest rows:
     ```bash
     python "${CLAUDE_SKILL_DIR}/scripts/csv_tool.py" top data/sales.csv --column amount --n 5
     ```
     Add `--asc` for the smallest.
3. Report numbers from the JSON output. Do not recompute them by hand.

Column names are case-sensitive. If you get `"error": "Unknown column"`, the error lists
the valid columns - pick the right one and run again.

For the exact JSON shape of each command, see [reference.md](reference.md).
````

analyzing-text

.claude/skills/analyzing-text/SKILL.md

````
---
name: analyzing-text
description: Measures prose in .txt or .md documents - word, sentence and paragraph counts, reading
    time, Flesch reading-ease score, average sentence length, and most frequent words. Use when the
    user asks how long, how readable, or how wordy a text, article, essay, or README is. Not for
    tables or .csv data (use analyzing-csv).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Analyzing text

## Statistics for a file

```bash
python "${CLAUDE_SKILL_DIR}/scripts/text_stats.py" stats data/article.txt
```

## Most frequent words

```bash
python "${CLAUDE_SKILL_DIR}/scripts/text_stats.py" top-words data/article.txt --n 10
```

Common English stop words ("the", "and", ...) are excluded. Use `--min-len 5` to skip short words.

## Text the user pasted into chat

Pipe it in with `-` instead of a file name:

```bash
echo "Paste the text here." | python "${CLAUDE_SKILL_DIR}/scripts/text_stats.py" stats -
```

## Reading the results

- `reading_time_min` assumes 238 words per minute (average adult silent reading speed).
- `flesch_reading_ease`: 90-100 very easy, 60-70 plain English, 30-50 difficult, below 30 very hard.
  The syllable count is a heuristic, so treat the score as approximate (about +/- 5).
- When the user wants the text improved, quote the longest sentences from `longest_sentences`.
````

calculating-dates

.claude/skills/calculating-dates/SKILL.md

````
---
name: calculating-dates
description: Calculates with calendar dates - days or weeks between two dates, adding days, adding or
    counting business days (skipping weekends and holidays), day of the week, ISO week number, and
    age from a birthdate. Use when the user asks about deadlines, due dates, "how many days until",
    working days, or what weekday a date falls on.
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: high
---

# Calculating dates

Date math is easy to get wrong by one day. Always run the script.

Each Python function in `scripts/dates.py` is a subcommand; each parameter is a `--flag`.
Dates are ISO format `YYYY-MM-DD`. If the user gives "today" or a relative date, resolve it
to an ISO date first.

| Question | Command |
|---|---|
| Days between two dates | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" days_between --start 2026-
    01-01 --end 2026-12-25` |
| Date + N days | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" add_days --date 2026-10-15 --days
    30` |
| Date + N working days | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" add_business_days --date
    2026-10-15 --days 10 --holidays 2026-11-26` |
| Working days in a range | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" business_days_between --
    start 2026-10-01 --end 2026-10-31` |
| Weekday / ISO week | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" weekday --date 2026-12-25` |
| Age | `python "${CLAUDE_SKILL_DIR}/scripts/dates.py" age --birthdate 1990-05-17` |

`--holidays` takes comma-separated ISO dates and is optional.

If you need a function that is not in the table, list them all with their docstrings:

```bash
python "${CLAUDE_SKILL_DIR}/scripts/dates.py" --list
```

Output is JSON. An `{"error": ...}` result explains which argument was wrong - fix it and rerun.
````

calculating-invoices

.claude/skills/calculating-invoices/SKILL.md

````
---
name: calculating-invoices
description: Builds and checks invoices and quotes from line items - validates the JSON, computes
    subtotal, discount, sales tax/VAT, and grand total with correct cent rounding, and renders a
    Markdown invoice. Use when the user asks to total a bill, price a quote, apply a discount or tax
    to items, or produce an invoice document.
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: high
---

# Calculating invoices

Money math must be exact (no float rounding errors), so the script uses `Decimal`.
Follow the workflow in order. Copy this checklist into your reply and tick it off:

```
- [ ] 1. Invoice JSON exists
- [ ] 2. Validation passes
- [ ] 3. Totals computed
- [ ] 4. Rendered (only if the user wants a document)
````
````

````
```

## 1. Create or locate the invoice JSON

If the user gave items in chat, write them to a file such as `invoice.json`:

```json
{
  "number": "INV-001",
  "customer": "Acme Ltd",
  "currency": "USD",
  "discount_pct": 10,
  "tax_pct": 8.25,
  "items": [
    {"description": "Consulting hours", "qty": 12, "unit_price": 95.00},
    {"description": "Travel", "qty": 1, "unit_price": 240.50}
  ]
}
```

`discount_pct` and `tax_pct` are optional (default 0). Discount applies before tax.
A sample is in `data/invoice.json`.

## 2. Validate - repeat until it passes

```bash
python "${CLAUDE_SKILL_DIR}/scripts/invoice.py" validate invoice.json
```

If it prints `"ok": false`, fix every listed problem in the JSON and validate again.
Do not compute totals from an invalid file.

## 3. Compute totals

```bash
python "${CLAUDE_SKILL_DIR}/scripts/invoice.py" total invoice.json
```

Report `subtotal`, `discount`, `tax`, and `total` exactly as printed.

## 4. Render (optional)

```bash
python "${CLAUDE_SKILL_DIR}/scripts/invoice.py" render invoice.json --out invoice.md
```
````
````

tracking-todos

.claude/skills/tracking-todos/SKILL.md

```
---
name: tracking-todos
description: Keeps a persistent to-do list for this project in data/todos.json - adds tasks with
    optional due date and priority, lists open or all tasks, and marks tasks done. Use when the user
    says "remind me to", "add a task", "what's on my list", "what's due", or "mark X as done".
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Tracking to-dos

The list lives in `data/todos.json` (relative to the project root). The script creates it if missing.
Run commands from the project root.

| Action | Command |
|---|---|
| Add | `python "${CLAUDE_SKILL_DIR}/scripts/todo.py" add "Send the Q3 report" --due 2026-10-20 --
    priority high` |
| List open tasks | `python "${CLAUDE_SKILL_DIR}/scripts/todo.py" list` |
| List everything | `python "${CLAUDE_SKILL_DIR}/scripts/todo.py" list --all` |
| Mark done | `python "${CLAUDE_SKILL_DIR}/scripts/todo.py" done 3` |

- `--priority` is `high`, `normal` (default) or `low`. `--due` is `YYYY-MM-DD`; resolve "Friday" or
  "tomorrow" to a real date first.
- To mark a task done by its text, run `list` first to find its `id`.
- `list` sorts by priority, then due date, and flags `"overdue": true`. Mention overdue tasks first.

Deleting tasks is deliberately **not** part of this skill. If the user wants the list cleared,
tell them to run `/clear-todos` themselves.
```

clear-todos

.claude/skills/clear-todos/SKILL.md

```
---
name: clear-todos
description: Deletes tasks from data/todos.json - finished tasks only by default, or every task with
    the "all" argument.
argument-hint: "[done|all]"
disable-model-invocation: true
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Clear to-dos (manual only)

Claude never loads this skill on its own (`disable-model-invocation: true`), because deleting
data should happen only when the user types `/clear-todos`.

The user's argument was: "$ARGUMENTS"

1. If the argument is `all`, run:
   ```bash
   python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --all
   ```
   Otherwise (empty or `done`), run:
   ```bash
   python "${CLAUDE_SKILL_DIR}/../tracking-todos/scripts/todo.py" clear --done-only
   ```
2. Report how many tasks were removed and how many remain.
```

analyzing-excel

.claude/skills/analyzing-excel/SKILL.md

````
---
name: analyzing-excel
description: Reads Excel workbooks (.xlsx) with pandas - lists sheets and columns, summarizes a sheet
    (types, missing values, min/max/mean/sum, date ranges), and builds pivot tables with totals (for
    example amount by region and product). Use when the user asks about the contents, sheets,
    statistics, or a pivot of an .xlsx or Excel file. Not for .csv files (use analyzing-csv) or
    charts (use plotting-charts).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: low
---

# Analyzing Excel workbooks

The script uses pandas and openpyxl from the activated virtual environment.

## Workflow

1. List the sheets and their columns first:
   ```bash
   python "${CLAUDE_SKILL_DIR}/scripts/excel_tool.py" sheets data/sales.xlsx
   ```
2. Then, depending on the question:
   - Statistics for one sheet:
     ```bash
     python "${CLAUDE_SKILL_DIR}/scripts/excel_tool.py" summary data/sales.xlsx --sheet Sales
     ```
   - Pivot table with a Total row and column (`--columns` is optional, `--agg` is sum, mean, count,
     min or max):
     ```bash
     python "${CLAUDE_SKILL_DIR}/scripts/excel_tool.py" pivot data/sales.xlsx --sheet Sales --index
     region --columns product --values amount --agg sum
     ```
3. Report the numbers from the JSON output. Do not recompute them by hand.

Sheet and column names are case-sensitive. An `{"error": ...}` result lists the valid sheets or
columns - pick the right one and run again.

## If it fails

- `"Python package ... is not installed"`: Claude Code was started without the virtual environment.
  Do not install anything. Tell the user to exit, activate `C:\pyenvsmy\claudeclioct26`, and start
  `claude` again.
````

plotting-charts

.claude/skills/plotting-charts/SKILL.md

````
---
name: plotting-charts
description: Draws bar and line charts as PNG images from .csv or .xlsx data with matplotlib - totals
    or averages per category, and values over time grouped by day, week or month. Use when the user
    asks for a chart, graph, plot, or picture of data in a CSV or Excel file. Not for numeric answers
    without a picture (use analyzing-csv or analyzing-excel).
allowed-tools:
  - Bash(python *)
  - PowerShell(python *)
effort: medium
---

# Plotting charts

The script uses pandas and matplotlib from the activated virtual environment.

## Bar chart: one value per category

```bash
python "${CLAUDE_SKILL_DIR}/scripts/chart.py" bar data/sales.csv --x region --y amount --agg sum
````
````

````
```

## Line chart: values over time

```bash
python "${CLAUDE_SKILL_DIR}/scripts/chart.py" line data/sales.csv --x date --y amount --agg sum --
    freq M
```

`--freq` groups dates by `D` (day), `W` (week) or `M` (month). Leave it out to plot each x value as
    is.

Common options: `--agg` (sum, mean, count, min, max), `--sheet NAME` for .xlsx files, `--title
    "..."`,
`--out charts/name.png`. The default output is `charts/<kind>_<y>_by_<x>.png`.

## After the chart is written

1. The JSON output has `written` (the PNG path) and `points` (the plotted numbers).
2. Open the PNG with the Read tool and check that it shows what the user asked for.
3. Tell the user the file path and the two or three most important numbers from `points`.

## If it fails

- `"Python package ... is not installed"`: Claude Code was started without the virtual environment.
  Do not install anything. Tell the user to exit, activate `C:\pyenvsmy\claudeclioct26`, and start
  `claude` again.
- An unknown column or sheet returns an error that lists the valid names. Fix the argument and rerun.
````
````

Appendix B - Python code examples

fncli.py turns typed Python functions into subcommands; dates.py is a skill script built on it; convert.py is the classic argparse pattern whose convert() function is also reused as an Agent SDK @tool; chart.py uses pip packages (pandas, matplotlib) from the skill's venv, with an import guard. The other scripts are in the project folder.

fncli.py

.claude/skills/calculating-dates/scripts/fncli.py

```
"""fncli: turns plain Python functions into a command line, so a SKILL.md can call them by name
    (Lesson 3, Pattern B).

WHAT
    run([func_a, func_b]) makes one command per function, and one --option per parameter. It prints
    each function's return value as JSON. --list prints the names, signatures and docstrings of all
    functions as JSON.

WHY
    Writing argparse code for every function is repetitive, and easy to get out of step with the
    functions. With fncli you write normal typed functions, and the command line follows them. --list
    lets Claude find a function the SKILL.md doesn't mention, without reading the code.

HOW TO USE IT  (in your own script, next to fncli.py)
    import fncli

    def add_days(date: str, days: int) -> dict:
        "Add (or subtract, if negative) calendar days to a date."
        ...

    if __name__ == "__main__":
        fncli.run([add_days])

    Then:  python dates.py add_days --date 2026-10-15 --days 30
           python dates.py --list
           python dates.py add_days --help         (the help comes from the docstring)

HOW IT WORKS
    inspect.signature() lists the parameters of each function. Each one becomes --name (underscores
    stay), converted with its type hint: int, float, str, or bool (true/false, yes/no, 1/0). A
    parameter without a default is required. A ValueError raised by your function becomes
    {"error": ..., "function": ..., "signature": ...} with exit code 1, so Claude can read what went
    wrong and try again.

NEEDS
    Only the standard library. It never calls Claude. Used by dates.py.

HOW TO TEST
    1. Unit tests: free, no Claude. In cmd, from the project folder, with the venv activated:
           python -m unittest discover -s tests -k fncli
       Expected (2 tests; the time differs):
           ..
           ----------------------------------------------------------------------
           Ran 2 tests in 0.635s

           OK
       What they check: calling a function by its name with typed options, and the JSON error with
    the
       function's signature after bad input. Add -v to see the name of each test.
    2. By hand: free.
           python .claude/skills/calculating-dates/scripts/dates.py add_days --help
               usage: dates.py add_days [-h] --date DATE --days DAYS

               Add (or subtract, if negative) calendar days to a date.

               options:
                 -h, --help   show this help message and exit
                 --date DATE
```

```
                --days DAYS
              (fncli made this help from the function's parameters and its docstring.)
    3. With Claude: uses a little Pro allowance.
           See dates.py: every Claude run of that skill goes through fncli.

IF SOMETHING GOES WRONG
    - A parameter without a type hint, or with a type other than int, float, str or bool, arrives as
    text.
    - Only ValueError becomes a JSON error. Any other exception stops with a traceback, so check the
    input
      and raise ValueError with a clear message.

FILES IT READS AND WRITES
    None.

SEE ALSO
    dates.py (uses it); Lesson 3, Steps B2 and B4.
"""

import argparse
import inspect
import json
import sys
from typing import Callable

_PARSERS = {int: int, float: float, str: str}


def _parse_bool(text: str) -> bool:
    if text.lower() in ("1", "true", "yes", "y", "on"):
        return True
    if text.lower() in ("0", "false", "no", "n", "off"):
        return False
    raise argparse.ArgumentTypeError(f"expected true/false, got '{text}'")


def describe(functions: list[Callable]) -> list[dict]:
    return [
        {"name": fn.__name__, "signature": f"{fn.__name__}{inspect.signature(fn)}", "doc":
    inspect.getdoc(fn) or ""}
        for fn in functions
    ]


def run(functions: list[Callable], argv: list[str] | None = None) -> None:
    try:
        sys.stdout.reconfigure(encoding="utf-8")
    except AttributeError:
        pass

    parser = argparse.ArgumentParser(description=inspect.getdoc(sys.modules["__main__"]) or None)
    parser.add_argument("--list", action="store_true", help="list functions as JSON and exit")
    sub = parser.add_subparsers(dest="function")

    by_name = {fn.__name__: fn for fn in functions}
    for fn in functions:
        doc = inspect.getdoc(fn) or ""
        cmd = sub.add_parser(fn.__name__, help=doc.splitlines()[0] if doc else None, description=doc)
        for param in inspect.signature(fn).parameters.values():
            annotation = param.annotation if param.annotation is not inspect.Parameter.empty else str
            convert = _parse_bool if annotation is bool else _PARSERS.get(annotation, str)
            flag = "--" + param.name
            if param.default is inspect.Parameter.empty:
                cmd.add_argument(flag, dest=param.name, type=convert, required=True)
            else:
                cmd.add_argument(flag, dest=param.name, type=convert, default=param.default)

    args = parser.parse_args(argv)
    if args.list or not args.function:
        print(json.dumps(describe(functions), indent=2, ensure_ascii=False))
        return

    fn = by_name[args.function]
```

```
    kwargs = {name: value for name, value in vars(args).items() if name not in ("list", "function")}
    try:
        result = fn(**kwargs)
    except ValueError as err:
        print(json.dumps({"error": str(err), "function": fn.__name__, "signature":
    describe([fn])[0]["signature"]}))
        sys.exit(1)
    print(json.dumps(result, ensure_ascii=False, default=str))
```

dates.py

.claude/skills/calculating-dates/scripts/dates.py

```
"""Days between dates, working days, weekdays and ages. The script of the calculating-dates skill
    (Lesson 3, Pattern B).

WHAT
    Six functions, each one a command (Python names with underscores, parameters as --options):
        days_between --start DATE --end DATE                        calendar days from start to end
        add_days --date DATE --days N                               a date plus (or minus) N days
        add_business_days --date DATE --days N [--holidays LIST]    plus N working days (Monday to
                                                                    Friday, without the holidays)
        business_days_between --start DATE --end DATE [--holidays LIST]   working days in a range
        weekday --date DATE                                         the day of the week and ISO week
        age --birthdate DATE [--on DATE]                            age in years, today or on a date
    Dates are YYYY-MM-DD; LIST is comma-separated dates, for example 2026-11-26,2026-12-25.

WHY
    "What date is 10 working days after October 15?" is easy to get wrong in your head, and for a
    language model too. Python's datetime module gets it right.

HOW CLAUDE USES IT
    The calculating-dates SKILL.md has a table with one command per function, and tells Claude to run
    --list when it needs something else. Claude reads only the JSON that the script prints.

HOW TO RUN IT YOURSELF  (in cmd, from the project folder)
    python .claude/skills/calculating-dates/scripts/dates.py --list
    python .claude/skills/calculating-dates/scripts/dates.py add_business_days --date 2026-10-15 --
    days 10
        {"date": "2026-10-15", "business_days": 10, "result": "2026-10-29", "weekday": "Thursday"}
    python .claude/skills/calculating-dates/scripts/dates.py weekday --date 2026-12-25
        {"date": "2026-12-25", "weekday": "Friday", "iso_week": 52}
    python .claude/skills/calculating-dates/scripts/dates.py add_business_days --help

ERRORS
    A wrong date prints {"error": ..., "function": ..., "signature": ...} with exit code 1:
        {"error": "date must be an ISO date like 2026-10-15, got '2026-02-30'", "function":
    "add_days", ...}

HOW IT WORKS
    The functions are plain Python with type hints and docstrings. The last lines hand them to
    fncli.run() (fncli.py, in this folder), which builds the command line from them: no argparse code
    here. To add a command, write a function and add it to the PUBLIC list (Lesson 3, Step B4).
    The same functions are offered to Claude as tools in agent/custom_tools_agent.py (Lesson 6, Part
    2).

NEEDS
    Only the standard library and fncli.py. It never calls Claude.

HOW TO TEST
    1. Unit tests: free, no Claude. In cmd, from the project folder, with the venv activated:
           python -m unittest discover -s tests -k CalculatingDates
       Expected (5 tests; the time differs):
           .....
           ----------------------------------------------------------------------
           Ran 5 tests in 0.608s

           OK
       What they check: working days that skip weekends and holidays, working days in a range, ages
       before and after a birthday, and calling functions by name through fncli. Add -v to see the
    name
       of each test.
    2. By hand: free.
```

```
           python .claude/skills/calculating-dates/scripts/dates.py days_between --start 2026-01-01 -
    -end 2026-12-25
               {"start": "2026-01-01", "end": "2026-12-25", "days": 358, "weeks": 51.14}
           python .claude/skills/calculating-dates/scripts/dates.py add_days --date 2026-10-15
               usage: dates.py add_days [-h] --date DATE --days DAYS
               dates.py add_days: error: the following arguments are required: --days       (exit
    code 2)
    3. With Claude: uses a little Pro allowance.
           claude -p "How many days are there from 2026-01-01 to 2026-12-25?" --model haiku --
    allowedTools "Skill,Read,Bash(python *),PowerShell(python *)"
               Expected: 358 days.

IF SOMETHING GOES WRONG
    - {"error": "date must be an ISO date like 2026-10-15, got ...", ...}: dates are YYYY-MM-DD.
    - The option names are the parameter names, with underscores: --birthdate, --start, --end.
    - Your new function is not a command: add it to the PUBLIC list at the end of the file.

FILES IT READS AND WRITES
    None.

SEE ALSO
    fncli.py (builds the command line); calculating-dates/SKILL.md; Lesson 3, Pattern B (Steps B1 to
    B4).
"""

import datetime as dt
from datetime import timedelta

import fncli

WEEKDAY_NAMES = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]


def _parse(text: str, field: str) -> dt.date:
    try:
        return dt.date.fromisoformat(text.strip())
    except ValueError:
        raise ValueError(f"{field} must be an ISO date like 2026-10-15, got '{text}'") from None


def _holidays(text: str) -> set[dt.date]:
    return {_parse(item, "holidays") for item in text.split(",") if item.strip()}


def _is_business_day(day: dt.date, holidays: set[dt.date]) -> bool:
    return day.weekday() < 5 and day not in holidays


def days_between(start: str, end: str) -> dict:
    """Calendar days from start to end (negative if end is earlier). Dates are YYYY-MM-DD."""
    d1, d2 = _parse(start, "start"), _parse(end, "end")
    delta = (d2 - d1).days
    return {"start": d1, "end": d2, "days": delta, "weeks": round(delta / 7, 2)}


def add_days(date: str, days: int) -> dict:
    """Add (or subtract, if negative) calendar days to a date."""
    base = _parse(date, "date")
    result = base + timedelta(days=days)
    return {"date": base, "days": days, "result": result, "weekday": WEEKDAY_NAMES[result.weekday()]}


def add_business_days(date: str, days: int, holidays: str = "") -> dict:
    """Add business days (Mon-Fri), skipping comma-separated holiday dates. Days must be >= 0."""
    if days < 0:
        raise ValueError("days must be zero or positive for add_business_days")
    base, skip = _parse(date, "date"), _holidays(holidays)
    current, remaining = base, days
    while remaining > 0:
        current += timedelta(days=1)
        if _is_business_day(current, skip):
            remaining -= 1
```

```
    return {"date": base, "business_days": days, "result": current, "weekday":
    WEEKDAY_NAMES[current.weekday()]}


def business_days_between(start: str, end: str, holidays: str = "") -> dict:
    """Count business days after start up to and including end, skipping weekends and holidays."""
    d1, d2, skip = _parse(start, "start"), _parse(end, "end"), _holidays(holidays)
    if d2 < d1:
        raise ValueError("end must be on or after start")
    count = sum(1 for offset in range(1, (d2 - d1).days + 1) if _is_business_day(d1 +
    timedelta(days=offset), skip))
    return {"start": d1, "end": d2, "business_days": count}


def weekday(date: str) -> dict:
    """Day of the week and ISO week number for a date."""
    day = _parse(date, "date")
    return {"date": day, "weekday": WEEKDAY_NAMES[day.weekday()], "iso_week": day.isocalendar().week}


def age(birthdate: str, on: str = "") -> dict:
    """Age in whole years on a given date (default: today)."""
    born = _parse(birthdate, "birthdate")
    today = _parse(on, "on") if on else dt.date.today()
    if today < born:
        raise ValueError("'on' date is before the birthdate")
    years = today.year - born.year - ((today.month, today.day) < (born.month, born.day))
    return {"birthdate": born, "on": today, "years": years}


PUBLIC = [days_between, add_days, add_business_days, business_days_between, weekday, age]

if __name__ == "__main__":
    fncli.run(PUBLIC)
```

convert.py

.claude/skills/converting-units/scripts/convert.py

```
"""Converts values between units with exact factors. The script of the converting-units skill
    (Lessons 1 and 3).

WHAT
    Two commands:
        convert VALUE FROM TO   one conversion: length, mass, volume, speed, temperature or data size
        units                   the supported units, by category
    Common other names are accepted too: miles, kilometres, lbs, celsius, fahrenheit, kph, ...

WHY
    A language model can get a conversion slightly wrong; this script uses exact factors (one mile is
    1609.344 metres) and Python's arithmetic. The skill tells Claude to run it for every conversion.

HOW CLAUDE USES IT
    The converting-units SKILL.md tells Claude to run
        python "${CLAUDE_SKILL_DIR}/scripts/convert.py" convert 100 km mi
    Claude reads only the one line of JSON that the script prints, never this file.

HOW TO RUN IT YOURSELF  (in cmd, from the project folder)
    python .claude/skills/converting-units/scripts/convert.py convert 100 km mi
        {"value": 100.0, "from": "km", "to": "mi", "result": 62.137119, "category": "length"}
    python .claude/skills/converting-units/scripts/convert.py units
        {"length": ["mm", "cm", "m", "km", "in", "ft", "yd", "mi", "nmi"], "mass": [...], ...}
    python .claude/skills/converting-units/scripts/convert.py --help

ERRORS
    A wrong unit or category prints {"error": ..., "hint": ...} and ends with exit code 1, for
    example
        {"error": "Cannot convert length (km) to mass (kg).", "hint": "Valid length units: mm, cm,
    ..."}
    The hint tells Claude how to correct its command and try again.

HOW IT WORKS
```

```
    Each unit of a category has a factor to one base unit (metres, kilograms, litres, ...): the value
    is
    multiplied by the first factor and divided by the second. Temperatures go through kelvin, because
    they also have an offset. ALIASES maps the other names to the short ones.

ALSO USED BY
    tests/test_skill_scripts.py, which imports convert() directly (Lesson 5), and
    agent/custom_tools_agent.py, which offers convert() to Claude as a tool (Lesson 6, Part 2).

NEEDS
    Only the standard library. It never calls Claude.

HOW TO TEST
    1. Unit tests: free, no Claude. In cmd, from the project folder, with the venv activated:
           python -m unittest discover -s tests -k ConvertingUnits
       Expected (5 tests; the time differs):
           .....
           ----------------------------------------------------------------------
           Ran 5 tests in 0.328s

           OK
       What they check: exact factors for lengths, temperatures through kelvin, binary data units
    (GiB),
       error hints, and the command itself. Add -v to see the name of each test.
    2. By hand: free.
           python .claude/skills/converting-units/scripts/convert.py convert 1 mi km
               {"value": 1.0, "from": "mi", "to": "km", "result": 1.609344, "category": "length"}
           python .claude/skills/converting-units/scripts/convert.py convert 5 km kg
               {"error": "Cannot convert length (km) to mass (kg).", "hint": "Valid length units: mm,
    cm, m, km, in, ft, yd, mi, nmi"}
           echo %errorlevel%
               1          (0 after a good conversion, 1 after an error)
    3. With Claude: uses a little Pro allowance.
           claude -p "Convert 1 mile to km." --model haiku --allowedTools "Skill,Read,Bash(python
    *),PowerShell(python *)"
               Expected: an answer with 1.609344 km. A real run: "1 mile = **1.609 km** (exactly
    1.609344 km)."

IF SOMETHING GOES WRONG
    - {"error": "Unknown unit 'parsec'.", "hint": "Run the 'units' command to see valid units."}: the
    unit
      is not in LINEAR_UNITS or ALIASES. To accept a new name, add it to ALIASES.
    - "python" is not recognized, or the Microsoft Store opens: activate the venv first (Lesson 0).
    - A test fails after you changed a factor: Lesson 5, Step 2 shows how to read the failure.

FILES IT READS AND WRITES
    None: it reads only its arguments, and writes only to the screen (stdout).

SEE ALSO
    converting-units/SKILL.md (how Claude is told to run this script); tests/test_skill_scripts.py,
    class ConvertingUnits; agent/custom_tools_agent.py and agent/tutor_mcp_server.py (convert() as a
    tool); Lesson 1, Step 4; Lesson 3, Part 1; Lesson 5, Step 2.
"""

import argparse
import json
import sys

# Every linear unit is stored as "how many base units is 1 of this unit".
# Base units: meter, kilogram, liter, meter/second, byte.
LINEAR_UNITS = {
    "length": {
        "mm": 0.001, "cm": 0.01, "m": 1.0, "km": 1000.0,
        "in": 0.0254, "ft": 0.3048, "yd": 0.9144, "mi": 1609.344, "nmi": 1852.0,
    },
    "mass": {
        "mg": 1e-6, "g": 0.001, "kg": 1.0, "t": 1000.0,
        "oz": 0.028349523125, "lb": 0.45359237, "st": 6.35029318,
    },
    "volume": {
        "ml": 0.001, "l": 1.0, "m3": 1000.0,
        "tsp": 0.00492892159375, "tbsp": 0.01478676478125, "cup": 0.2365882365,
        "pt": 0.473176473, "qt": 0.946352946, "gal": 3.785411784, "floz": 0.0295735295625,
```

```
    },
    "speed": {
        "m/s": 1.0, "km/h": 1000.0 / 3600.0, "mph": 1609.344 / 3600.0, "kn": 1852.0 / 3600.0,
    },
    "data": {
        "b": 1.0, "kb": 1e3, "mb": 1e6, "gb": 1e9, "tb": 1e12,
        "kib": 1024.0, "mib": 1024.0**2, "gib": 1024.0**3, "tib": 1024.0**4,
    },
}

TEMPERATURE_UNITS = ("c", "f", "k")

# Words people actually type -> canonical unit key.
ALIASES = {
    "meter": "m", "meters": "m", "metre": "m", "metres": "m",
    "kilometer": "km", "kilometers": "km", "kilometre": "km", "kilometres": "km",
    "centimeter": "cm", "centimeters": "cm", "millimeter": "mm", "millimeters": "mm",
    "inch": "in", "inches": "in", "foot": "ft", "feet": "ft", "yard": "yd", "yards": "yd",
    "mile": "mi", "miles": "mi",
    "gram": "g", "grams": "g", "kilogram": "kg", "kilograms": "kg", "kgs": "kg",
    "tonne": "t", "tonnes": "t", "ounce": "oz", "ounces": "oz",
    "pound": "lb", "pounds": "lb", "lbs": "lb", "stone": "st",
    "liter": "l", "liters": "l", "litre": "l", "litres": "l",
    "milliliter": "ml", "milliliters": "ml", "gallon": "gal", "gallons": "gal",
    "cups": "cup", "pint": "pt", "pints": "pt", "quart": "qt", "quarts": "qt",
    "kph": "km/h", "kmh": "km/h", "knot": "kn", "knots": "kn", "mps": "m/s",
    "celsius": "c", "°c": "c", "fahrenheit": "f", "°f": "f", "kelvin": "k",
    "byte": "b", "bytes": "b",
}


def normalize(unit: str) -> str:
    u = unit.strip().lower()
    return ALIASES.get(u, u)


def category_of(unit: str) -> str | None:
    if unit in TEMPERATURE_UNITS:
        return "temperature"
    for category, table in LINEAR_UNITS.items():
        if unit in table:
            return category
    return None


def _to_kelvin(value: float, unit: str) -> float:
    if unit == "c":
        return value + 273.15
    if unit == "f":
        return (value - 32.0) * 5.0 / 9.0 + 273.15
    return value


def _from_kelvin(value: float, unit: str) -> float:
    if unit == "c":
        return value - 273.15
    if unit == "f":
        return (value - 273.15) * 9.0 / 5.0 + 32.0
    return value


def convert(value: float, from_unit: str, to_unit: str) -> dict:
    """Convert `value` from `from_unit` to `to_unit`. Returns a JSON-ready dict.

    Errors are returned as {"error": "..."} with a hint, never raised, so the
    agent can read the message and retry ("solve, don't punt").
    """
    src, dst = normalize(from_unit), normalize(to_unit)
    src_cat, dst_cat = category_of(src), category_of(dst)

    if src_cat is None or dst_cat is None:
        bad = from_unit if src_cat is None else to_unit
```

```
        return {"error": f"Unknown unit '{bad}'.", "hint": "Run the 'units' command to see valid
    units."}
    if src_cat != dst_cat:
        return {
            "error": f"Cannot convert {src_cat} ({from_unit}) to {dst_cat} ({to_unit}).",
            "hint": f"Valid {src_cat} units: {', '.join(list_units()[src_cat])}",
        }

    if src_cat == "temperature":
        result = _from_kelvin(_to_kelvin(value, src), dst)
    else:
        table = LINEAR_UNITS[src_cat]
        result = value * table[src] / table[dst]

    return {"value": value, "from": src, "to": dst, "result": round(result, 6), "category": src_cat}


def list_units() -> dict:
    units = {category: list(table) for category, table in LINEAR_UNITS.items()}
    units["temperature"] = list(TEMPERATURE_UNITS)
    return units


def _print_json(data) -> None:
    print(json.dumps(data, ensure_ascii=False))


def main(argv: list[str] | None = None) -> int:
    try:
        sys.stdout.reconfigure(encoding="utf-8")  # Windows consoles default to cp1252
    except AttributeError:
        pass

    parser = argparse.ArgumentParser(description="Convert between units.")
    sub = parser.add_subparsers(dest="command", required=True)

    p_convert = sub.add_parser("convert", help="convert VALUE FROM TO")
    p_convert.add_argument("value", type=float)
    p_convert.add_argument("from_unit")
    p_convert.add_argument("to_unit")

    sub.add_parser("units", help="list supported units by category")

    args = parser.parse_args(argv)
    if args.command == "convert":
        result = convert(args.value, args.from_unit, args.to_unit)
        _print_json(result)
        return 1 if "error" in result else 0
    _print_json(list_units())
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

chart.py

.claude/skills/plotting-charts/scripts/chart.py

```
"""Bar and line charts as PNG pictures, from .csv or .xlsx data. The script of the plotting-charts
    skill (Lesson 7).

WHAT
    Three commands:
        check                                                   which Python runs the script, and the
                                                                pandas and matplotlib versions
        bar FILE --x COL --y COL [--agg sum|mean|count|min|max] [--sheet NAME] [--title T] [--out
    PNG]
                                                                one bar per value of --x
        line FILE --x COL --y COL [--agg ...] [--freq D|W|M] [--sheet NAME] [--title T] [--out PNG]
                                                                values over time; --freq groups the
                                                                dates by day, week or month
    Without --out the picture goes to charts/<kind>_<y>_by_<x>.png. --sheet is for .xlsx files.
```

```
WHY
    Claude can't draw a picture by itself. matplotlib can, and the script also prints the numbers of
    each bar or point, so Claude can describe the chart without seeing it.

HOW CLAUDE USES IT
    The plotting-charts SKILL.md tells Claude to run bar or line, and to tell you where the PNG file
    is.
    Claude runs plain `python`, so start claude from a cmd window where the venv is activated.

HOW TO RUN IT YOURSELF  (in cmd, from the project folder, with the venv activated)
    python .claude/skills/plotting-charts/scripts/chart.py check
    python .claude/skills/plotting-charts/scripts/chart.py bar data/sales.csv --x region --y amount
        {"written": "charts/bar_amount_by_region.png", "kind": "bar", "x": "region", "y": "amount",
         "agg": "sum", "points": [{"region": "North", "amount": 2150.0}, ...]}
    python .claude/skills/plotting-charts/scripts/chart.py line data/sales.xlsx --sheet Sales --x
    date --y amount --freq M
    start charts/bar_amount_by_region.png          (opens the picture)

ERRORS
    - A wrong file, sheet or column name prints {"error": ...} with exit code 1.
    - Without the packages (the venv was not activated) it prints an error and a hint with exit code
    2,
      the same as excel_tool.py.

HOW IT WORKS
    pandas groups and aggregates the data; matplotlib draws it with the "Agg" backend, which writes
    files and never opens a window, so it works when Claude runs it.

NEEDS
    pandas, openpyxl and matplotlib, listed in requirements.txt next to this file. Install them once:
        python -m pip install -r .claude/skills/plotting-charts/scripts/requirements.txt
    It never calls Claude.

HOW TO TEST
    1. Unit tests: free, no Claude. In cmd, from the project folder, with the venv activated:
           python -m unittest discover -s tests -k chart -k WrongInterpreter
       Expected (3 tests; the time differs):
           ...
           ----------------------------------------------------------------------
           Ran 3 tests in 6.417s

           OK
       What they check: a bar chart that is a real PNG file, a line chart by month from the .xlsx
    file,
       and the helpful message when the packages are missing. Add -v to see the name of each test.
    2. By hand: free.
           python .claude/skills/plotting-charts/scripts/chart.py bar data/sales.csv --x region --y
    amount --out charts/test_bar.png
               {"written": "charts/test_bar.png", "kind": "bar", "x": "region", "y": "amount", "agg":
    "sum",
                "points": [{"region": "North", "amount": 2150.0}, {"region": "East", "amount":
    910.0}, ...]}
           start charts/test_bar.png              (look at it; then delete charts/test_bar.png)
           python .claude/skills/plotting-charts/scripts/chart.py bar data/sales.csv --x Region --y
    amount
               {"error": "Unknown column 'Region'.", "columns": ["date", "region", "product",
    "units", "amount"]}     (exit code 1)
    3. With Claude: uses a little Pro allowance.
           claude -p "Make a bar chart of the total amount per region from data/sales.csv." --model
    haiku --allowedTools "Skill,Read,Bash(python *),PowerShell(python *)"
               Expected: the file charts/bar_amount_by_region.png, and the totals North 2,150, East
    910, West 900,
               South 670 in the answer.

IF SOMETHING GOES WRONG
    - Exit code 2 and "is not installed in ...": the venv is not active (same as excel_tool.py).
    - The picture is not in charts/: charts/ is relative to the current folder; the "written" field
    shows
      where the file went.

FILES IT READS AND WRITES
    Reads the .csv or .xlsx file. Writes the PNG file (charts/ is ignored by git).
```

```
SEE ALSO
    requirements.txt next to this file; plotting-charts/SKILL.md; Lesson 3, Step E3; Lesson 7, Step
    5.
"""

import argparse
import json
import sys
from pathlib import Path

REQUIREMENTS = Path(__file__).with_name("requirements.txt")

try:
    import matplotlib

    matplotlib.use("Agg")  # draw to files only; no window (Claude runs this without a screen)
    import matplotlib.pyplot as plt
    import pandas as pd
except ImportError as missing:
    # Wrong interpreter (or packages not installed yet). Say exactly how to fix the call.
    print(json.dumps({
        "error": f"Python package '{missing.name}' is not installed in {sys.executable}",
        "hint": "Claude Code was started without the skills' virtual environment. Activate the venv,
    "
                f"then start claude again. Missing packages: python -m pip install -r
    \"{REQUIREMENTS}\"",
    }))
    sys.exit(2)

AGGREGATIONS = ("sum", "mean", "count", "min", "max")
# pandas resample codes: day, week (ending Sunday), month (end of month).
FREQUENCIES = {"D": "D", "W": "W", "M": "ME"}
# 8 x 4.5 inches at 120 dpi = 960 x 540 px: readable on screen, small file.
FIGSIZE, DPI = (8, 4.5), 120


class ChartError(Exception):
    def __init__(self, message: str, **extra):
        super().__init__(message)
        self.payload = {"error": message, **extra}


def load(path: str, sheet: str | None = None) -> "pd.DataFrame":
    file = Path(path)
    if not file.exists():
        raise ChartError(f"File not found: {path}", hint="Check the path relative to the current
    directory.")
    if file.suffix.lower() in (".xlsx", ".xlsm"):
        book = pd.ExcelFile(file, engine="openpyxl")
        if sheet is not None and sheet not in book.sheet_names:
            raise ChartError(f"Unknown sheet '{sheet}'.", sheets=book.sheet_names)
        return book.parse(sheet if sheet is not None else book.sheet_names[0])
    return pd.read_csv(file, encoding="utf-8-sig")


def _require(df: "pd.DataFrame", *columns: str) -> None:
    for column in columns:
        if column not in df.columns:
            raise ChartError(f"Unknown column '{column}'.", columns=[str(c) for c in df.columns])


LABEL_FORMATS = {"M": "%Y-%m", "W": "week to %Y-%m-%d", "D": "%Y-%m-%d"}


def _label(value, freq: str | None = None) -> str:
    if isinstance(value, pd.Timestamp):
        return value.strftime(LABEL_FORMATS.get(freq, "%Y-%m-%d"))
    return str(value)


def _points(series: "pd.Series", x: str, y: str, freq: str | None = None) -> list[dict]:
    return [{x: _label(k, freq), y: round(float(v), 4)} for k, v in series.items()]
```

```
def _save(fig, out: Path) -> str:
    out.parent.mkdir(parents=True, exist_ok=True)
    fig.tight_layout()
    fig.savefig(out, dpi=DPI)
    plt.close(fig)
    return out.as_posix()


def bar(path: str, x: str, y: str, agg: str = "sum", sheet: str | None = None,
        title: str | None = None, out: str | None = None) -> dict:
    df = load(path, sheet)
    _require(df, x, y)
    series = df.groupby(x)[y].agg(agg).sort_values(ascending=False)
    fig, ax = plt.subplots(figsize=FIGSIZE)
    ax.bar([str(k) for k in series.index], series.values, color="#4C72B0")
    for i, v in enumerate(series.values):
        ax.annotate(f"{v:,.0f}", (i, v), ha="center", va="bottom", fontsize=9)
    ax.set_title(title or f"{agg} of {y} by {x}")
    ax.set_xlabel(x)
    ax.set_ylabel(f"{y} ({agg})")
    ax.spines[["top", "right"]].set_visible(False)
    written = _save(fig, Path(out or f"charts/bar_{y}_by_{x}.png"))
    return {"written": written, "kind": "bar", "x": x, "y": y, "agg": agg, "points": _points(series,
    x, y)}


def line(path: str, x: str, y: str, agg: str = "sum", freq: str | None = None, sheet: str | None =
    None,
         title: str | None = None, out: str | None = None) -> dict:
    df = load(path, sheet)
    _require(df, x, y)
    if freq:
        if freq not in FREQUENCIES:
            raise ChartError(f"Unknown --freq '{freq}'.", valid=list(FREQUENCIES))
        dates = pd.to_datetime(df[x], errors="coerce")
        if dates.isna().all():
            raise ChartError(f"Column '{x}' does not contain dates, so --freq cannot be used.")
        series = df.assign(**{x: dates}).set_index(x)[y].resample(FREQUENCIES[freq]).agg(agg)
    else:
        series = df.groupby(x)[y].agg(agg).sort_index()
    labels = [_label(k, freq) for k in series.index]
    fig, ax = plt.subplots(figsize=FIGSIZE)
    # Plot against positions with text labels, so each point sits exactly on its period's label.
    ax.plot(range(len(series)), series.values, marker="o", color="#4C72B0")
    ax.set_xticks(range(len(series)), labels)
    for i, v in enumerate(series.values):
        ax.annotate(f"{v:,.0f}", (i, v), textcoords="offset points", xytext=(0, 6), ha="center",
    fontsize=9)
    period = {"D": "day", "W": "week", "M": "month"}.get(freq)
    ax.set_title(title or f"{agg} of {y} per {period}" if period else title or f"{agg} of {y} by
    {x}")
    ax.set_xlabel(x)
    ax.set_ylabel(f"{y} ({agg})")
    ax.grid(alpha=0.3)
    ax.spines[["top", "right"]].set_visible(False)
    fig.autofmt_xdate()
    written = _save(fig, Path(out or f"charts/line_{y}_by_{x}.png"))
    return {"written": written, "kind": "line", "x": x, "y": y, "agg": agg, "freq": freq,
            "points": _points(series, x, y, freq)}


def check() -> dict:
    return {"python": sys.executable, "pandas": pd.__version__, "matplotlib": matplotlib.__version__}


def main(argv: list[str] | None = None) -> int:
    try:
        sys.stdout.reconfigure(encoding="utf-8")
    except AttributeError:
        pass
```

```
    parser = argparse.ArgumentParser(description="Draw a chart from a .csv or .xlsx file.")
    sub = parser.add_subparsers(dest="command", required=True)
    sub.add_parser("check", help="show the interpreter and package versions")
    for kind in ("bar", "line"):
        p = sub.add_parser(kind)
        p.add_argument("file")
        p.add_argument("--x", required=True)
        p.add_argument("--y", required=True)
        p.add_argument("--agg", default="sum", choices=AGGREGATIONS)
        p.add_argument("--sheet")
        p.add_argument("--title")
        p.add_argument("--out", help="PNG path (default: charts/<kind>_<y>_by_<x>.png)")
        if kind == "line":
            p.add_argument("--freq", choices=list(FREQUENCIES), help="group dates by day, week or
    month")

    args = parser.parse_args(argv)
    try:
        if args.command == "check":
            result = check()
        elif args.command == "bar":
            result = bar(args.file, args.x, args.y, args.agg, args.sheet, args.title, args.out)
        else:
            result = line(args.file, args.x, args.y, args.agg, args.freq, args.sheet, args.title,
    args.out)
    except ChartError as err:
        print(json.dumps(err.payload, ensure_ascii=False))
        return 1
    print(json.dumps(result, ensure_ascii=False))
    return 0


if __name__ == "__main__":
    sys.exit(main())
```